

Design and Implementation of a MIPI CSI-2 compatible receiver and pre- processor for low-resolution image processing

**Masterthesis im Studiengang Energiesysteme an der Fachhochschule
Dortmund**

Vorgelegt von: Sven Krause

Prüfer/in: Prof. Dr.-Ing. Michael Karagounis

Datum: 21.07.2025

Abstract

In this thesis, a MIPI CSI-2 compatible receiver for use with the GateMate FPGA by Cologne Chip AG was designed using the Verilog hardware description language and tested. Additionally, a demonstrator system, capable of pre-processing and forwarding the received data to a display via DVI, was developed to facilitate the design and testing of the receiver. This demonstrator system includes a dedicated evaluation board for the GateMate FPGA that was designed and assembled as a part of this work. A camera module that uses a CSI-2 interface for transmitting video data was used as a real data source to verify the function of the receiver and demonstrator.

The key challenge was the handling of the D-PHY physical layer used by the CSI-2 interface with generic input buffers. To achieve this, some of the specified functionality of CSI-2 and the D-PHY physical layer, which is not necessary for video data reception, was omitted from the designed receiver, resulting in a compact design that does not require workarounds like doubling the number of I/O pins used for each “lane” of the interface.

The results of initial tests show that the receiver manages to reliably receive video data from the camera module over several hours without any significant errors occurring. However, no in-depth characterization of the system’s performance could be done within the timeframe of this project.

Kurzzusammenfassung

In dieser Thesis wurde, unter Verwendung der Hardwarebeschreibungssprache Verilog, ein MIPI CSI-2 kompatibler Empfänger für die Nutzung im GateMate FPGA der Cologne Chip AG entwickelt und getestet. Zusätzlich wurde ein Demonstrator entwickelt, der die empfangenen Daten vorverarbeiten und über DVI an einen Bildschirm weiterleiten kann. Dieser Demonstrator beinhaltet ein dediziertes Evaluation Board für den GateMate FPGA, das im Rahmen dieser Arbeit entworfen und aufgebaut wurde. Ein Kameramodul mit CSI-2 Schnittstelle wurde als Quelle für reale Videodaten für die Verifikation von Empfänger und Demonstrator verwendet.

Die zentrale Herausforderung bestand in der Handhabung des D-PHY Physical Layers, das von der CSI-2 Schnittstelle verwendet wird, mit generischen Input Buffern. Dazu wurden Teile der spezifizierten Funktionalität von CSI-2 und D-PHY im entworfenen Empfänger nicht umgesetzt, wodurch ein kompaktes Design erzielt werden konnte, das ohne Behelfsmaßnahmen wie die Verdopplung der für jede „Lane“ der Schnittstelle verwendeten I/O-Pins auskommt.

Die Ergebnisse der ersten Tests zeigen, dass der Empfänger zuverlässig, über mehrere Stunden Videodaten vom Kameramodul empfangen kann, ohne dass signifikante Fehler auftreten. Eine umfangreiche Charakterisierung des Systems konnte allerdings im Rahmen dieses Projektes nicht durchgeführt werden.

Selbstständigkeitserklärung

Hiermit versichere ich, dass die von mir vorgelegte Prüfungsleistung selbständig und ohne unzulässige fremde Hilfe erstellt worden ist. Alle verwendeten Quellen sind in der Arbeit so aufgeführt, dass Art und Umfang der Verwendung nachvollziehbar sind.

Table of Contents

List of Figures	- 6 -
List of Tables	- 9 -
Listings	- 10 -
List of Abbreviations	- 11 -
1 Introduction	- 12 -
2 Theory and Components	- 13 -
2.1 Digital Video Basics.....	- 13 -
2.2 CSI-2 and D-PHY.....	- 16 -
2.2.1 Basics	- 16 -
2.2.2 Low-Level Protocol.....	- 18 -
2.2.3 Packet Structure	- 18 -
2.2.4 Error Correction Code and Checksum.....	- 19 -
2.2.5 Lane Distribution	- 22 -
2.2.6 Data Formats	- 24 -
2.2.7 D-PHY Architecture	- 25 -
2.2.8 D-PHY Signal Levels	- 26 -
2.2.9 D-PHY Operating Modes.....	- 30 -
2.2.10 High-Speed Clock Lane Operation	- 33 -
2.2.11 D-PHY Timing.....	- 33 -
2.3 HDMI and DVI	- 36 -
2.3.1 TMDS General Operation	- 36 -
2.3.2 TMDS Encoding.....	- 37 -
2.3.3 TMDS Physical Layer	- 40 -
2.3.4 Transmitter Specification.....	- 41 -
2.3.5 Receiver Specification.....	- 42 -
2.4 Camera Module	- 43 -
2.4.1 Digital Camera Basics	- 43 -
2.4.2 Raspberry Pi Camera Module V2.....	- 44 -
2.5 GateMate FPGA	- 46 -
3 Evaluation Board	- 51 -
3.1 Requirements and design constraints	- 51 -
3.2 Schematic Design	- 52 -
3.2.1 I/O-Banks	- 52 -
3.2.2 Power Supply	- 54 -
3.2.3 Camera Connection	- 56 -

3.2.4	HDMI Output.....	- 57 -
3.2.5	Miscellaneous Components	- 58 -
3.3	PCB Layout	- 60 -
3.3.1	Layer Stack and board outline.....	- 60 -
3.3.2	Component Placement	- 60 -
3.3.3	Escape Routing	- 61 -
3.3.4	Camera and HDMI Connections	- 62 -
3.3.5	Power Supply	- 63 -
3.3.6	GPIO and Other.....	- 65 -
3.4	Assembly.....	- 66 -
3.5	Initial Testing.....	- 66 -
4	Design and Implementation.....	- 68 -
4.1	Demonstrator Concept	- 68 -
4.2	D-PHY / CSI-2 Receiver Design	- 69 -
4.2.1	Module Structure	- 70 -
4.2.2	Clock Lane Module	- 71 -
4.2.3	Data Lane Module	- 72 -
4.2.4	Byte Alignment.....	- 74 -
4.2.5	Align Word Module	- 76 -
4.2.6	CSI-2 Protocol Handling	- 77 -
4.2.7	CSI-2 Receiver Module	- 81 -
4.3	CCI-Handler	- 82 -
4.3.1	Initialization Module	- 82 -
4.3.2	Initialization Control State-Machine	- 87 -
4.4	Framebuffer	- 88 -
4.5	DVI/HDMI Output.....	- 94 -
4.5.1	Colour Balance	- 95 -
4.5.2	VGA Core	- 96 -
4.5.3	DVI Core	- 97 -
4.6	Top Module	- 98 -
5	Initial Testing	- 99 -
5.1	RTL-Simulation	- 99 -
5.1.1	D-PHY/CSI-2 Receiver	- 99 -
5.1.2	Buffer and Output Test.....	- 104 -
5.2	Hardware Tests	- 110 -
5.2.1	D-PHY Input Test	- 110 -

5.2.2	Output Test	- 113 -
5.2.3	Full System Test	- 113 -
6	Conclusion and Outlook.....	- 119 -
7	References.....	- 120 -
8	Appendix.....	- 122 -

List of Figures

Figure 1: Illustration of blanking intervals in a video frame [2]	- 14 -
Figure 2: Timing diagram of the synchronization pulses for formats with negative sync polarity [3]	- 15 -
Figure 3: Basic diagram of CSI-2 and CCI interface using D-PHY [7]	- 16 -
Figure 4: CSI-2 layer definitions [7]	- 17 -
Figure 5: Overview of packet based data transmission [7].....	- 18 -
Figure 6: Long Packet structure for the D-PHY physical layer [7].....	- 18 -
Figure 7: Short packet structure for the D-PHY physical layer [7].....	- 19 -
Figure 8: Example of ECC generation [7]	- 20 -
Figure 9: ECC handling on the receiver side [7].....	- 20 -
Figure 10: 16-bit CRC shift register [7]	- 21 -
Figure 11: Lane distribution concept for D-PHY [7]	- 22 -
Figure 12: Lane merging concept for D-PHY [7]	- 23 -
Figure 13: Example of a two-lane D-PHY transmission [7].....	- 23 -
Figure 14: Byte packing for RAW10 transmission [7]	- 24 -
Figure 15: RAW10 frame format [7]	- 24 -
Figure 16: Block diagram of a universal lane module [8].....	- 25 -
Figure 17: Minimal receiver lane module, modified from [8].....	- 26 -
Figure 18: Illustration of D-PHY line levels [8].....	- 26 -
Figure 19: High-speed data transmission burst [8].....	- 31 -
Figure 20: Trigger-reset command in escape mode [8]	- 32 -
Figure 21: Mode transitions on the clock lane [8].....	- 33 -
Figure 22: TMDS-link architecture [4]	- 36 -
Figure 23: TMDS channel map [4]	- 37 -
Figure 24: Flow chart of TMDS encoding algorithm [4]	- 39 -
Figure 25: TMDS transmission line [4]	- 40 -
Figure 26: Normalized eye diagram for TMDS transmitter [4]	- 41 -
Figure 27: Normalized eye diagram for TMDS receiver [4].....	- 42 -
Figure 28: Normalized wavelength-dependent absorbance of sensory cells in the human eye [13]	- 43 -
Figure 29: Illustration of Bayer filter pixel matrix	- 44 -
Figure 30: Raspberry Pi Camera 2 and pin numbering (right) [15]	- 44 -
Figure 31: Overview and comparison GateMate A1 and A2 [17].....	- 46 -
Figure 32: CPE architecture [17]	- 47 -
Figure 33: Simplified architecture overview GateMate A1 [17].....	- 47 -
Figure 34: GateMate GPIO cell in LVDS mode [17]	- 48 -
Figure 35: GateMate A1 BGA pinout [17]	- 52 -
Figure 36: Sketch of connector layout for the evaluation board	- 53 -
Figure 37: Schematic of a voltage regulator.....	- 54 -
Figure 38: Schematic showing the wiring scheme of the voltage regulators	- 55 -
Figure 39: Capacitor banks for core voltage smoothing	- 55 -
Figure 40: Schematic of camera connector wiring	- 56 -
Figure 41: Schematic of HDMI connector wiring.....	- 57 -
Figure 42: Schematic of programmer connector wiring.....	- 58 -
Figure 44: Reset logic using NAND-gates	- 59 -
Figure 43: LED drive using BJTs	- 59 -

Figure 45: FPGA escape routing on top layer, including copper plane. The holes (marked “12”) are for mounting an optional socket.	61 -
Figure 46: Differential pairs connecting FPGA and HDMI connector	62 -
Figure 47: Differential pairs connecting FPGA and camera connector	63 -
Figure 48: Layout for the voltage regulators. Top layer (left), power layer (centre) and manufacturer's recommendation.....	64 -
Figure 49: Placement of footprints for voltage regulators and auxiliary components	64 -
Figure 50: Placement of all components, including silkscreen	65 -
Figure 51: Top view of the PCB before (left) and after (right) mounting of surface-mounted components.....	66 -
Figure 52: Simplified block diagram of the demonstrator system	68 -
Figure 53: Module structure of the CSI-2 receiver	70 -
Figure 54: Flowchart of the state-machine for packet handling	79 -
Figure 55: Relationship between frame rate and register values [9]	83 -
Figure 56: Clock system in the IMX219 [9]	84 -
Figure 57: Flowchart of the state-machine controlling the initialization of the camera.....	87 -
Figure 58: Simplified illustration of the framebuffer interface.....	88 -
Figure 59: Configuration options for block-RAM in GateMate [17].....	89 -
Figure 60: Mapping of incoming image data to the memory addresses	90 -
Figure 61: Block diagram of the internal structure of the output module.....	94 -
Figure 62: Simulation results showing the header bytes being the first valid output of the align word module	100 -
Figure 63: Selection of signals from simulation. Depicted are the registers of one of the deserializers and the byte aligners from both data lanes at the start of transmission.	101 -
Figure 64: Selected simulated waveforms from the “packet handler” module at the start of a long packet.....	102 -
Figure 65: Selected simulated waveforms from “packet handler” at the end of a long packet -	103 -
Figure 66: Simulated waveforms from framebuffer write side showing interaction between counters	104 -
Figure 67: Simulated waveforms from framebuffer write side at the end of a frame write operation	105 -
Figure 68: Simulated waveforms from framebuffer read side at the end of a frame read operation	105 -
Figure 69: Simulated waveforms of read and write counters during write-side reset.....	106 -
Figure 70: Simulated waveforms from output simulation at the end of a frame	106 -
Figure 71: Simulated waveforms of serializer operation	107 -
Figure 72: Simulated waveforms of colour balance module	107 -
Figure 73: Simulated waveforms from output module during start of transmission	108 -
Figure 74: Excerpt from ILA configuration showing the selection of sample count.....	111 -
Figure 75: Zoomed out view of data stream captured by ILA.....	112 -
Figure 76: Zoomed in view of start of a packet, showing the expected header bytes	112 -
Figure 77: A photo of the test frame on a PC monitor	113 -
Figure 78: A photo of the demonstrator filming the author taking the picture	113 -
Figure 79: 100ms sample of the D-PHY clock lane activity	114 -
Figure 80: 100ms sample of D-PHY data lane activity	115 -
Figure 81: 50ms sample of the "csi_raw_valid" signal	115 -
Figure 82: Pixel clock signal of the DVI output on the LVDS side	117 -

Figure 83: Pixel clock signal of the DVI output on the TMDS side	- 117 -
Figure 84: Pixel clock and red channel data signal of the DVI output on the LVDS side	- 118 -

List of Tables

Table 1: Timing definitions for common video formats with 60Hz field rate from [3]	- 14 -
Table 2: Data type classes [7]	- 19 -
Table 3: ECC parity generation rules [7].....	- 21 -
Table 4: RAW image data types and respective codes [7]	- 24 -
Table 5: HS transmitter DC specifications [8]	- 27 -
Table 6: LP transmitter DC specifications [8].....	- 27 -
Table 7: HS receiver DC specifications [8].....	- 28 -
Table 8: HS receiver AC specifications [8]	- 28 -
Table 9: LP receiver DC specifications [8]	- 28 -
Table 10: LP receiver AC specifications [8]	- 29 -
Table 11: Lane states and their meanings for HS and LP receiver [8].....	- 29 -
Table 12: Start-of-Transmission sequence for HS transmission [8]	- 30 -
Table 13: End-of-Transmission sequence for HS data burst [8].....	- 31 -
Table 14: Escape mode entry codes [8].....	- 32 -
Table 15: Global timing parameters for D-PHY operation [8]	- 35 -
Table 16: Symbols for flowchart [4].....	- 38 -
Table 17: Nominal Ratings for TMDS Link [4]	- 40 -
Table 18: Absolute maximum ratings for TMDS link [4]	- 40 -
Table 19: Voltage swings and timings for TMDS transmitter [4]	- 41 -
Table 20: TMDS receiver characteristics [4].....	- 42 -
Table 21: Pin allocation on the Raspberry Pi Camera 2 connector [15]	- 45 -
Table 22: Configuration mode setup [17]	- 49 -
Table 23: Configuration bank signals [17].....	- 50 -
Table 24: Functional test for FPGA.....	- 67 -
Table 25: Chosen clock settings for the IMX219 in this project	- 84 -
Table 26: Initialization sequence for the camera module	- 86 -
Table 27: Structure of the test input for simulation	- 100 -
Table 28: Comparison of intended and actual ratios between input and output of colour balance module	- 108 -
Table 29: Expected header and distribution on the data lanes.....	- 110 -

Listings

Listing 1: Code snippet from clock lane module, including interface definition	- 71 -
Listing 2: Code snippet from clock lane module showing byte clock generation	- 72 -
Listing 3: Interface definition of data lane module	- 73 -
Listing 4: Module definition of deserializer	- 73 -
Listing 5: Interface definition for align byte module	- 75 -
Listing 6: Code snippet showing logic for byte alignment	- 75 -
Listing 7: Interface definition for align word module	- 76 -
Listing 8: Code snippet showing logic for determining validity of synchronization signals.....	- 77 -
Listing 9: Interface definition of packet handler module	- 78 -
Listing 10: Logic for evaluation of packet headers	- 78 -
Listing 11: Instantiation of the "lsfr_crc" module configured for the CSI-2 CRC	- 80 -
Listing 12: CSI-2 receiver top module interface definition	- 81 -
Listing 13: CCI handler interface definition	- 82 -
Listing 14: Dual-clock RAM module interface definition.....	- 89 -
Listing 15: Write-side signals and asynchronous logic in the framebuffer	- 91 -
Listing 16: Write-side address counter operation in the framebuffer	- 92 -
Listing 17: Read-side signals and asynchronous logic in the framebuffer	- 92 -
Listing 18: Read-side counter operation in the framebuffer.....	- 93 -
Listing 19: Colour balance module interface definition.....	- 95 -
Listing 20: Verilog code for colour balance module	- 96 -
Listing 21: Excerpt from Verilog code of VGA core showing how signals are determined based on counters	- 97 -
Listing 22: TMDS pre-encoder	- 97 -
Listing 23: Comparison between old serializer code (left) and new serializer code (right)	- 98 -

List of Abbreviations

ASIC	<i>Application Specific Integrated Circuit</i>
CCI	<i>Camera Control Interface</i>
CMOS	<i>Complementary Metal-Oxide Semiconductor</i>
CRC	<i>Cyclic Redundancy Check</i>
CSI-2	<i>Camera Serial Interface 2</i>
DDR	<i>Double Data Rate</i>
DVI	<i>Digital Visual Interface</i>
ECC	<i>Error Correction Code</i>
FIFO	<i>First In - First Out</i>
FPGA	<i>Field Programmable Gate Array</i>
HDL	<i>Hardware Description Language</i>
HDMI	<i>High Definition Multimedia Interface</i>
I ² C	<i>Inter-Integrated Circuit</i>
ILA	<i>Integrated Logic Analyzer</i>
LSB	<i>Least Significant Bit</i>
LVC MOS	<i>Low Voltage Complementary Metal Oxide Semiconductor</i>
LVDS	<i>Low Voltage Differential Signalling</i>
MSB	<i>Most Significant Bit</i>
PCB	<i>Printed Circuit Board</i>
PCBA	<i>Printed Circuit Board Assembly</i>
SERDES	<i>Serializer/Deserializer</i>
SPI	<i>Serial Peripheral Interface</i>
TMDS	<i>Transition-Minimized Differential Signaling</i>
VGA	<i>Video Graphics Array</i>

1 Introduction

Digital processing of image data is an increasingly important application for FPGAs in embedded systems. The ability of FPGAs to perform many complex operations in parallel combined with the possibility of rapid and cheap design iteration makes them particularly suited for dealing with the large amounts of data that need to be handled in modern image processing applications.

The first step in an image processing system is the image acquisition, which is performed by digital cameras. The data from these cameras is then transferred to a processing unit. That can be a GPU, an FPGA, an ASIC or any combination of such devices working together. For reduced design complexity and cost, it is desirable to limit the number of components to a minimum, for example by implementing as much of the functionality as possible in a single FPGA. Therefore, it is of great value if the FPGA used in such a system is capable of directly receiving image data from a digital camera.

While several interface standards exist for the transmission of image data from a camera, one of the most widely used interface standards today is the Camera Serial Interface 2 (CSI-2), developed by the Mobile Industry Processor Interface (MIPI) alliance, an industry organisation dedicated to the development of interface standards, primarily for mobile devices. Although the interface is generally suited for implementation in FPGAs, there are some challenges, especially when low-cost FPGAs with no built-in support for the D-PHY or C-PHY physical layers, also defined by the MIPI Alliance, specifically for CSI-2 and other interface standards.

In this project, an IP-core for receiving CSI-2 transmissions on the GateMate A1 FPGA from the German manufacturer Cologne Chip AG is designed and implemented, along with a demonstrator system that forwards the received data to a display. This comprises the design of a dedicated evaluation board with the components required to connect a camera and a display to the FPGA chip, the receiver logic for the D-PHY physical layer, a CSI-2 protocol decoder and some rudimentary image processing hardware. The designed components are then initially tested to verify their functionality.

While the implementation of a CSI-2 receiver in an FPGA is, in itself, not a novel undertaking, doing so in a low-cost FPGA by the only German/European manufacturer targeting consumer electronics applications is useful in evaluating the capabilities of such low-cost devices and may help to sustain healthy competition in the FPGA market. Additionally, it can serve as an example of the many functionalities that can be bundled together in a single component using modern FPGAs.

Chapter 2 of this thesis provides the theoretical background regarding basics of digital video, the CSI-2 interface, the HDMI interface used for displaying the received video data and the key devices, namely the GateMate FPGA and the Raspberry Pi Camera Module.

The third chapter deals with the design and assembly of the new evaluation board for imaging applications using the GateMate A1, followed by initial functional testing.

In the fourth chapter the logic for the CSI-2 receiver and the demonstrator that will be implemented on the GateMate are explained in detail and the results of initial tests are presented.

Finally, the results of the project are summarized and evaluated and potential further optimizations or expansions to the system or individual components thereof are outlined.

2 Theory and Components

The aim of this project is to implement a receiver capable of receiving and processing video data transmitted via a CSI-2 interface using the D-PHY physical layer (with limited bandwidth) on the GateMate FPGA by Cologne Chip AG. To demonstrate and verify the function of the receiver, a Raspberry Pi Camera V2 is used as a data source with a CSI-2 transmitter using the D-PHY physical layer. To allow for a quick and relatively easy validation of the receiver and processing implemented in the FPGA, a video output interface, such as HDMI, is also desirable. In this section, the components of the system, namely the GateMate A1 FPGA, the video interfaces CSI-2 and HDMI, DVI and the Camera Module are described.

This chapter begins with a brief introduction to digital video, followed by a section on the CSI-2 interface and D-PHY physical layer. After that the parts of the HDMI and DVI specification that are relevant to this project, that is primarily the Transition Minimized Differential Signalling (TMDS), are explained. Then the Camera Module is briefly described, along with some fundamental concepts of digital camera technology. In the final part, the GateMate FPGA is presented.

2.1 Digital Video Basics

Digital images are made up of discrete pixels, the colour of which is usually represented by three component values: red, green and blue. For video data transmission, the pixel is usually the unit of reference for timings and formats. The size and/or level of detail in an image is determined by the number of pixels, referred to as the resolution of the image, which is usually given in the format $P_H \times P_V$, where P_H is the number of horizontal pixels and P_V the number of vertical pixels.

When video data is transferred to a display, that transfer usually does not occur as a continuous stream of image data. Since displays, especially cathode-ray-tube displays, generally need time to address the individual pixels, some pauses are required in the transmission, which are referred to as “blanking intervals”. Historically, the primary purpose of these was to enable tube displays to move the electron beam into the proper position before the start of the next line or frame. As video interface and format specifications should ideally be independent of display technology, the concept of blanking intervals is also used with modern digital displays, which may use these intervals for data handling tasks, such as dealing with transmission overhead or as is the case with HDMI, to transmit additional, non-image data [1]. In the timing specifications, blanking intervals are described in terms of “blanking pixels”, that do not contain actual image data. The concept and the placement of blanking pixels within a frame are illustrated in Figure 1.

Standards prescribe or recommend basic data structures, formats and protocols to enable the easy exchange of video data in various formats between devices of different manufacturers. One such standard is ANSI/CTA-861, which defines “protocols, requirements, and recommendations for the utilization of uncompressed digital interfaces by consumer electronics devices” including “Video Formats and waveforms; colorimetry and quantization” [3]. These requirements and recommendations primarily prescribe when to send which data and in which format, whereas technical details of how the transmission is implemented are defined in the interface specifications, such as DVI or HDMI. It should be noted that many “standard formats” have developed over time for different sectors, with film, television and computer graphics having their own preferred formats that usually result from historical technological limitations. For modern computer monitors, televisions and similar digital displays, however, the 60Hz refresh rate appears to have become the de-facto baseline. Therefore, in this chapter 60Hz video formats will be used as examples to introduce the most relevant concepts for this work.

The formats that shall serve as examples here are 1920×1080i @60Hz, 1280×720p @60Hz and 640×480p @60Hz, as these are commonly used “default formats”. The “i” and “p” suffixes indicate if it is a progressive or interlaced format. In progressive formats, the entire frame is transmitted at once, whereas in interlaced formats, even and odd lines are transmitted alternately. Support for one of the first two formats is commonly required for HDTV-capable displays that accept 60Hz video and the latter is the most common “minimum requirement” format for any 60Hz display [1] [3] [4].

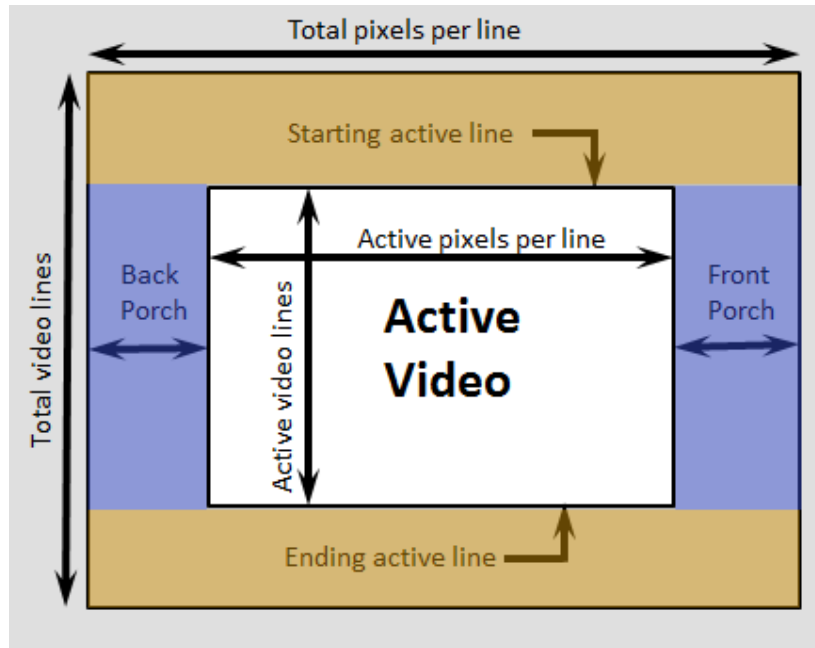


Figure 1: Illustration of blanking intervals in a video frame [2]

Table 1 shows the format definitions, consisting of the number of active pixels per line and column (Hactive, Vactive), whether the format is “progressive” or “interlaced” (I/P), the number of blanking pixels per line and column (Hblank, Vblank) and the pixel frequency. In addition, the table shows derived values, such as the sum of blanking and active pixels (per line and column) or the “line frequency” (H Freq) and frame rate (V Freq), and the video identification code (VIC), which can be used to easily identify/reference any of the formats specified in [3]. Table 1 also shows the length and polarity of the horizontal and vertical synchronization pulses are given along with their relative position in the blanking intervals. The unit is number of pixels, which can be used in combination with the pixel frequency to calculate absolute timings, if required.

VIC	Figure	Hactive	Vactive	I / P	Htotal	Hblank ^d	Hfront	Hsync	Hback	Hpol ^v	Vtotal	Vblank ^d	Vfront	Vsync	Vback	Vpol ^v	H Freq ^d (kHz)	V Freq (Hz)	Pixel Freq ^d (MHz)	Ln
1	1	640	480	Prog	800	160	16	96	48	N	525	45	10	2	33	N	31.469	59.940 ^c	25.175	1
4, 69	2	1280	720	Prog	1650	370	110	40	220	P	750	30	5	5	20	P	45.000	60.000 ^c	74.250	1
5	4	1920	1080	Int	2200	280	88	44	148	P	1125	22.5 ^a	2	5	15	P	33.750	60.000 ^c	74.250	1

Table 1: Timing definitions for common video formats with 60Hz field rate from [3]

The synchronization pulses denote the start of each line (Hsync) and the start of each frame (Vsync), respectively. A graphical representation of how the synchronization pulse timing is given in Figure 2.

Another important aspect of digital video are data formats. These are characterized by the underlying colour model and the resolution. The two commonly used colour models are RGB (red, green, blue) and YUV (not an acronym), which have their own distinct advantages and drawbacks, depending on the application. RGB is a very intuitive colour model, as all three values correspond to an actual colour component, which directly corresponds with the discrete red, green and blue sub-pixels used in digital camera and display systems. The YUV model on the other hand is based on a luminance (brightness) component and two chrominance components that contain the colour information and was historically used to enable black and white and colour TVs to use the same broadcast signals. While not being as intuitive, the YUV model has some distinct advantages in terms of compressibility, making it especially useful where bandwidth is constrained [5]. This is due to the human eye being more sensitive to the luminance component than the colour component, allowing for a variety of ways to reduce the resolution of the latter, with little effect on the perceived image quality. Note that YUV is often (and confusingly) used as a blanket term for any model based on luminance and chrominance components, including the YPbPr (analog) and YCbCr (digital) models [6]. The digital formats described as YUV normally use YCbCr encoding.

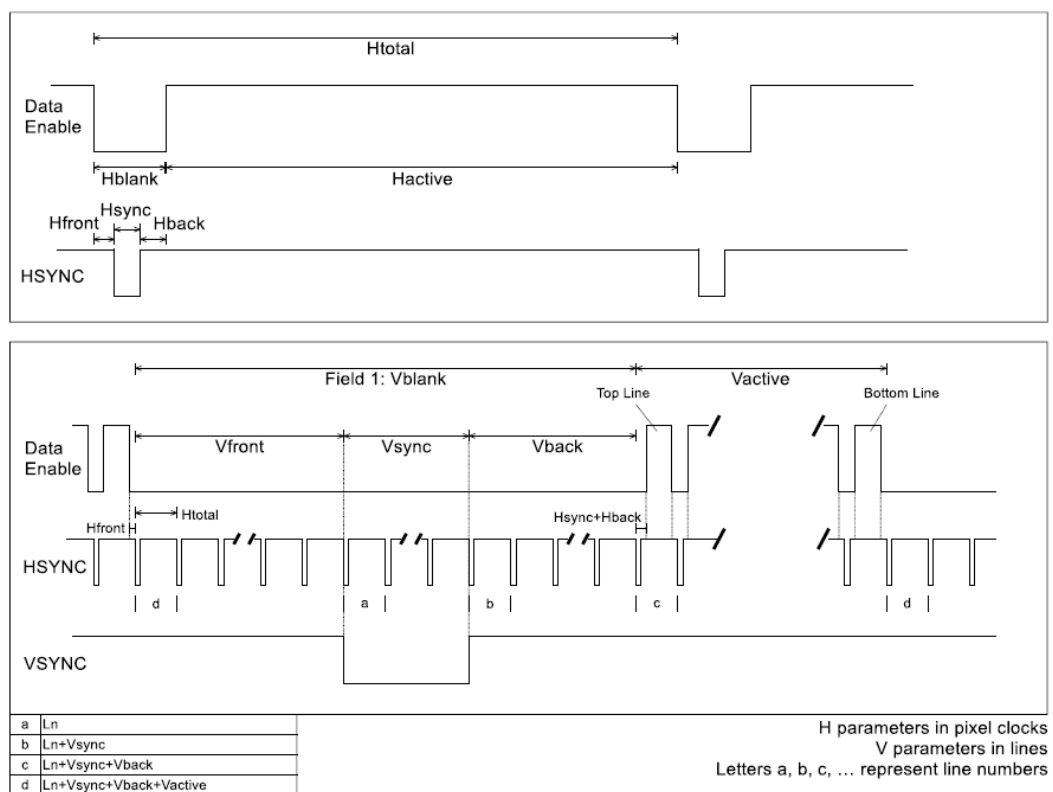


Figure 2: Timing diagram of the synchronization pulses for formats with negative sync polarity [3]

In terms of colour-depth, that is number of bits per pixel, there are many different formats in use, although the most common ones for displays use 24 bits per pixel (8 bits per component), with RGB being the default colour model in ANSI/CTA-861.

Digital cameras do not necessarily use these formats, however. Cameras often output image data in a “RAW” format, which are usually between 6 and 12 bits wide and represent the raw (digital) output of one sub-pixel. This means that for a meaningful evaluation, it must be known what exactly was measured by the sensor. In the context of sub-pixels of a regular digital camera, for example, that is usually the intensity of either red, green or blue light.

2.2 CSI-2 and D-PHY

2.2.1 Basics

The MIPI Camera Serial Interface 2 (CSI-2) is a dedicated serial image data transmission interface developed by the MIPI alliance. It is designed to connect cameras and processing systems in mobile devices over short distances [7]. It is used extensively in mobile consumer electronics as well as automotive and industrial systems. Many small camera chips utilize this interface to communicate with a processing system.

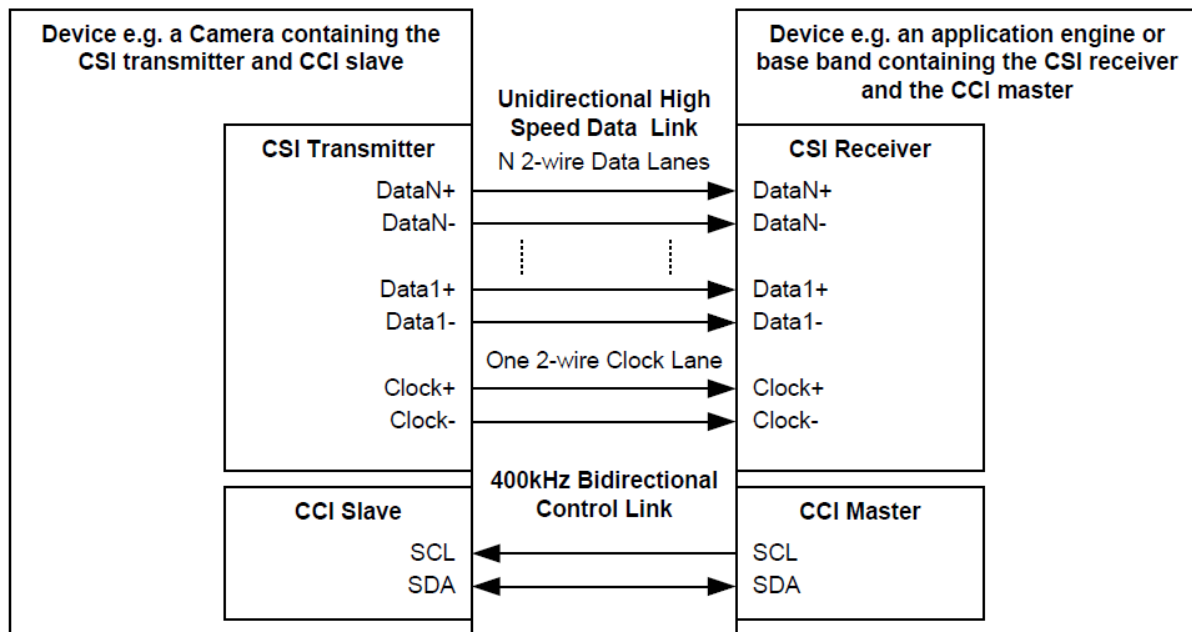


Figure 3: Basic diagram of CSI-2 and CCI interface using D-PHY [7]

The CSI-2 Interface consists of several parallel 2-wire “lanes”, one of which is reserved for clock transmission (Clock Lane), while the remaining lanes (Data Lanes) are used for data transmission. The specification requires at least one Clock Lane and one Data Lane, but more lanes may be added to increase the bandwidth without increasing the clock frequency.

In addition to the main datalink, the CSI-2 specification requires an additional interface for control functions. This Camera Control Interface (CCI) is also specified by the MIPI alliance but is compatible with the commonly used I²C Interface. Therefore, it is sufficient to implement an I²C interface.

The CSI-2 specification defines five functional layers of the interface, as depicted in Figure 4. The top layer is the application layer, that is concerned with the encoding and interpretation of data, most of which is not within the scope of the interface definition. The specification describes the packing of pixel data into Bytes, the packet-based protocol, and the distribution of data across the multiple lanes for multi-lane links. Details of the physical layer are not part of the CSI-2 specification, which can operate with one of two physical layers specified by the MIPI alliance, namely MIPI D-PHY or MIPI C-PHY. In this work the D-PHY physical layer, used by the popular, low-cost camera module utilized in this project, is implemented.

D-PHY (not an acronym) is a physical layer specification, developed for the CSI-2 and DSI (Display Serial Interface) interfaces, but also to be usable as a physical layer for various high speed communication tasks [8]. Whereas CSI-2 is intended for the communication between a camera and a host processor, DSI targets the communication between a host processor and a display. Therefore, the specification allows for a variety of configurations, covering a wide spectrum of possible applications. A combination of high data rates and low (idle) power consumption is achieved by operating in different modes. The primary operating modes are referred to as Low Power (LP) and High Speed (HS). The LP mode is designed to handle basic communication tasks, that do not require high data rates, whereas the HS mode handles the fast transmission of large amounts of data. Beside these primary modes of operation, an optional third mode is defined, that allows slow, asynchronous communication in LP mode.

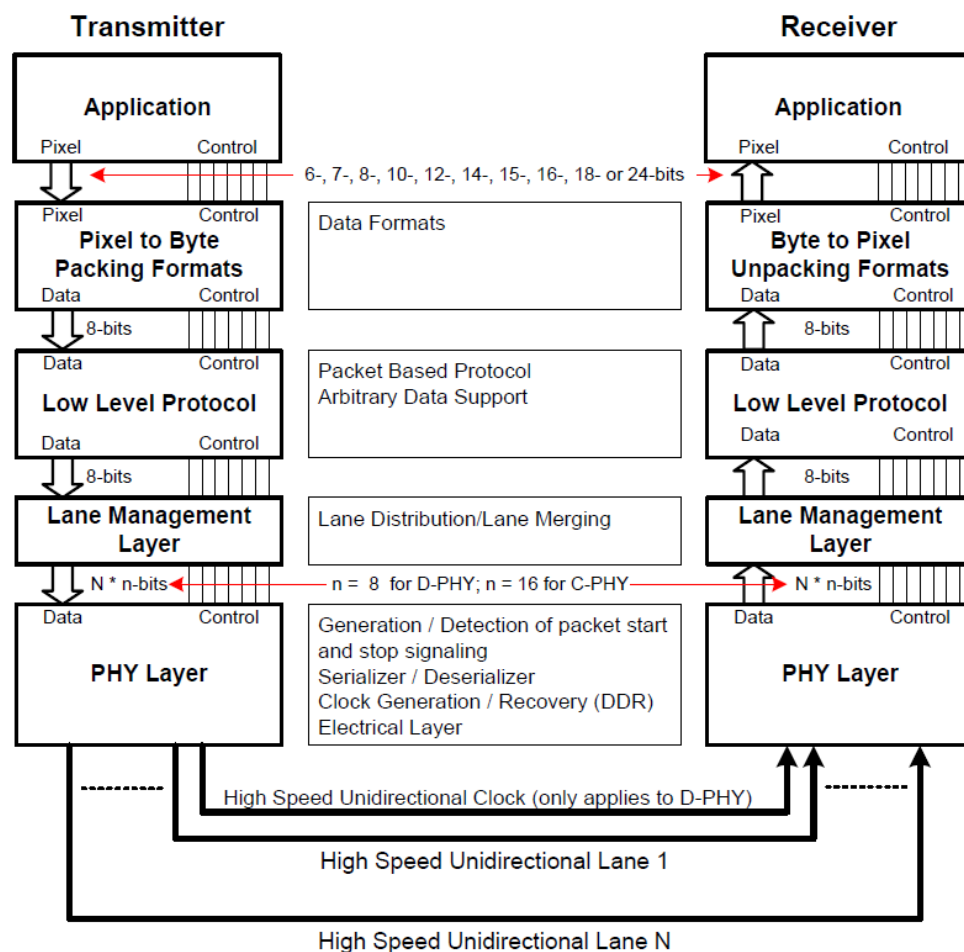


Figure 4: CSI-2 layer definitions [7]

These modes utilize the same two transmissions lines of the respective lane, but in very different ways. The HS mode drives a synchronous differential signal at high frequencies of hundreds of MHz, using a differential, Double Data Rate (DDR) clock. That means that data is sampled at both the rising and falling edges of the clock signal. The LP mode uses both lines independently for single ended signals with higher voltage. These two single-ended signals are evaluated at the receiving side as an asynchronous 2-bit input.

2.2.2 Low-Level Protocol

The Low-Level Protocol specifies the data structure for the transmissions in a Data Lane. Data is structured in bytes that are grouped into packets. Two types of packets, short and long, are specified, with the long packets being used for the bulk of the data transmissions. Short packets are used for synchronisation data, such as codes marking the start and end of frames or lines in the image data, as well as data required for auxiliary functions, such as shutter control and triggering of flashes. The concept of packet-based transmission is illustrated in Figure 5. The structure of both long and short packets depends on the physical layer that is being used. As this work is using D-PHY as the physical layer, only the corresponding packet structures will be introduced.

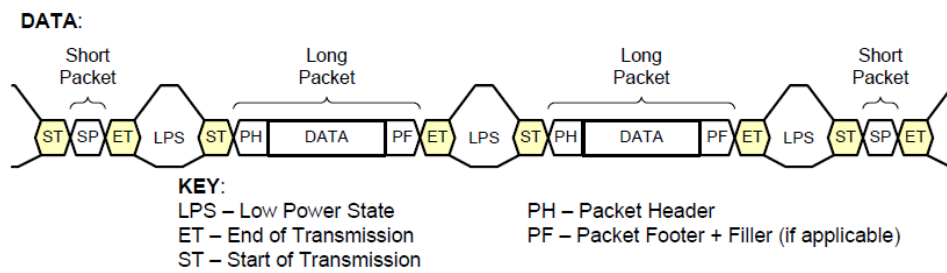


Figure 5: Overview of packet based data transmission [7]

2.2.3 Packet Structure

As depicted in Figure 5, the long packets consist of three parts. First is a 32-bit packet header, containing a Data Identifier, a Word Count indicating the number of payload data bytes (excluding header and footer), and an 8-bit Error Correction Code (ECC). The second part is the actual payload data. The length of this segment is limited only by the maximum Word Count to 65535 bytes and constrained to a multiple of an integer number by the data type. The last part is the packet footer, which contains a 16-bit Checksum to detect errors in the transmission. The full packet structure is shown in Figure 6.

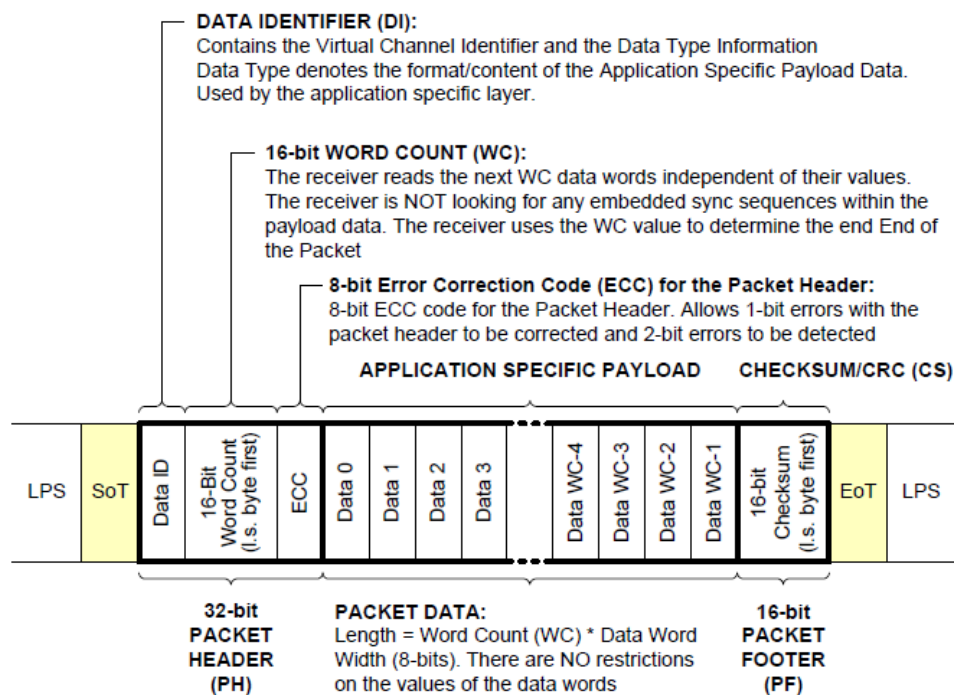


Figure 6: Long Packet structure for the D-PHY physical layer [7]

The structure of short packets matches that of the long packet's packet header. The structure is shown in Figure 7. In the place of the Word Count the short packet has a 16-bit data field for two bytes of payload data.

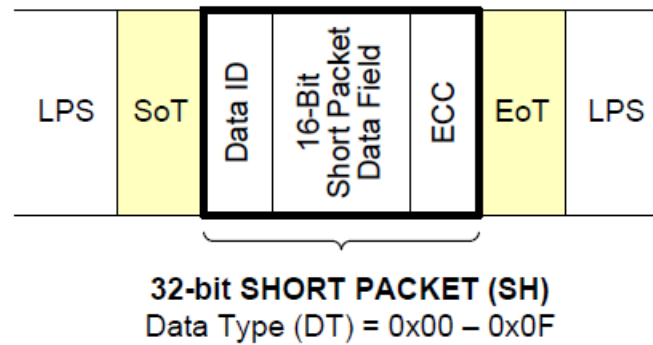


Figure 7: Short packet structure for the D-PHY physical layer [7]

In case the short packet contains frame or line synchronization data, as indicated by the Data ID, the Data Field contains the respective frame or line number. For any other application specific data, the generic short packet data type is used.

Short packets and packet headers for long packets contain two bytes that require further explanation, the data identifier and the ECC. The data identifier byte contains information on the data type and a “virtual channel identifier”. The data types are represented by the six least significant bits. Table 2 provides an overview of the data type classes that are defined in the specification and how they are mapped on the value range. A detailed list of all data types will not be given here. The relevant data types for this work will be shown in a later chapter.

Data Type	Description
0x00 to 0x07	Synchronization Short Packet Data Types
0x08 to 0x0F	Generic Short Packet Data Types
0x10 to 0x17	Generic Long Packet Data Types
0x18 to 0x1F	YUV Data
0x20 to 0x27	RGB Data
0x28 to 0x2F	RAW Data
0x30 to 0x37	User Defined Byte-based Data
0x38 to 0x3F	Reserved

Table 2: Data type classes [7]

The virtual channel identifier indicates which of four virtual channels the data shall be assigned to. These virtual channels are a way of interleaving independent data in the same serial data stream. This allows, for example, several video streams to be transmitted on the same CSI-2 link simultaneously.

2.2.4 Error Correction Code and Checksum

The ECC plays an important role in ensuring the integrity of the transmission. It allows single-bit errors in the data identifier and word count (or packet data in short packets) to be corrected and two-bit errors to be detected. It is only used for the D-PHY physical layer. The ECC uses a Hamming code, modified by an additional parity bit to allow single-bit error correction and two-bit error detection simultaneously. In Figure 8 an example of an ECC for a long packet header is shown, including the mapping of the data identifier and word count bytes (LSB first) to the 24-bit sequence used for the ECC calculation.

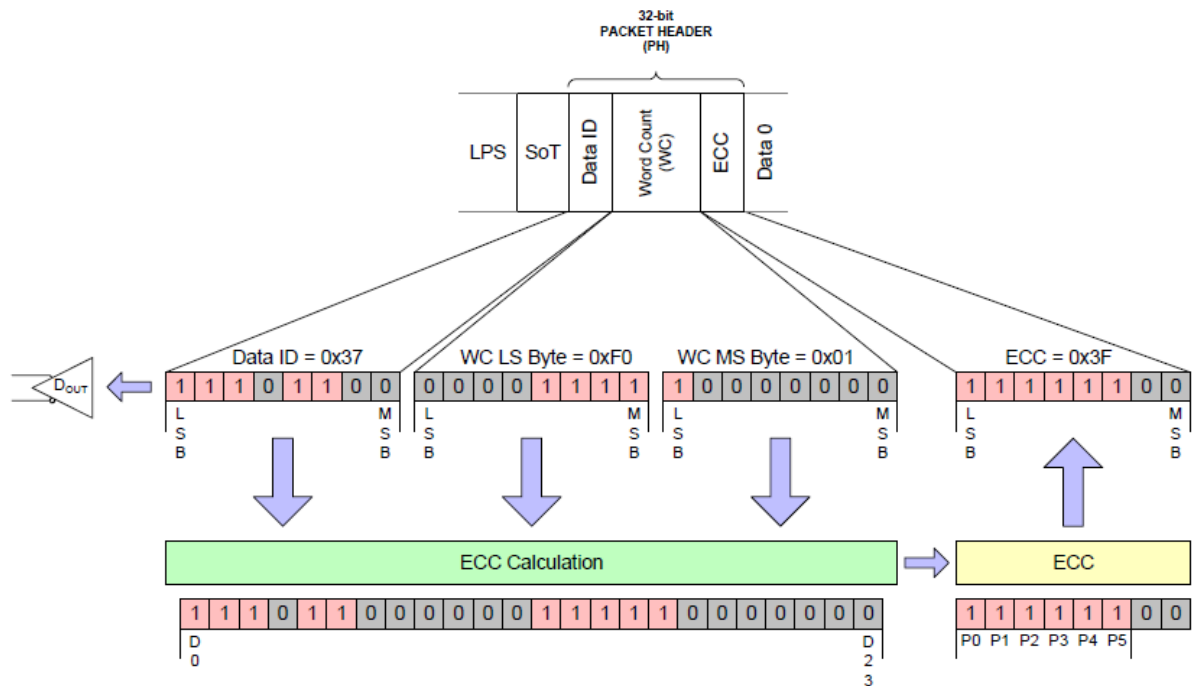


Figure 8: Example of ECC generation [7]

The MSBs of the ECC (P6 and P7) are always 0. The calculation of the remaining bits is done by XORing a defined set of bits from the 24-bit sequence made up of the Data ID and the two Word Count bytes, arranged as depicted in Figure 8.

Which bits of the 24-bit sequence are used to calculate each parity bit, can be seen in Table 3. The 1s in a column indicate if a bit is used for in the calculation of the respective parity bit. The table also shows the syndromes that indicate an error in the respective bit position.

To use the ECC in the receiver a new ECC must be computed, based on the received header, which is then compared in bitwise XOR with the received ECC. A block diagram of the logic required on the receiver side is given in Figure 9.

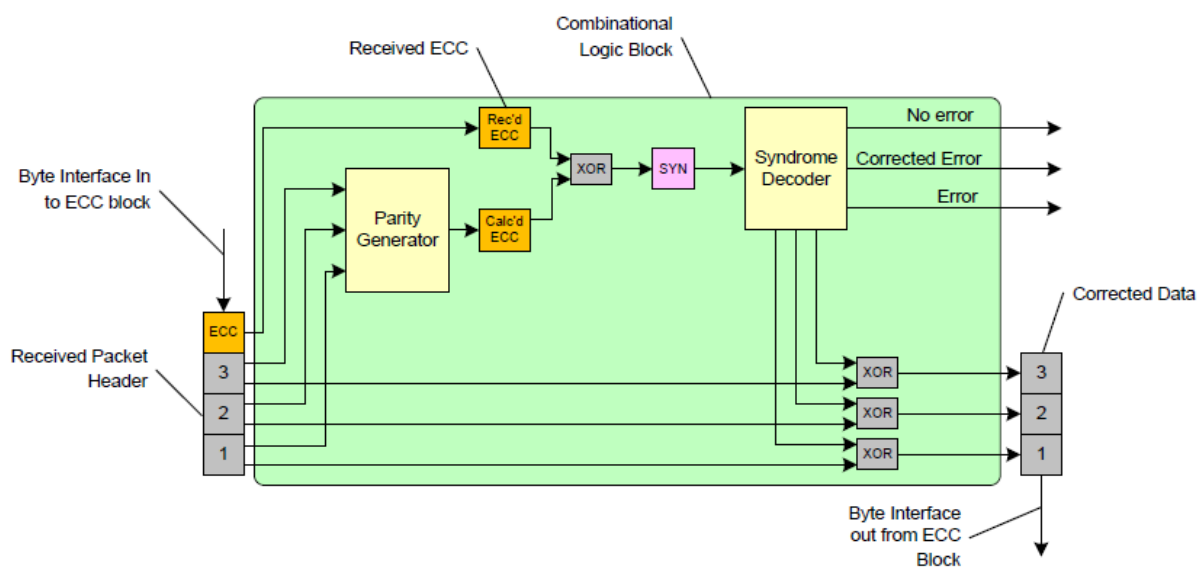
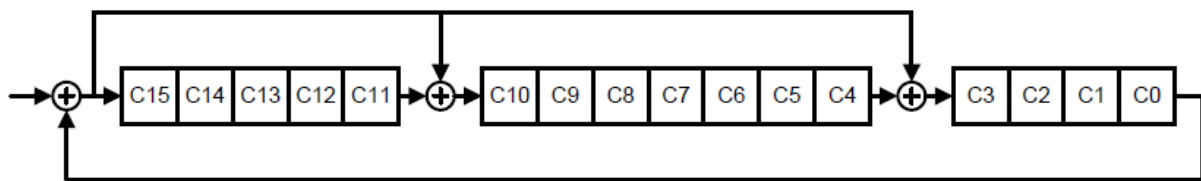


Figure 9: ECC handling on the receiver side [7]

Bit	P7	P6	P5	P4	P3	P2	P1	P0	Hex
0	0	0	0	0	0	1	1	1	0x07
1	0	0	0	0	1	0	1	1	0x0B
2	0	0	0	0	1	1	0	1	0x0D
3	0	0	0	0	1	1	1	0	0x0E
4	0	0	0	1	0	0	1	1	0x13
5	0	0	0	1	0	1	0	1	0x15
6	0	0	0	1	0	1	1	0	0x16
7	0	0	0	1	1	0	0	1	0x19
8	0	0	0	1	1	0	1	0	0x1A
9	0	0	0	1	1	1	0	0	0x1C
10	0	0	1	0	0	0	1	1	0x23
11	0	0	1	0	0	1	0	1	0x25
12	0	0	1	0	0	1	1	0	0x26
13	0	0	1	0	1	0	0	1	0x29
14	0	0	1	0	1	0	1	0	0x2A
15	0	0	1	0	1	1	0	0	0x2C
16	0	0	1	1	0	0	0	1	0x31
17	0	0	1	1	0	0	1	0	0x32
18	0	0	1	1	0	1	0	0	0x34
19	0	0	1	1	1	0	0	0	0x38
20	0	0	0	1	1	1	1	1	0x1F
21	0	0	1	0	1	1	1	1	0x2F
22	0	0	1	1	0	1	1	1	0x37
23	0	0	1	1	1	0	1	1	0x3B

Table 3: ECC parity generation rules [7]

Beside the ECC in the header, every long packet has a 16-bit checksum in the packet footer, that is calculated based on the payload data bytes. The checksum is a cyclic redundancy check (CRC) based on the generator polynomial $x^{16}+x^{12}+x^5+x^0$. The byte order used in the calculation is the same as the byte order before distribution on the lanes, if more than one lane is used.



Polynomial: $x^{16} + x^{12} + x^5 + x^0$
Note: C15 represents x^0 , C0 represents x^{15}

Figure 10: 16-bit CRC shift register [7]

Figure 10 shows the structure of a CRC shift register as it could be used for the CRC generation. According to the specification, the register must be initialized to 0xFFFF before starting the calculation for each packet. The checksum is the value stored in the register after all payload data has passed through. This checksum is transmitted to the receiver as the packet footer so that the receiver may verify if any errors occurred during transmission. The exact implementation of this check is not specified.

2.2.5 Lane Distribution

The distribution of data to the lanes of the CSI-2 link follows simple rules, allowing for easy scalability of the interface. The exact rules depend on the choice of physical layer, D-PHY or C-PHY but in both cases the distribution happens byte-wise. Here, only the D-PHY variant will be described.

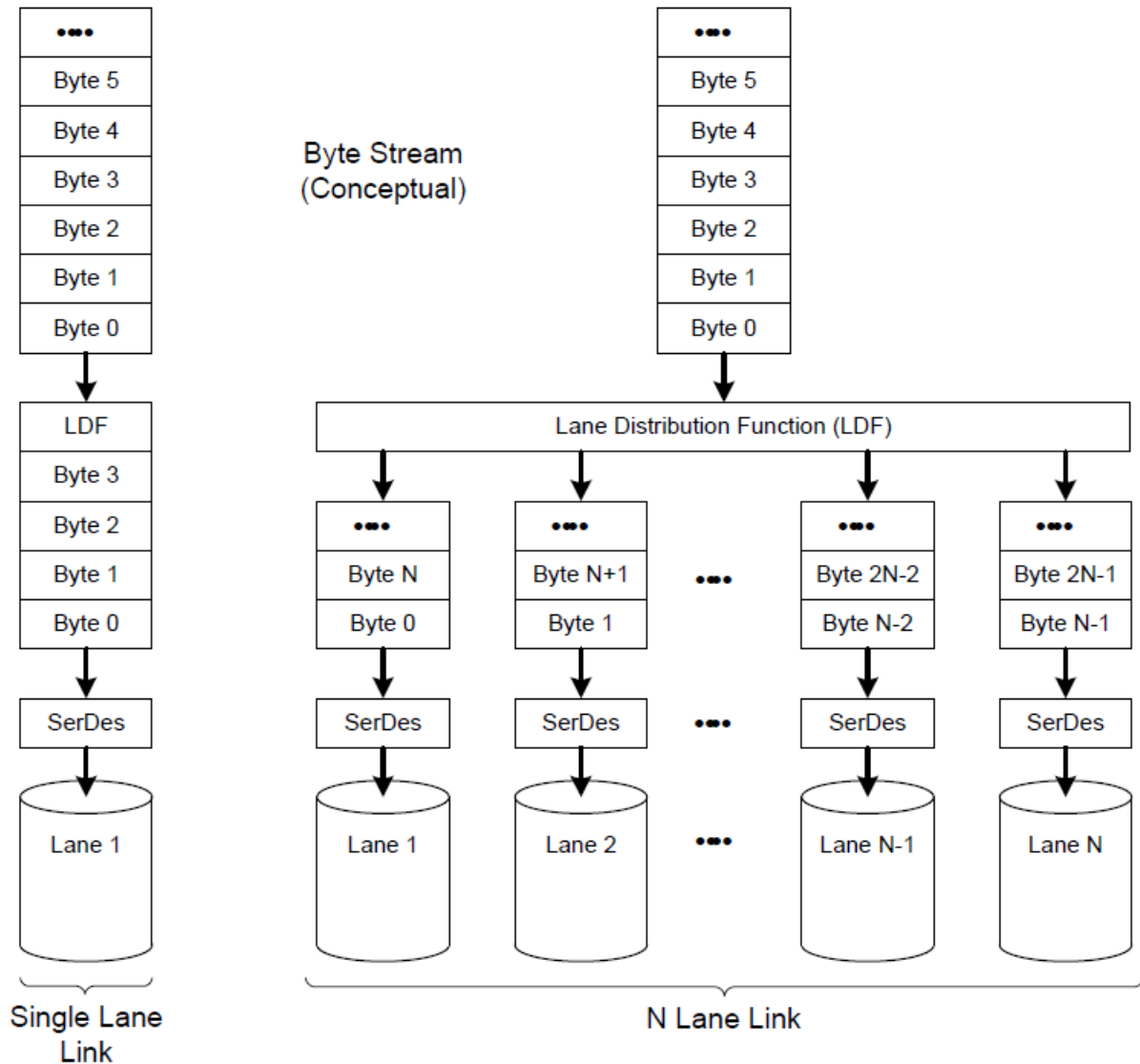


Figure 11: Lane distribution concept for D-PHY [7]

The distribution of bytes on the lanes for a CSI-link using D-PHY is illustrated in Figure 11. The first byte is assigned to the first lane, the second byte to the second and so on until all lanes have been assigned one byte. Then the process repeats. At the receiver side, the bytes are put back in the original order, as shown in Figure 12.

The transmission begins on all lanes simultaneously and ends whenever the last byte on the lane is sent. Unlike the start of the transmission, the end of transmission only occurs synchronously if the number of bytes transmitted is an integer multiple of the number of lanes. An illustration for a 2-lane link is given in Figure 13.

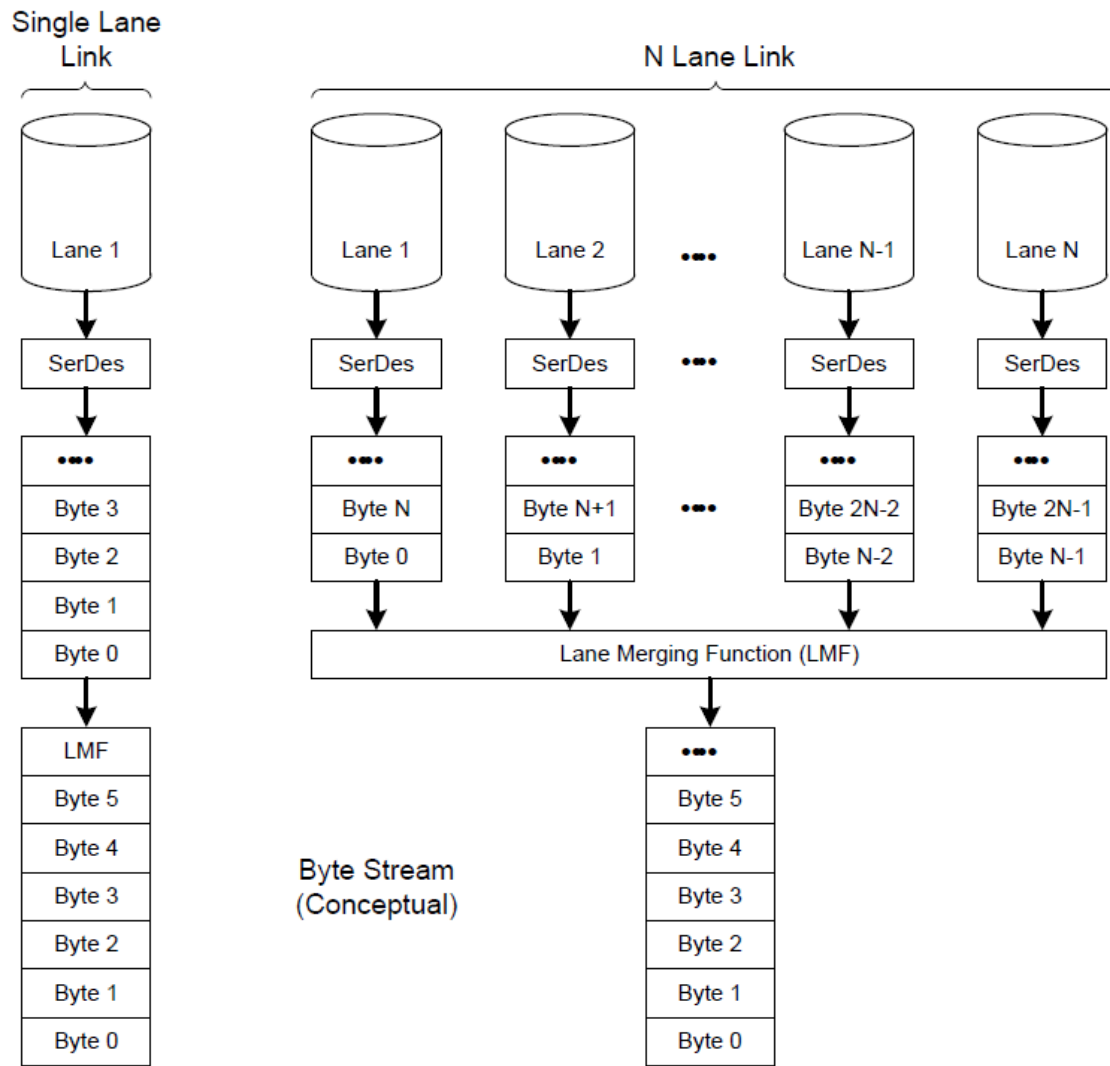
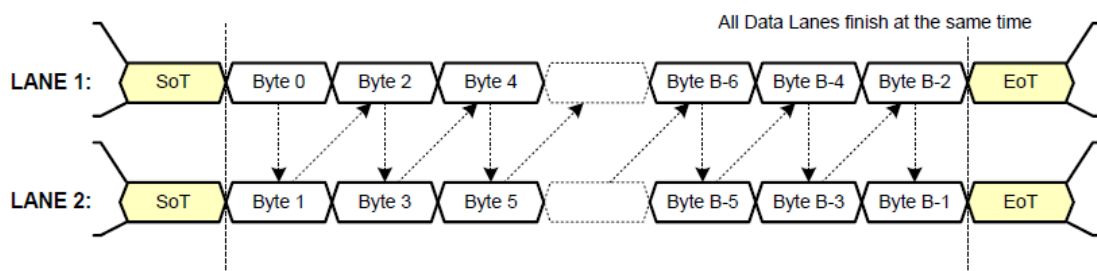
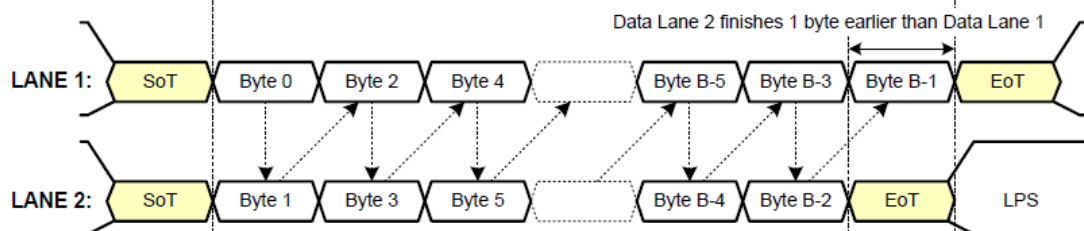


Figure 12: Lane merging concept for D-PHY [7]

Number of Bytes, B, transmitted is an integer multiple of the number of lanes:



Number of Bytes, B, transmitted is NOT an integer multiple of the number of lanes:



KEY:
LPS – Low Power State SoT – Start of Transmission EoT – End of Transmission

Figure 13: Example of a two-lane D-PHY transmission [7]

2.2.6 Data Formats

CSI-2 supports a variety of common data formats for video data, including YUV (colour model based on luminance and chrominance values), RGB and RAW formats. Each of these is assigned a data type code, that is transferred with each packet in the Data Identifier byte. An overview of the data type classes is given in Table 2. Due to the many specific data types, this section will focus on the RAW data types that are relevant for this work because the camera module uses the RAW8 or RAW10 data format and will be configured for RAW10 [9].

Data Type Code	Description
0x28	RAW6
0x29	RAW7
0x2A	RAW8
0x2B	RAW10
0x2C	RAW12
0x2D	RAW14
0x2E	Reserved
0x2F	Reserved

Table 4: RAW image data types and respective codes [7]

The RAW10 data format encodes the information for each pixel in a 10-bit data word. Due to the 8-bit byte size used by the CSI-2 interface, the pixel data has to be rearranged accordingly for the transmission. This is done by packing the data for four pixels into five bytes with a total length of 40 bits according to the rule depicted in Figure 14.

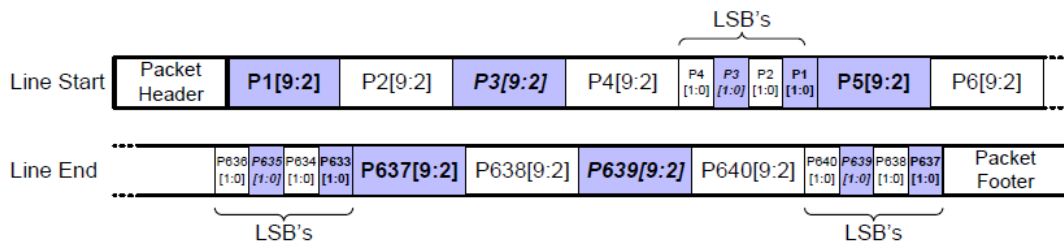


Figure 14: Byte packing for RAW10 transmission [7]

The eight MSBs of each pixel are sent as one byte in the original order of the pixels, followed by one byte containing the LSBs of the pixels, with the order reversed. Each byte is sent LSB first, as is the rule in CSI-2 transmissions. As a result of this packing rule any packet containing RAW10 data must contain a multiple of 5 payload data bytes. Each line of the image is sent in one long packet, resulting in the frame format shown in Figure 15.

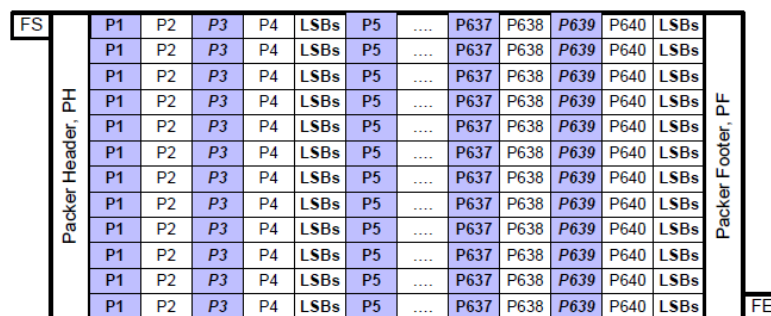


Figure 15: RAW10 frame format [7]

2.2.7 D-PHY Architecture

A D-PHY link consists of one clock lane and one or more data lanes. Figure 16 shows a universal Lane Module, that contains all the necessary and optional components. Note that not all of these functions are required in all Lanes to comply with the specification.

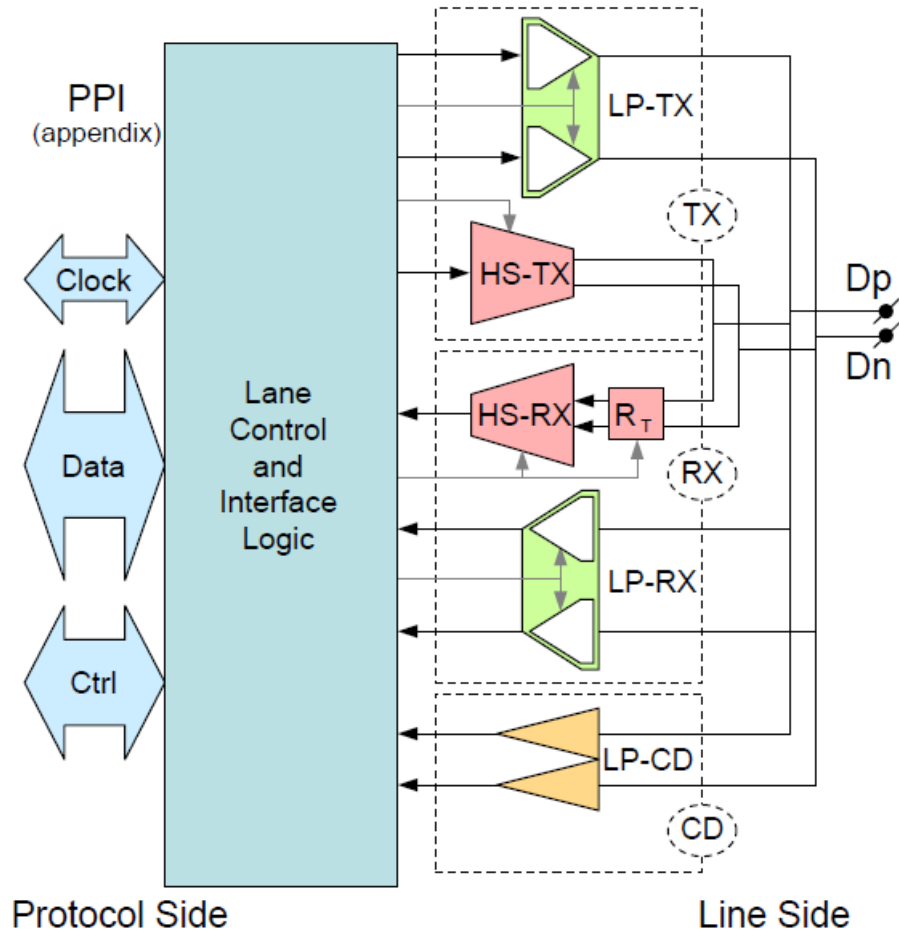


Figure 16: Block diagram of a universal lane module [8]

The universal Lane Module comprises a transmitter and receiver for HS signals, a transmitter and receiver for LP signals as well as an LP contention detector (LP-CD), which is required in the turnaround procedure for bi-directional datalinks. For the sake of brevity, this section will focus on the necessary functionality and not explore the full set of possible configurations.

A D-PHY link has a master and a slave side and therefore assumes a primary direction for data transmission. Although data transmission is possible in the reverse direction, the High-Speed DDR clock is always generated at the master side and received at the slave side.

While there is a multitude of possible configurations for Data Lane Modules, depending on the required functionality, there are only two types of Clock Lane Modules. The Master Side Module contains the same HS TX and LP TX components as a unidirectional Data Lane Module, while the Slave Side Module contains the corresponding RX components. The Clock Lane operates very similarly to the Data Lanes, however the HS clock signal is transmitted with a 90-degree phase-difference (quadrature phase) with the Data Lane signals to ensure that the data signals have reached a stable state when the clock edges occur. Also, the Escape Mode works differently for Clock Lanes.

The clock signal generation is handled outside the PHY and is not covered by the D-PHY specification [8].

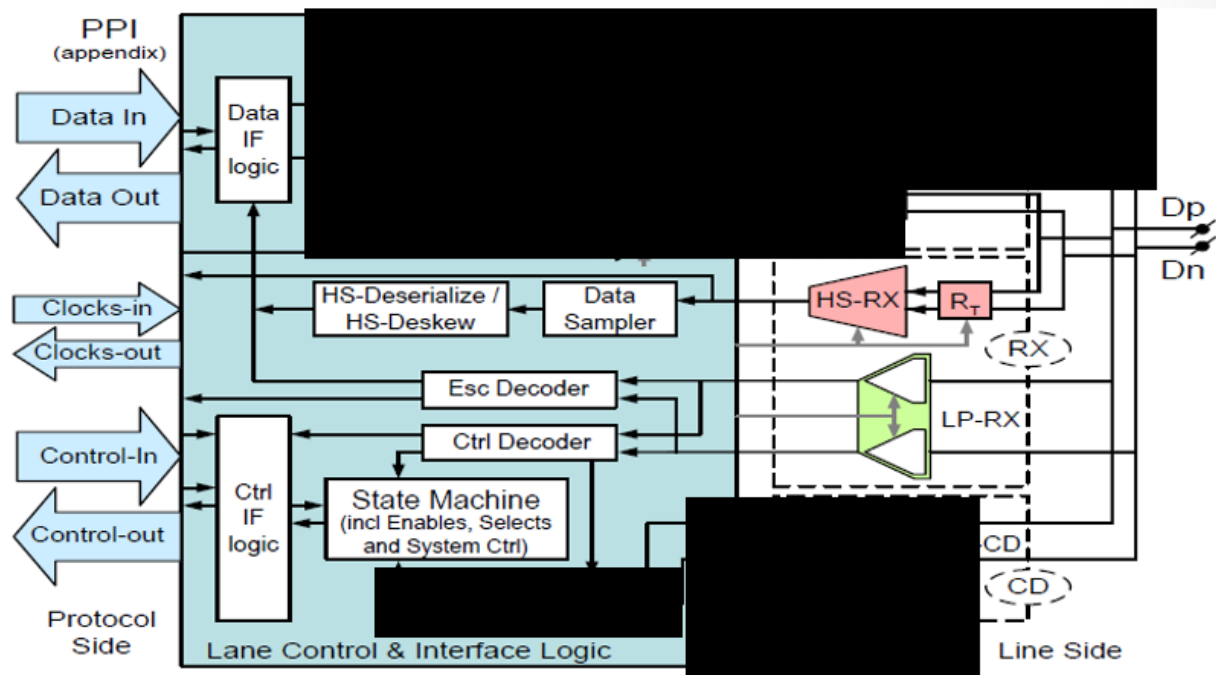


Figure 17: Minimal receiver lane module, modified from [8]

Figure 17 shows the minimum required components for a data lane receiver module. For a unidirectional link, the receiver side does not require any transmitters and no contention module, greatly simplifying the implementation.

2.2.8 D-PHY Signal Levels

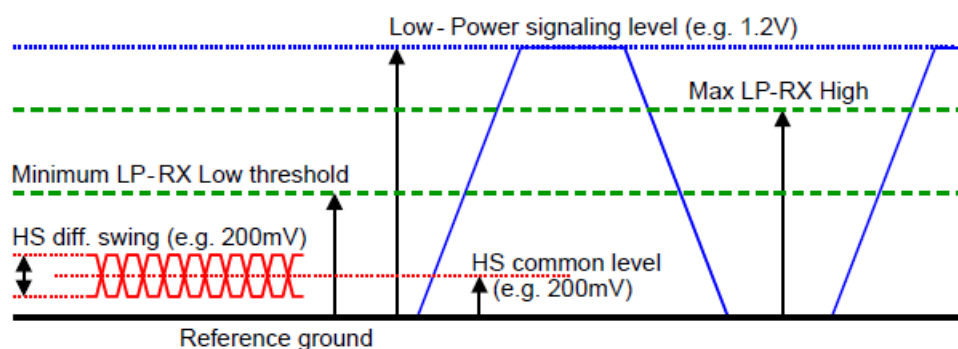


Figure 18: Illustration of D-PHY line levels [8]

Figure 18 shows the voltage levels used for the HS and LP signalling in D-PHY transmissions. The HS mode uses differential signals according to the SLVS specification [10] with a low voltage swing ($\pm 200\text{mV}$) and low common mode voltage of around 200mV . This allows high data rates at low power consumption and enables the use of low supply voltages of 0.8V [11].

The LP signals are single-ended and relatively slow, with any state being held for a minimum of 50ns . The normal HIGH voltage is 1.2V , normal LOW voltage is 0V .

Parameter	Description	Min	Nom	Max	Unit	Notes
C_{CMTX}	HS transmit static common-mode voltage	150	200	250	mV	1
$ \Delta V_{\text{CMTX}(1,0)} $	V_{CMTX} mismatch when output is Differential-1 or Differential-0			5	mV	2
$ V_{\text{OD}} $	HS transmit differential voltage	140	200	270	mV	1
$ \Delta V_{\text{OD}} $	VOD mismatch when output is Differential-1 or Differential-0			14	mV	2
V_{OHHS}	HS output high voltage			360	mV	1
Z_{OS}	Single ended output impedance	40	50	62.5	Ω	
ΔZ_{OS}	Single ended output impedance mismatch			10	%	
Notes: - Value when driving into load impedance anywhere in the Z_{ID} range - A transmitter should minimize ΔV_{OD} and $\Delta V_{\text{CMTX}(1,0)}$ in order to minimize radiation and optimize signal integrity						

Table 5: HS transmitter DC specifications [8]

Parameter	Description	Min	Nom	Max	Unit	Notes
V_{OH}	Thevenin output high level	1.1	1.2	1.3	V	1
		0.95		1.3	V	2
V_{OL}	Thevenin output low level	50		50	mV	
Z_{OLP}	Output impedance of LP transmitter	110			Ω	
Notes: - Applicable when the supported data rate ≤ 1.5 Gbps. - Applicable when the supported data rate > 1.5 Gbps.						

Table 6: LP transmitter DC specifications [8]

The exact DC specifications for HS and LP transmitters are shown in Table 5 and Table 6 respectively.

The AC specifications for the transmitters are omitted here, as they are of no great significance for the receiver side, which is the main subject of this work.

For the HS receiver the DC and AC specifications are given in Table 5 and Table 8 respectively. While it is not explicitly stated in the specification, it can be calculated from the DC specification, that the HS transmitter drives a current of 2mA through the 100 Ohm input impedance of the receiver to achieve the 200mV differential voltage. This is relevant for the design of the receiver in Section 3.

Parameter	Description	Min	Nom	Max	Unit	Notes
$V_{CMRX(DC)}$	Common-mode voltage HS receive mode	70		330	mV	1, 2
V_{IDTH}	Differential input high threshold			70	mV	3
				40	mV	4
V_{IDTL}	Differential input low threshold	-70			mV	3
		-40			mV	4
V_{IHHS}	Single-ended input high voltage			460	mV	1
V_{ILHS}	Single-ended input low voltage	-40			mV	1
$V_{TERM-EN}$	Single-ended threshold for HS termination enable			450	mV	
Z_{ID}	Differential input impedance	80	100	125	Ω	
Notes: - Excluding possible additional RF interference of 100mV peak sine wave beyond 450MHz - This table value includes a ground difference of 50mV between transmitter and receiver, the static common-mode level tolerance and variations below 450MHz. - For devices supporting data rates ≤ 1.5Gbps. - For devices supporting data rates > 1.5Gbps.						

Table 7: HS receiver DC specifications [8]

Parameter	Description	Min	Nom	Max	Unit	Notes
$V_{CMRX(HF)}$	Common-mode interference beyond 450MHz			100	mV	2, 5
				50	mV	2, 6
$V_{CMRX(LF)}$	Common-mode interference 50MHz - 450MHz	-50		50	mV	1, 4, 5
		-25		25	mV	1, 4, 6
C_{CM}	Common-mode termination			60	pF	3
Notes: - Excluding 'static' ground shift of 50mV. $V_{CMRX(HF)}$ is the peak amplitude of a sine wave superimposed on the receiver inputs. - For higher bit rates a 14pF capacitor will be needed to meet the common-mode return loss specification. - Voltage difference compared to the DC average common-mode potential. - For devices supporting data rates ≤ 1.5Gbps. - For devices supporting data rates > 1.5Gbps.						

Table 8: HS receiver AC specifications [8]

Parameter	Description	Min	Nom	Max	Unit	Notes
V_{IH}	Logic 1 input voltage	880			mV	1
		740			mV	2
V_{IL}	Logic 0 input voltage, not in ULP State			550	mV	
$V_{IL-ULPS}$	Logic 0 input voltage, ULP State			300	mV	
V_{HYST}	Input hysteresis	25			mV	
Notes: - For devices supporting data rates ≤ 1.5Gbps. - For devices supporting data rates > 1.5Gbps.						

Table 9: LP receiver DC specifications [8]

Parameter	Description	Min	Nom	Max	Unit	Notes
e_{SPIKE}	Input pulse rejection			300	V*ps	1, 2, 3
$T_{\text{MIN-RX}}$	Minimum pulse width response	20			ns	4
V_{INT}	Peak interference amplitude			200	mV	
f_{INT}	Interference frequency	450			MHz	
Notes: 1. Time-voltage integration of a spike above V_{IL} when being in LP-0 state or below V_{IH} when being in LP-1 state. e_{SPIKE} generation will ensure the spike is crossing both $V_{\text{IL,max}}$ and $V_{\text{IH,min}}$ levels. 2. An impulse less than this will not change the receiver state. 3. In addition to the required glitch rejection, implementers shall ensure rejection of known RF-interferers. 4. An input pulse greater than this shall toggle the output.						

Table 10: LP receiver AC specifications [8]

Since the maximum LOW threshold for the LP signals is well above the maximum voltages of the HS signals, any HS transmission will always be read as LP-00 by the LP receiver. In contrast, the HS signalling requires a switchable termination to be compliant with the specification, which ensures that LP signals are not falsely read as HS signals by the receiver and reduces the power consumption, as a resistor between the signal lines would cause significant leakage current between the signal lines during LP transmissions [8].

Table 11 provides an overview of how the different possible Lane States are interpreted in different operating modes. These operating modes are described in detail in the following section.

State Code	Line Voltage Levels		High-Speed	Low-Power	
	Dp-Line	Dn-Line	Burst Mode	Control Mode	Escape Mode
HS-0	HS Low	HS High	Differential-0	N/A, Note 1	N/A, Note 1
HS-1	HS High	HS Low	Differential-1	N/A, Note 1	N/A, Note 1
LP-00	LP Low	LP Low	N/A	Bridge	Space
LP-01	LP Low	LP High	N/A	HS-Rqst	Mark-0
LP-10	LP High	LP Low	N/A	LP-Rqst	Mark-1
LP-11	LP High	LP High	N/A	Stop	N/A, Note 2

Note:

1. During High-Speed transmission the Low-Power Receivers observe LP-00 on the Lines.
2. If LP-11 occurs during Escape mode the Lane returns to Stop state (Control Mode LP-11).

Table 11: Lane states and their meanings for HS and LP receiver [8]

2.2.9 D-PHY Operating Modes

The D-PHY specification describes three distinct operating modes, implemented as state-machines in the transmitter and receiver. These are Control Mode, High-Speed Mode and Escape Mode. Control and Escape Mode utilize the slow LP signalling and use a very limited set of commands, whereas the High-Speed mode is used for fast transmission, using the HS signalling.

At start-up and during times when no High-Speed data transmission takes place, the lanes are in Control Mode, with the HS terminations switched off. Control mode serves as a transition state between High-Speed transmissions or Escape Mode and handles such functions as data-lane turnaround for bi-directional Data Lanes. The default state of a Data Lane in control mode is LP-11, referred to as “Stop state”. A transition to the Stop state always initiates a return to Control Mode from any other mode.

High-Speed Mode (HS Mode) is used for the primary data transmissions. These transmissions occur in bursts whose length is not specified by the D-PHY specification but depends on the protocol with which the D-PHY interface is used. Between each burst the Lane enters Control Mode. An HS transmission is initiated by the transmitter by High-Speed transmission request (LP-11, LP-01, LP-00) and ends in a Stop state. At the start and end of each HS burst the transmitter sends a leader/trailer sequence to help receiver synchronisation. The payload data must be at least one byte in length and must always be an integer number of bytes.

The full Start-of-Transmission sequence for an HS transmission burst is shown in Table 12.

TX Side	RX Side
Drives Stop state (LP-11)	Observes Stop State
Drives HS-Rqst state (LP-01) for time T_{LPX}	Observes transition from LP-11 to LP-01 on the lines
Drives Bridge State (LP-00) for time $T_{HS-PREPARE}$	Observes transition from LP-01 to LP-00 on the lines, enables HS termination after time $T_{D-TERM-EN}$
Enables High-Speed driver and disables Low-Power drivers simultaneously	
Drives HS-0 for time $T_{HS-ZERO}$	Enables HS-RX and waits for timer $T_{HS-SETTLE}$ to expire in order to neglect transition effects
	Starts looking for leader sequence
Inserts the HS Sync-Sequence ‘00011101’ beginning on a rising clock edge	
	Synchronizes upon recognition of leader sequence ‘011101’
Continues to transmit High-Speed payload data	
	Receives Payload Data

Table 12: Start-of-Transmission sequence for HS transmission [8]

At the end of the HS Data Burst, the Data Lane reverts to a Stop state via an End-of-Transmission (EoT) sequence, such as shown in Table 13. The D-PHY specification does not prescribe the details of the procedure and allows for the protocol to handle the EoT process.

TX Side	RX Side
Completes transmission of payload data	Receives payload data
Toggles differential state immediately after last payload data bit and keeps that state for a time $T_{HS-TRAIL}$	
Disables the HS-TX, enables the LP-TX, and drives Stop state (LP-11) for a time $T_{HS-EXIT}$	Detects the Lines leaving LP-00 state and entering Stop state and disables termination
	Neglect bits of last period $T_{HS-SKIP}$ to hide transition effects
	Detect last transition in valid data, determine last valid data byte and skip trailer sequence

Table 13: End-of-Transmission sequence for HS data burst [8]

A visual representation of the Lane states is presented in Figure 19. Several timing constraints for SoT and EoT sequences are also illustrated.

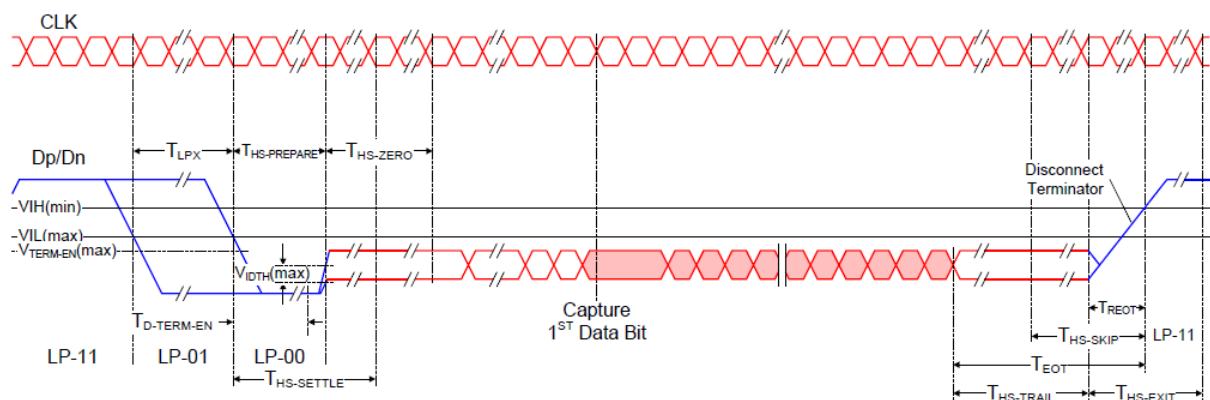


Figure 19: High-speed data transmission burst [8]

Escape Mode also uses LP signals and is designed to send “triggers” to the receiving side. These triggers are used to request a reset or send flags, the meaning of which is not specified and depends on the application. Escape mode must be supported in the forward direction and is optional in the reverse direction. A compliant receiver must therefore be able to process Escape Mode commands. Like the HS Mode, Escape Mode is entered from Command Mode via a defined sequence (LP-11, LP-10, LP-00, LP-01, LP-00) and exited upon detection of a Stop state (LP-11). In Escape mode Spaced-One-Hot bit encoding is used to transmit signals without the need for a clock. An example of Escape Mode operation is shown in Figure 20, depicting a “Trigger-Reset” command in Escape Mode.

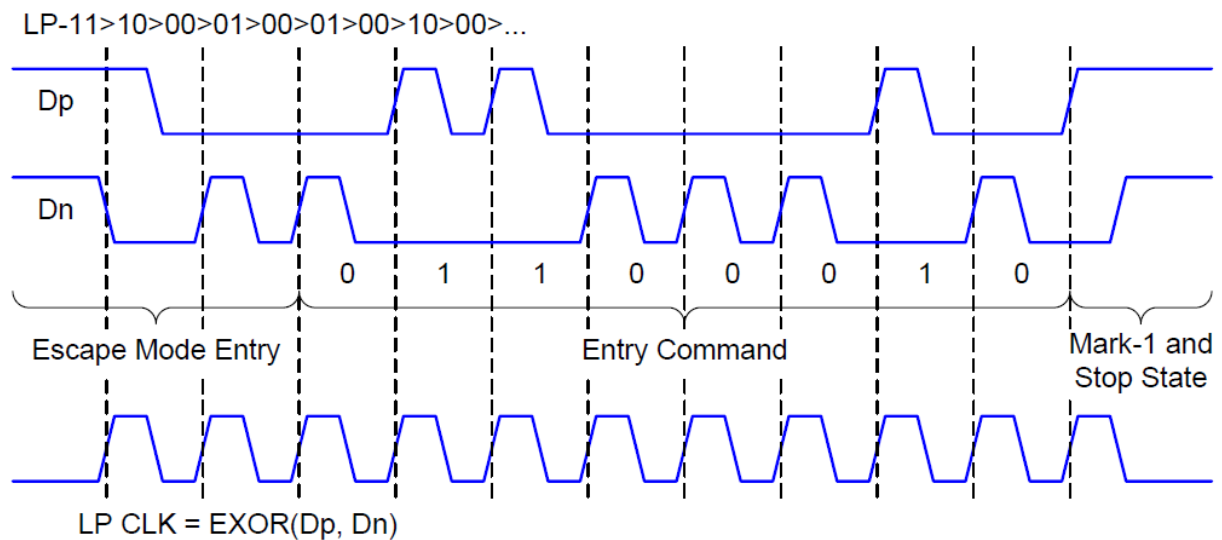


Figure 20: Trigger-reset command in escape mode [8]

Beyond the basic Trigger commands, Escape Mode is also used to put Lanes into the ultra-low power state (UPLS), in which the lines are kept in the LP-00 state, or for the Low Power Data Transmission, which is an optional feature and shall therefore not be further explored in this work.

The full list of Escape Mode commands is given in Table 14. The “Trigger” commands, including Reset-Trigger do not directly impact the behaviour of the physical layer, but only serve as generic signals for the Protocol Layer. Any invalid commands in Escape mode are to be ignored by the system.

Escape Mode Action	Command Type	Entry Command Pattern
Low-Power Data Transmission	Mode	11100001
Ultra-Low Power State	Mode	00011110
Undefined-1	Mode	10011111
Undefined-2	Mode	11011110
Reset-Trigger [Remote-Application]	Trigger	01100010
Unknown-3	Trigger	01011101
Unknown-4	Trigger	00100001
Unknown-5	Trigger	10100000

Table 14: Escape mode entry codes [8]

2.2.10 High-Speed Clock Lane Operation

Although very similar to a unidirectional Data Lane, the clock lane operates differently than the Data Lanes. During HS Data Bursts, the clock lane transmits a DDR clock signal in quadrature-phase with the signals on the Data Lanes. The LP operation for a clock lane is different than in Data Lanes and there is no regular Escape Mode functionality. Only the ultra-low power state (ULPS), where the lines are driven to LP-00 to minimize power consumption, is available through a special entry sequence.

The clock lane is controlled by the protocol and only stops sending the clock signal if no HS transmission is active in any Data Lane. The sequence of Lane states for the transition between LP and HS mode on the Clock Lane are mostly the same as on the Data Lanes but with different timing constraints. The Clock Lane will always enter HS mode before the Data Lanes and return to LP mode after the last Data Lane. Figure 21 shows the transitions between modes on the Clock Lane relative to the Data Lane in a single Data Lane D-PHY and illustrates important timing constraints.

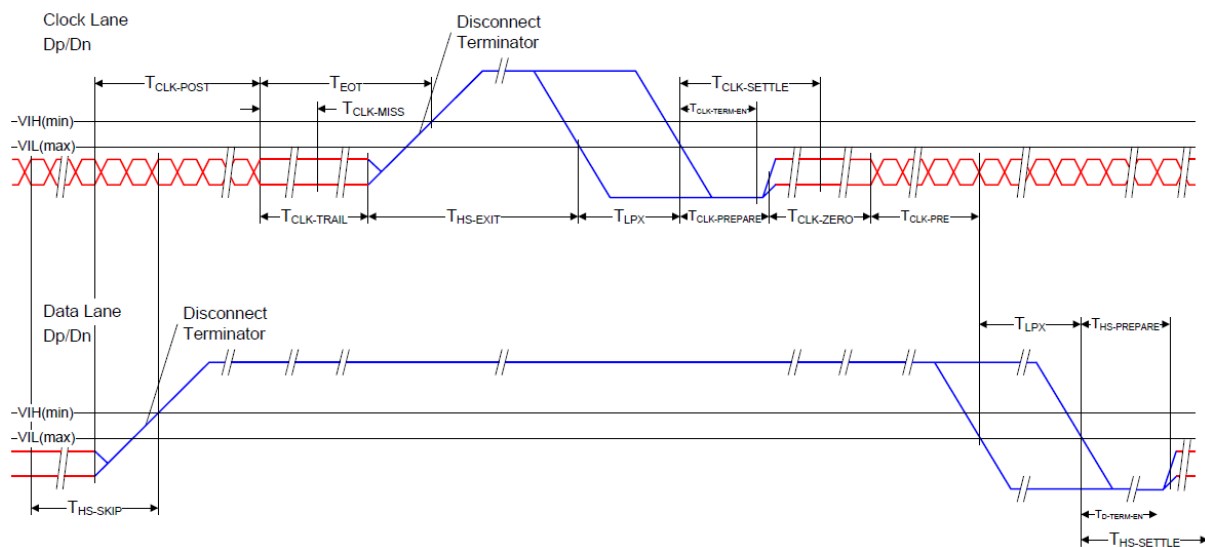


Figure 21: Mode transitions on the clock lane [8]

2.2.11 D-PHY Timing

The timing constraints are crucial for the implementation of a receiver and must be observed. Table 15 contains the important timing parameters, most of which are also illustrated in previous chapters. Many timing parameters are specified only in terms of minimum or maximum durations, allowing a wide range of actual values in any implementation.

Note that most of the timing parameters pertaining to the HS transmissions are defined using so called Unit Intervals (UI), often in combination with fixed time values. A Unit Interval is defined as the duration of any HS state on the Clock Lane and therefore depends on the clock frequency in operation.

Parameter	Description	Min	Typ	Max	Unit	Notes
$T_{CLK-MISS}$	Timeout for receiver to detect absence of Clock transitions and disable the Clock Lane HS-RX.			60	ns	1, 6
$T_{CLK-POST}$	Time that the transmitter continues to send HS clock after the last associated Data Lane has transitioned to LP Mode. Interval is defined as the period from the end of THS-TRAIL to the beginning of TCLK-TRAIL.	60 ns + 52*UI			ns	5
$T_{CLK-PRE}$	Time that the HS clock shall be driven by the transmitter prior to any associated Data Lane beginning the transition from LP to HS mode.	8			UI	5
$T_{CLK-PREPARE}$	Time that the transmitter drives the Clock Lane LP-00 Line state immediately before the HS-0 Line state starting the HS transmission.	38		95	ns	5
$T_{CLK-SETTLE}$	Time interval during which the HS receiver should ignore any Clock Lane HS transitions, starting from the beginning of TCLK-PREPARE.	95		300	ns	6, 7
$T_{CLK-TERM-EN}$	Time for the Clock Lane receiver to enable the HS line termination, starting from the time point when Dn crosses $V_{IL,MAX}$.	Time for Dn to reach $V_{TERM-EN}$		38	ns	6
$T_{CLK-TRAIL}$	Time that the transmitter drives the HS-0 state after the last payload clock bit of a HS transmission burst.	60			ns	5
$T_{CLK-PREPARE} + T_{CLK-ZERO}$	TCLK-PREPARE + time that the transmitter drives the HS-0 state prior to starting the Clock.	300			ns	5
$T_{D-TERM-EN}$	Time for the Data Lane receiver to enable the HS line termination, starting from the time point when Dn crosses $V_{IL,MAX}$.	Time for Dn to reach $V_{TERM-EN}$		35ns + 4*UI		6
T_{EOT}	Transmitted time interval from the start of THS-TRAIL or TCLK-TRAIL, to the start of the LP-11 state following a HS burst.			105 ns + n*12*UI		3, 5
$T_{HS-EXIT}$	Time that the transmitter drives LP-11 following a HS burst.	100			ns	5
$T_{HS-PREAPRE}$	Time that the transmitter drives the Data Lane LP-00 Line state immediately before the HS-0 Line state starting the HS transmission.	40 ns + 4*UI		85ns + 6*UI	ns	5
$T_{HS-PREAPRE} + T_{HS-ZERO}$	THS-PREPARE + time that the transmitter drives the HS-0 state prior to transmitting the Sync sequence.	145 ns + 10*UI			ns	5
$T_{HS-SETTLE}$	Time interval during which the HS receiver shall ignore any Data Lane HS transitions, starting from the beginning of THS-PREPARE. The HS receiver shall ignore any Data Lane transitions before the minimum value, and the HS receiver shall respond to any Data Lane transitions after the maximum value.	85 ns + 6*UI		145 ns + 10*UI	ns	6

T_{HS-SKIP}	Time interval during which the HS-RX should ignore any transitions on the Data Lane, following a HS burst. The end point of the interval is defined as the beginning of the LP-11 state following the HS burst.	40		55 ns + 4*UI	ns	6
T_{HS-TRAIL}	Time that the transmitter drives the flipped differential state after last payload data bit of a HS transmission burst.	Max (n*8*UI, 60 ns + n*4*UI)			ns	2, 3, 5
T_{INIT}	After power-up, the Slave side PHY shall be initialized when the Master PHY drives a Stop State (LP-11) for a period longer than T _{INIT} .	100			μs	5
T_{LPX}	Transmitted length of any Low-Power state period.	50			ns	4, 5
Ratio T_{LPX}	Ratio of TLPX(MASTER)/TLPX(SLAVE) between Master and Slave side.	2/3		3/2		
T_{TA-GET}	Time that the new transmitter drives the Bridge state (LP-00) after accepting control during a Link Turnaround.	5*T _{LPX}			ns	5
T_{TA-GO}	Time that the transmitter drives the Bridge state (LP-00) before releasing control during a Link Turnaround.	4*T _{LPX}			ns	5
T_{TA-SURE}	Time that the new transmitter waits after the LP-10 state before transmitting the Bridge state (LP-00) during a Link Turnaround.	T _{LPX}		2*T _{LPX}	ns	5
T_{WAKEUP}	Time that a transmitter drives a Mark-1 state prior to a Stop state in order to initiate an exit from ULPS.	1			ms	5

Notes:

1. The minimum value depends on the bit rate. Implementations should ensure proper operation for all the supported bit rates.
2. If $a > b$ then $\max(a, b) = a$, otherwise $\max(a, b) = b$.
3. Where $n = 1$ for Forward-direction HS mode and $n = 4$ for Reverse-direction HS mode.
4. T_{LPX} is an internal state machine timing reference. Externally measured values differ slightly from the specified values due to asymmetrical rise and fall times.
5. Transmitter-specific parameter.
6. Receiver-specific parameter.
7. The stated values are considered informative guidelines rather than normative requirements since this parameter is untestable in typical applications.

Table 15: Global timing parameters for D-PHY operation [8]

2.3 HDMI and DVI

The High Definition Multimedia Interface (HDMI) is one of the most widely used interfaces for transmitting video data to digital displays. It is, at its core, an extension of the Digital Visual Interface (DVI) specification, albeit with greatly enhanced performance and entirely new mechanical components, i.e. cables, plugs and sockets. Although it supports high-resolution video transfer, audio transfer and a variety of other features, the relevance of the interface for this project lies in its ubiquity, modularity and its backwards compatibility with the much simpler and openly accessible DVI [4] [1].

Especially in the Open-Source FPGA developer community, bare-minimum HDMI/DVI implementations, that usually only comprise the Transition-Minimized Differential Signalling (TMDS) link, which is used in both DVI and HDMI to transmit the video data, has become a popular way of displaying graphics from an FPGA [12]. This is because such an implementation does not take a lot of resources in the FPGA, and most modern displays, from PC monitors to televisions have an HDMI port and can display at least the minimum required format of 640*480 pixels at a framerate of 60Hz without requiring any additional communication between the FPGA and the display [1]. This section will explain the requirements for such a minimalist “HDMI” implementation.

2.3.1 TMDS General Operation

The core component of both HDMI and DVI is the TMDS-link. TMDS is an interface-standard for the transmission of 24-Bit video data. A standard TMDS-link is made up of four differential pairs, referred to as “channels”, three of which are data channels whereas the last channel transmits the clock signal. Each data channel is assigned eight of the 24 input bits.

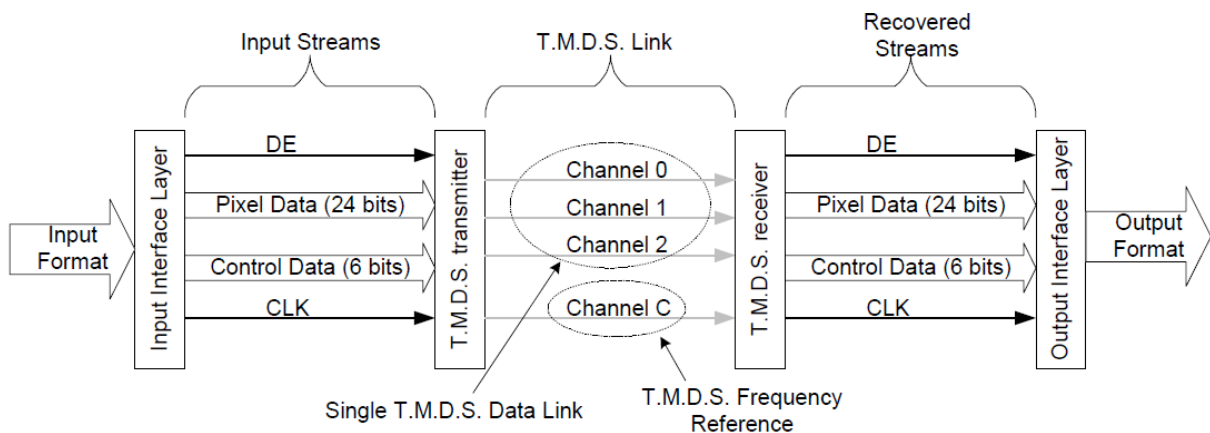


Figure 22: TMDS-link architecture [4]

To achieve a DC-balanced transmission and minimize the number of transitions on the data lines, a type of 8b10b-encoding is used, so that every byte of pixel data is encoded in a 10-bit word. This makes the transmission more reliable at the cost of reducing the effective data rate.

Figure 22 shows the architecture of a TMDS link. Aside from the pixel colour information, a clock signal, a “data enable” (DE) signal and six bits of “control data” are transmitted. Although six bits are reserved for control data, for simple video transmission, only two, “Hsync” and Vsync”, are actually required, while the others can be kept at a “LOW” level.

In Figure 23 the assignment of the signals to the individual channels is depicted with a more detailed view of the components. The link always expects 24 Bits of pixel data, with 24-bit MSB aligned RGB being the default format, which must be supported by any display conforming to the DVI specification [4].

Each channel has two ports for control signals, with the Hsync and Vsync signals being transmitted via Channel 0. The remaining control signals are not used in DVI and are therefore also not necessary for simple video transmissions via HDMI hardware.

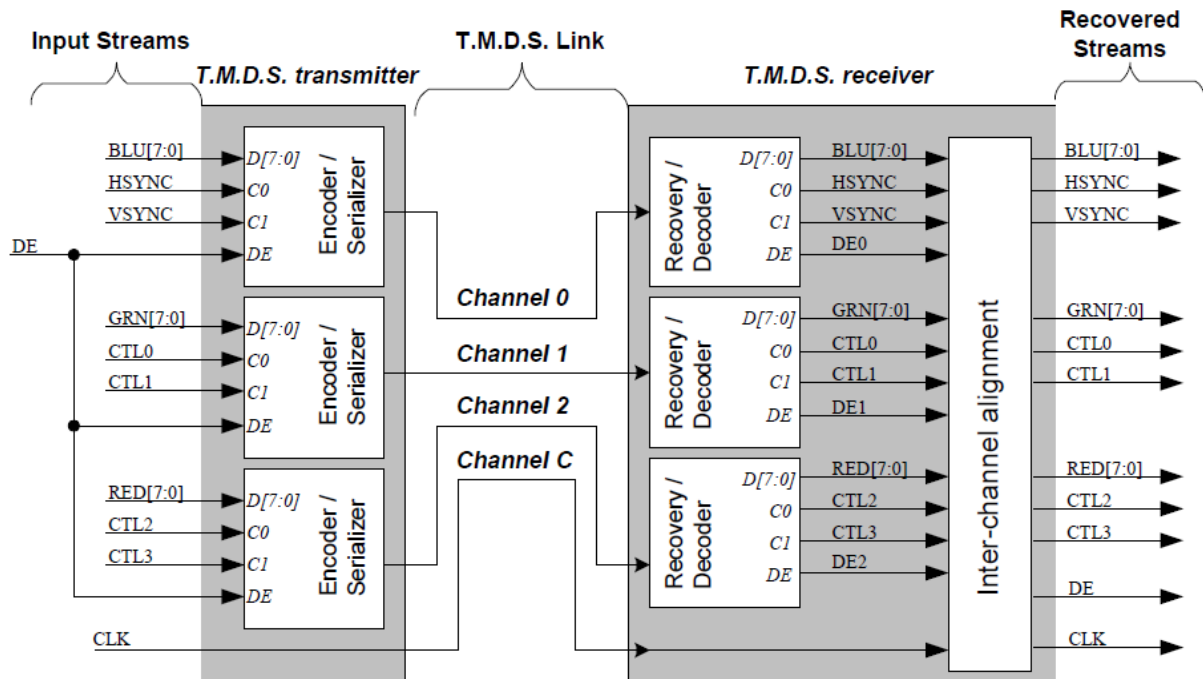


Figure 23: TMDS channel map [4]

Whereas all data channels have an identical structure, consisting of an encoder and serializer on the transmitter side and a decoder/deserializer on the receiver side, the clock channel only transmits the pixel clock, that is one clock cycle for each transmitted pixel. As each pixel requires a 10-bit word to be transmitted per channel, the data channels must be clocked at 10 times the pixel frequency.

2.3.2 TMDS Encoding

The TMDS-Encoder creates 10-Bit data words from the input data, either from the 8-Bit pixel data (8b10b-encoding) or the 2-Bit control data. The state of the DE signal indicates whether pixel or control data is to be encoded. To enable the receiver to differentiate between the two, they are encoded with different amounts of transitions per data word. Pixel data contains five transitions or fewer, control data contains seven or more transitions. This aids the synchronization between transmitter and receiver during the blanking intervals. The exact method of synchronization is not prescribed in the specification, only minimum performance characteristics are defined

In the first step of the encoding process, a 9-bit word is generated from the 8-bit input, with the LSB being the same as in the input byte. The remaining are generated by one of two methods, depending on the number of “1s” in the input byte. If that number is greater than or equal to 4 with the LSB being “0”, the following bits are determined by the XNOR-combination of the current bit from the input with the previously calculated bit of the codeword. In the other case, an XOR-combination is used instead. The MSB of this intermediary codeword indicates which method has

been used. Then, if required for DC-balancing, the codeword may be inverted and finally an additional bit is appended, which indicates whether an inversion was done or not. The codeword is then transmitted serially, with the LSB first.

For the control data signals, the encoding is very simple. Since there are only four possible states for a 2-bit signal, a fixed 10-bit codeword is used for each of these states.

A flowchart of the encoding algorithm is shown in Figure 24, with an explanation of the symbols used given in Table 16.

Symbol	Meaning
D, C0, C1, DE	Input data: D is 8-bit pixel data, C0 and C1 are the channels control data signals, DE is „Data Enable“.
cnt	Register used to keep track of data stream disparity. Positive values represent the number of excess 1s, negative values represent the number of excess 0s. cnt{t-1} indicates the previous value of the disparity for the previous set of input data. Cnt(t) indicates the new disparity with the current set of input data.
q_out	10-bit encoded output value
$N_1\{x\}$	Number of 1s in argument „x“
$N_0\{x\}$	Number of 0s in argument „x“.

Table 16: Symbols for flowchart [4]

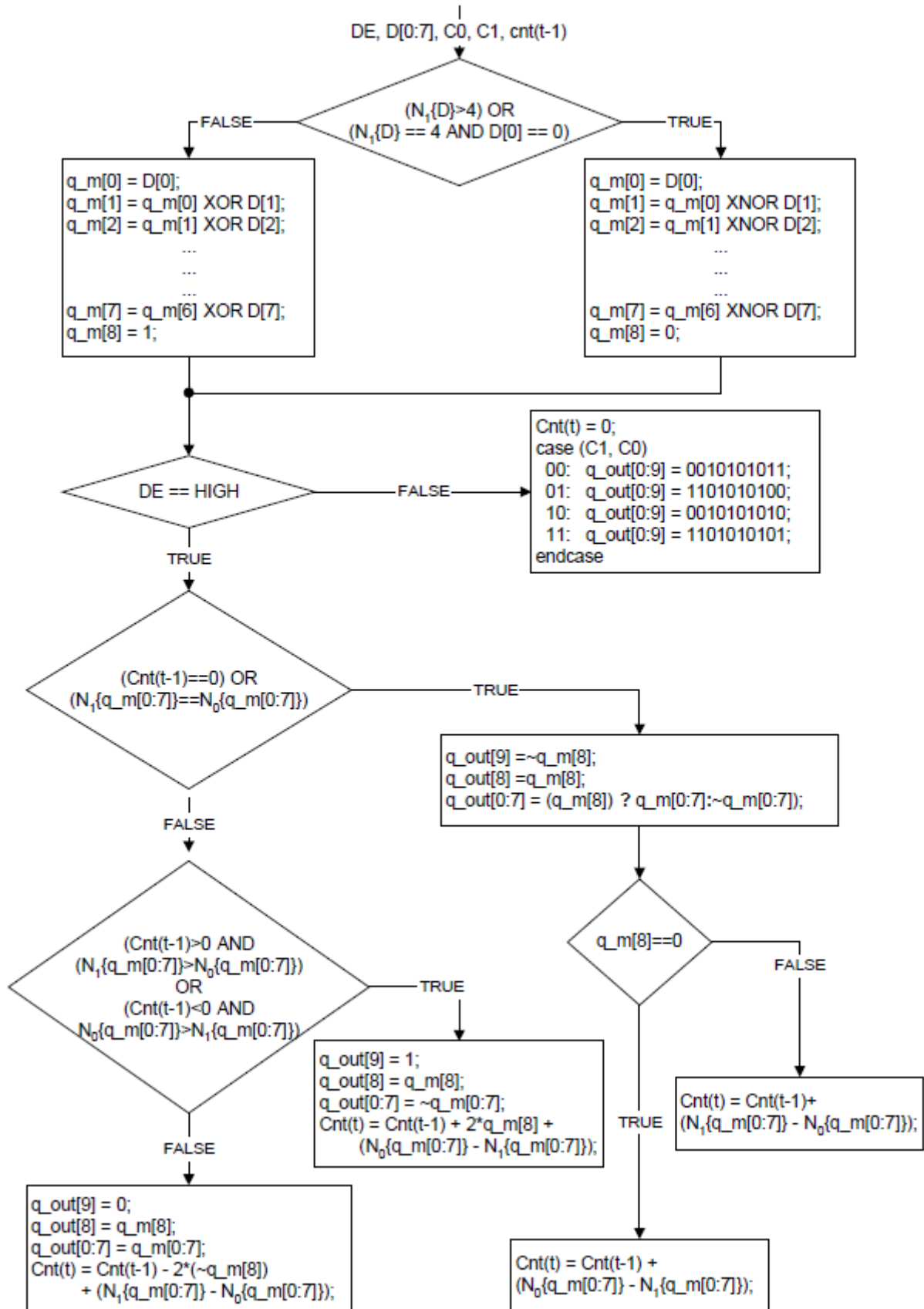


Figure 24: Flow chart of TMDS encoding algorithm [4]

2.3.3 TMDS Physical Layer

TMDS uses current mode logic (CML) with a switched current source on the transmitter side to drive the differential signal. On the receiver side, the HIGH voltage level is set through termination resistors R_T that tie the lines to a reference voltage AV_{CC} . Switching the current source to one of the lines causes a voltage drop V_{swing} across the termination resistance R_T . A simplified diagram of a TMDS differential pair is given in Figure 25. Note that the current introduced by the current source is only indirectly defined, being the ratio of V_{swing}/R_T , normally 10mA.

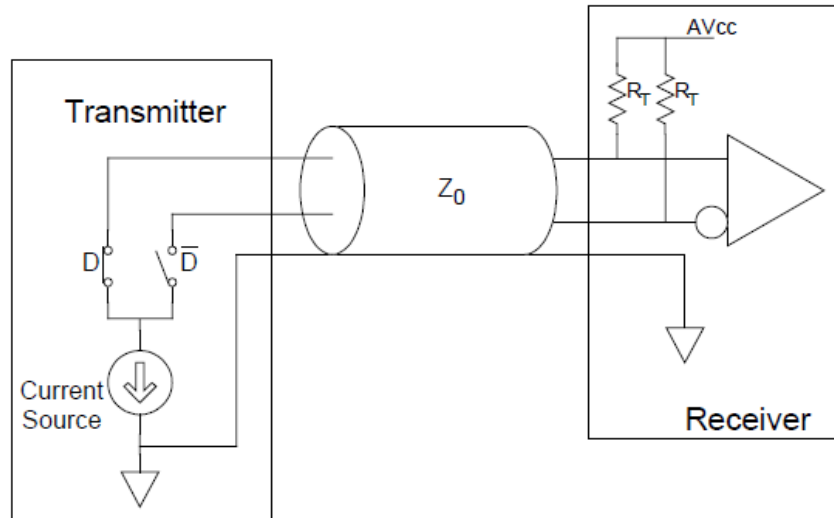


Figure 25: TMDS transmission line [4]

The nominal values for the reference voltage AV_{CC} and the termination resistance are 3.3V and 50Ω, respectively. Nominal and absolute maximum ratings are given in Table 17 and Table 18.

Item	Value
Termination Supply Voltage AV_{CC}	3,3V ±5%
Termination Resistance	50Ω ±10%
Operating Temperature Range	0°C to 70°C

Table 17: Nominal Ratings for TMDS Link [4]

Item	Value
Termination Supply Voltage AV_{CC}	4,0 V
Signal Voltage on Any Signal Wire	-0,5V to 4,0V
Common Mode Signal Voltage on Any Pair	-0,5V to 4,0V
Differential Mode Signal Voltage on Any Pair	± 3,3V
Termination Resistance	0Ω (to open circuit)
Storage Temperature Range	-40°C to 150°C

Table 18: Absolute maximum ratings for TMDS link [4]

2.3.4 Transmitter Specification

Under the conditions specified in Table 17 and Table 18, the transmitter must fulfil the specifications listed in Table 19. T_{bit} and T_{pixel} are the periods of the bit-clock and pixel-clock, respectively, which depend on the transmitted video formats.

Item	Value
Single-ended high level output voltage, V_H	$AV_{CC} \pm 10\text{mV}$
Single-ended low level output voltage, V_L	$(AV_{CC} - 600\text{mV}) \leq V_L \leq (AV_{CC} - 400\text{mV})$
Single-ended output swing voltage, V_{swing}	$400\text{mV} \leq V_{\text{swing}} \leq 600\text{mV}$
Single-ended standby (off) output voltage, V_{OFF}	$AV_{CC} \pm 10\text{mV}$
Risetime/Falltime (20%-80%)	$75\text{ps} \leq \text{Risetime/Falltime} \leq 0.4 T_{\text{bit}}$
Intra-Pair Skew at Transmitter Connector, max	$0.15 T_{\text{bit}}$
Inter-Pair Skew at Transmitter Connector, max	$0.20 T_{\text{pixel}}$
Clock Jitter, max	$0.25 T_{\text{bit}}$

Table 19: Voltage swings and timings for TMDS transmitter [4]

In any allowed operating conditions, all channels must meet the requirements shown in the eye-diagram in Figure 26. Therein, the limits for overshoot and undershoot, relative to the average voltage swing, and the minimum eye-opening are specified, with the time-axis being normalized to the bit time at the testing frequency and the amplitude axis normalized to the average differential swing voltage. The latter is defined as the difference between the average differential amplitude when driving a HIGH-level and the average differential amplitude when driving a LOW-level. The average HIGH-level appears at positive 0.5, the average LOW-level appears at negative 0.5 on the vertical axis. An overshoot of up to 15% and undershoot of up to 25%, relative to the average differential voltage swing are allowed.

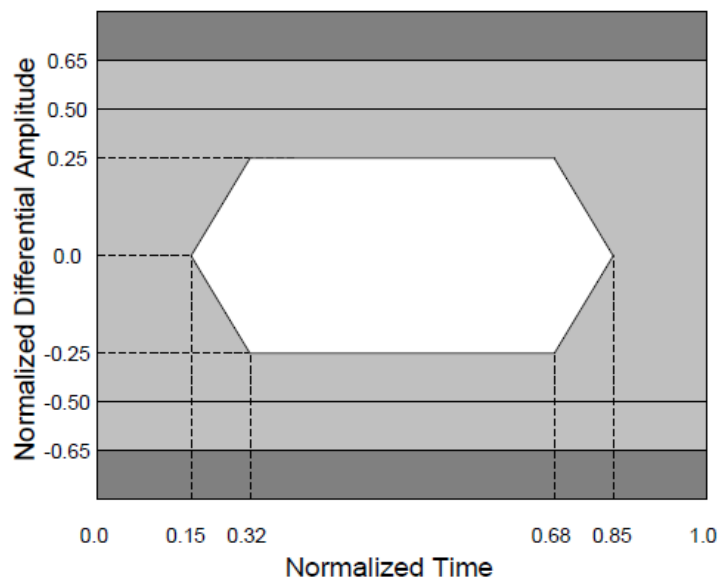


Figure 26: Normalized eye diagram for TMDS transmitter [4]

From the average voltage swing V_{swing} and the tolerances for overshoot and undershoot, the maximum and minimum voltages for HIGH and LOW levels can be calculated according to:

$$V_{high,max} = V_{swing,max} + 15\% * (2 * V_{swing,max}) = 600mV + 180mV = 780mV$$

$$V_{high,min} = V_{swing,min} - 25\% * (2 * V_{swing,min}) = 400mV - 200mV = 200mV$$

$$V_{low,max} = -V_{swing,max} - 15\% * (2 * V_{swing,max}) = -600mV - 180mV = -780mV$$

$$V_{low,min} = -V_{swing,min} + 25\% * (2 * V_{swing,min}) = -400mV + 200mV = -200mV$$

2.3.5 Receiver Specification

Table 20: TMDS receiver characteristics [4]

Item	Value
Differential Input Voltage, V_{idiff}	$150 \leq V_{idiff} \leq 1200 \text{ mV}$
Input Common Mode Voltage, V_{icm}	$(AV_{cc} - 300) \leq V_{icm} \leq (AV_{cc} - 37)$
Behavior when Transmitter Disabled or Disconnected	$AV_{cc} \pm 10 \text{ mV}$
Minimum differential sensitivity (peak-to-peak)	150 mV
Maximum differential input (peak-to-peak)	1560 mV
Allowable Intra-Pair Skew at Receiver Connector	0.4 Tbit
Allowable Inter-Pair Skew at Receiver Connector	0.6 Tpixel
TDR Rise Time	75 ps
Exception_window	500 ps
Through_connection	$100 \pm 20 \Omega$
At Termination	$100 \pm 10 \Omega$

The receiver must satisfy the specification given in Table 20 in all allowed operating conditions for a TMDS-Link. It must be able to reproduce a test data stream with the amplitude given in Figure 27 with a pixel error rate of 10^{-9} .

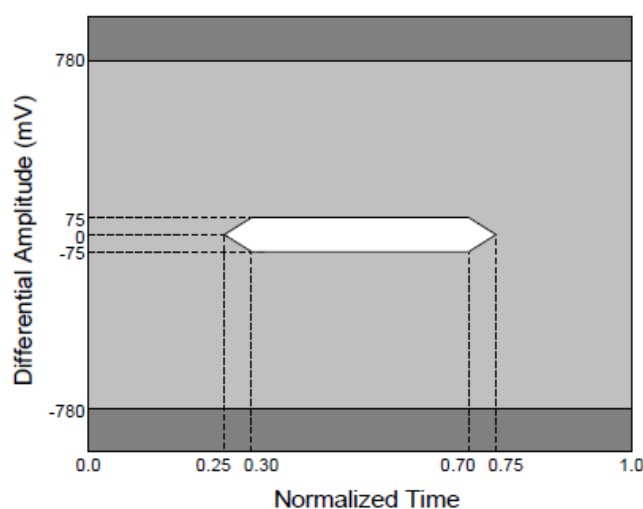


Figure 27: Normalized eye diagram for TMDS receiver [4]

2.4 Camera Module

2.4.1 Digital Camera Basics

Digital cameras use arrays of sensing elements to convert light into electrical signals proportional to the intensity of the incoming light to create digital images. To obtain the colour information, it is necessary to have sensing elements that are sensitive to at least two different wavelengths (or colours) of light. The most common way to achieve this is to simply overlay the sensor matrix with a corresponding matrix of colour filters. Generally, the three fundamental colours red, green and blue are used in analogy to the receptors in the human eye. Figure 28 shows the spectral sensitivity of the sensory cells in a human eye, where the curves corresponding for the three types of “cone” cells that are responsible for colour vision are labelled S, M and L respectively. The dotted curve, labelled R, represents the spectral sensitivity of the “rod” cells that are primarily relevant for vision in low-light environments and play only a minor role in colour-vision [13].

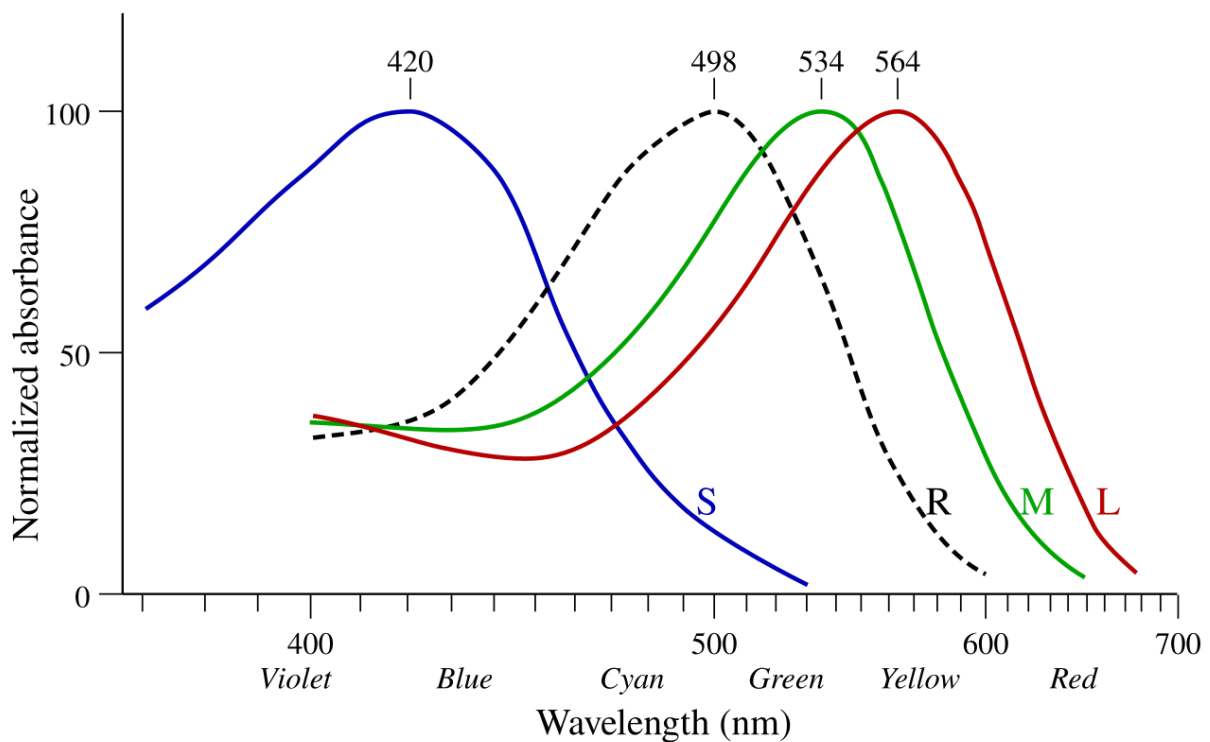


Figure 28: Normalized wavelength-dependent absorbance of sensory cells in the human eye [13]

The most popular filter matrix using red, green and blue used in digital camera chips is called the Bayer-filter and is illustrated in Figure 29. The Bayer-filter uses four sensor elements for one full-colour pixel, of which one for red, one for blue and two for green. The green component is given priority over the other colours because it has the most impact on the human brightness perception.

As a result of this patterning the effective resolution for colour images is lower than the number of sensor elements. However, this effect can be reduced by interpolating the missing colours for each sub-pixel from the neighbouring sub-pixels in subsequent processing steps. Another important consequence of the Bayer-filter is that the arrangement of colour filters on the sensor must be known in order to interpret the raw sensor data produced by most camera chips correctly.

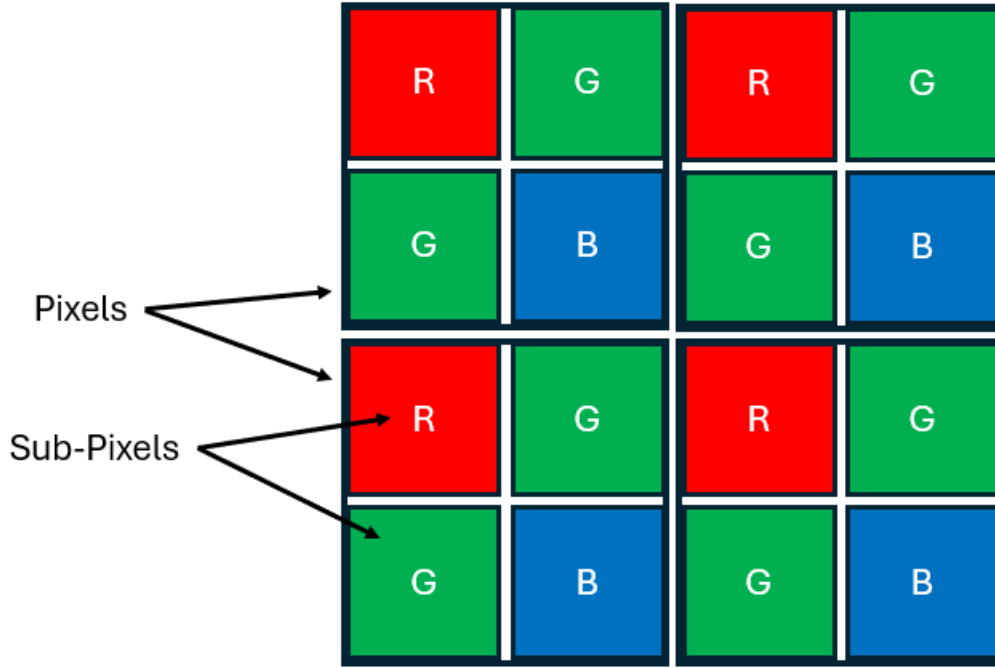


Figure 29: Illustration of Bayer filter pixel matrix

2.4.2 Raspberry Pi Camera Module V2

For the validation of the physical layer implementation, a camera utilizing the MIPI CSI-2 interface is used. For that purpose, it makes sense to choose a well-known camera module for which sufficient documentation is available to avoid unnecessary complications in its operation. The widely used Raspberry Pi Camera Module 2 fulfils these criteria, with the additional bonus of being cheap and easily available. The module employs a Sony IMX219 CMOS image sensor with a CSI-2 output. Although the chip is capable of using up to four data lanes, the module only supports two lane operation, as the remaining output lanes are not connected [9] [14].

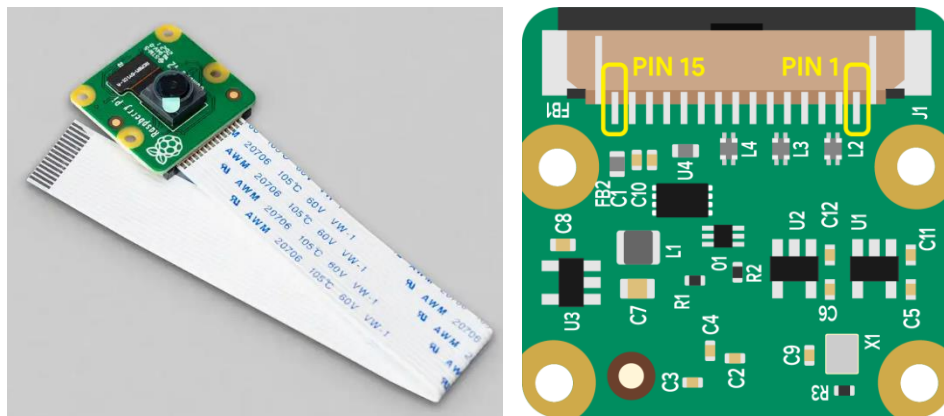


Figure 30: Raspberry Pi Camera 2 and pin numbering (right) [15]

As depicted in Figure 30, the camera module uses a single 15-pin, single row flexible printed circuit (FPC) cable for all outside connections. Eight of these pins are dedicated to the CSI-2 interface, with six pins for the three D-PHY lanes and two more for the I²C-compatible camera control interface. The remaining pins are used for the 3.3V supply voltage, a power enable signal and ground connections. The exact pin allocation is detailed in Table 21. The LED indicator assigned to pin 12 is only available on some variants of the module.

Pin	Description
1	Ground
2	CSI Data Lane 0 Negative
3	CSI Data Lane 0 Positive
4	Ground
5	CSI Data Lane 1 Negative
6	CSI Data Lane 1 Positive
7	Ground
8	CSI Clock Lane Negative
9	CSI Clock Lane Positive
10	Ground
11	Power Enable
12	LED Indicator
13	DDC SCL
14	DDC SDA
15	3.3V VCC

Table 21: Pin allocation on the Raspberry Pi Camera 2 connector [15]

It should be noted that neither the CSI-2 nor the D-PHY specification require a dedicated connector. Therefore, camera modules using CSI-2 can have different connectors and pinouts and a universal design cannot be assumed. However, most MIPI camera modules use FPC cables. The 15-Pin variant used for the Raspberry Pi Camera Module 2 is still very popular, due to its use on the Raspberry Pi single board computers but is limited in bandwidth as only two data lanes can be reasonably implemented without incurring a high risk of crosstalk issues [8]. Consequently, for higher performance camera modules, FPC cables with 22 or more pins are used, that support four or more data lanes [16].

2.5 GateMate FPGA

GateMate is a low-cost FPGA family from Cologne Chip AG, based in Cologne, Germany. Currently there are two variants, the A1 and A2, with the A1 being the base variant and the A2 consisting of essentially two A1 dies in a single package. An important benefit of this concept is that both variants are pin-compatible on a package level, meaning that the same PCB-footprint can be used for either variant, although the A2 variant may not be fully utilized when used on a board designed for the A1 [17]. The GateMate A1 and A2 use the same 15mm*15mm BGA package with 324 connections. This means that, although the evaluation board for this project is designed for the GateMate A1, it could also potentially be used with the more powerful GateMate A2, if desired.

Device	rel. size	Cologne Programmable Elements		Block RAMs ^{*3}			
		CPEs ^{*1 *2}	FF / Latches	20 Kb	40 Kb	PLLs	SerDes
CCGM1A1	1	20,480	40,960	64	32	4	1
CCGM1A2	2	40,960	81,920	128	64	8	2

^{*1} CPEs have 2 x 4 or 8 inputs connected to a LUT-tree
^{*2} Each CPE can be used as a 2 x 2 multiplier tile
^{*3} Block RAM can have a data width of 1..80 bits

Device	GPIOs		Package	
	single-ended	differential	balls	size [mm]
CCGM1A1	162	81	324 BGA	15 x 15
CCGM1A2	162	81	324 BGA	15 x 15

Figure 31: Overview and comparison GateMate A1 and A2 [17]

The fundamental building block of the GateMate series is a newly designed programmable element, called the Cologne Programmable Element (CPE), the general structure of which is depicted in Figure . It comprises a LUT tree with eight inputs and two outputs, two flipflops/latches as well as resources for signal routing. Each element can be configured as a 1- or 2-bit full adder or a 2x2-bit multiplier, which can be combined to form adders or multipliers of arbitrary size.

The GateMate A1 features 20480 CPEs, including 40960 flipflops/latches, four PLLs, 32 RAM blocks of 40Kb each, a dedicated SERDES interface, as well as 162 GPIOs in 81 I/O-cells, which can be individually configured to operate in single-ended or differential mode. The I/O-cells are arranged in eight I/O-banks that can use individual supply voltages between 1.2V and 2.5V. A simplified overview of the placement of components on the die is given in Figure . The A2 variant, being two A1 dies in one package, offers twice these resources, although the number of usable I/O-banks is the same.

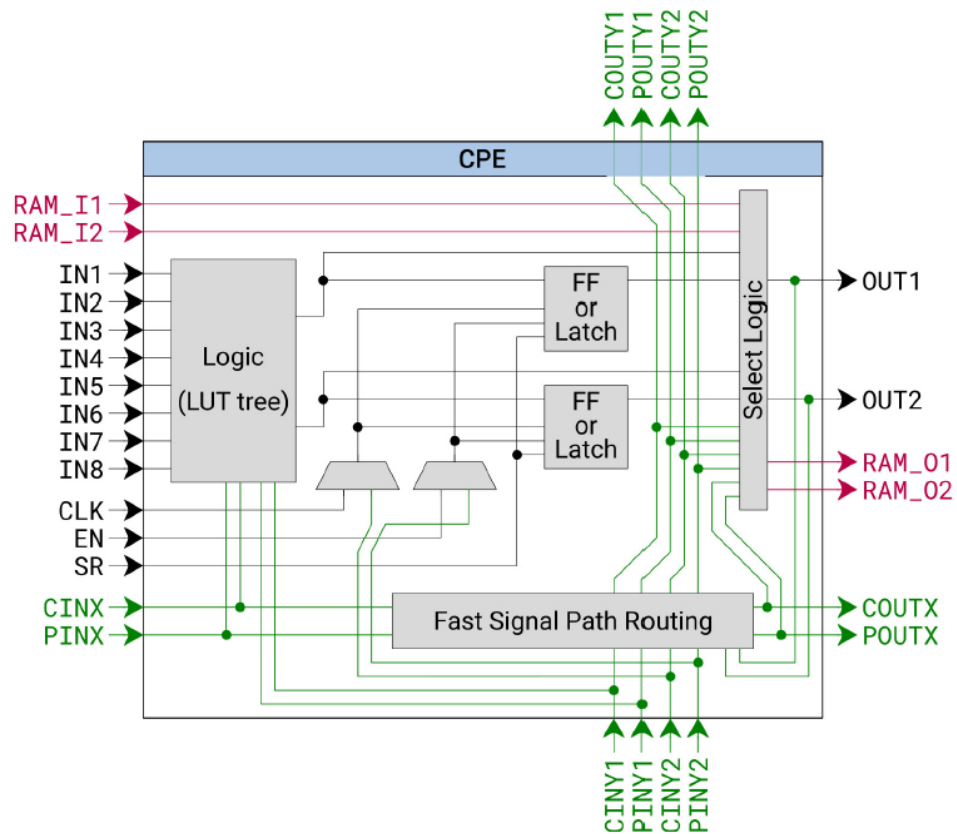


Figure 32: CPE architecture [17]

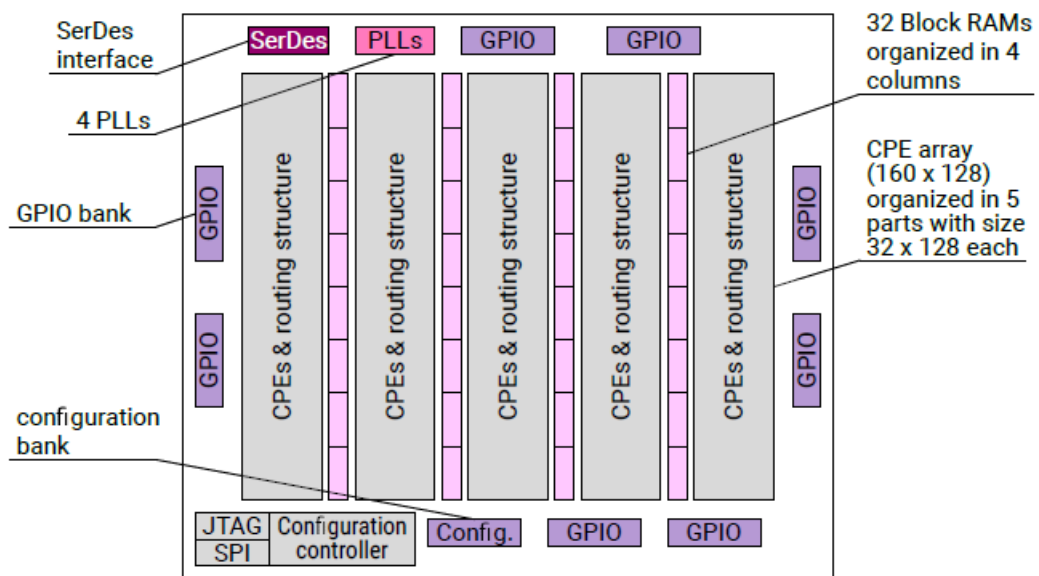


Figure 33: Simplified architecture overview GateMate A1 [17]

In this project, the I/O-cells of the GateMate have a special significance, as these have to be configured in a way that enables the reception D-PHY signals. Therefore, the possible configurations of the GateMate's I/O-cells must be examined with regard to their suitability for this task.

The GPIO pins of the GateMate are grouped into pairs, each of which is tied to a so-called “pad-cell”, which can be configured either as a pair of single-ended GPIOs or one differential pair. A simplified diagram of a pad-cell in LVDS mode is shown in Figure 34.

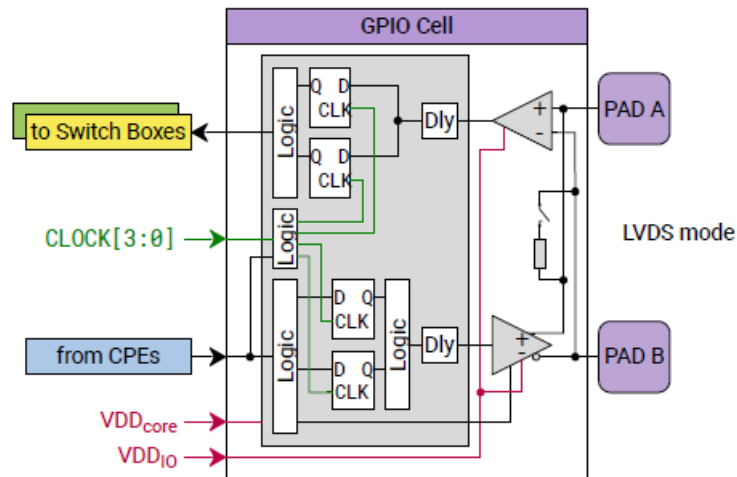


Figure 34: GateMate GPIO cell in LVDS mode [17]

The supported standards are LVCMOS with a supply voltage between 1.2V and 2.5V for the single-ended configuration and 2.5V LVDS, with the option to work with supply voltages down to nominally 1.8V, for the differential configuration. As described previously, D-PHY uses the same pair of lines for two different types of transmission, namely the low-power mode, where the two lines of a lane are used in a single-ended configuration and evaluated as a pair of signals, and the high-speed mode, in which each lane is used as a differential pair. This presents a problem for the GateMate FPGA, because the pad-cells can only be configured for one I/O-standard at a time. Consequently, it is necessary to connect each signal pair of the D-PHY interface to two separate pairs of I/O-pins, that are located in different I/O-banks to allow for different supply voltages. This way, one pair can be configured for the reception of the single-ended low-power signals, while the other is configured as an LVDS receiver.

Another issue that needs to be addressed, is the termination for the signals. According to the D-PHY specification, the differential termination between the signal lines of each lane is supposed to only be switched on for the high-speed transmissions. Although the GateMate's pad-cells feature optional on-chip 100Ω termination for the LVDS receiver, it can't be switched on and off arbitrarily during runtime. This means that, during low-power transmissions, whenever there is a LP-01 or LP-10 state, there will be a voltage drop across this termination of nominally 1.2V, which is a lot higher than the 350mV differential voltage used in LVDS and might damage the internal termination resistor. Thus, it is advisable to use an external termination resistor, which may not only be more resistant to such damage, but also easily replaceable.

There are notable differences between the LVDS specification the GateMate's pad-cells are designed for and the specification of differential high-speed signals in D-PHY, primarily in the common-mode voltage and the differential voltage, both of which are significantly lower in the D-PHY transmissions. According to the manufacturer however, the LVDS receivers in the pad-cells

are capable of reliably receiving such signals, so that there is no need to employ any kind of level-shifting network to achieve compatibility.

Besides the hardware, an important component for FPGA designs is software used to synthesize the actual FPGA configuration from an HDL design and program the FPGA accordingly. These are usually proprietary software tools, often integrated with a full development suite, including text editors, simulators and more [18]. For the GateMate such an integrated software suite does not yet exist. Instead in designing with the GateMate, a collection of open-source tools is used. For linting of the HDL code and the synthesis of the design, Yosys is used, where a netlist is generated, which is then used by a proprietary place and route tool, which maps it to the GateMate hardware and creates the bitstream for programming the device. For simulations, any general-purpose HDL simulator can be used.

The GateMate has a dedicated I/O-bank for configuration, called the “configuration bank”, which features an SPI interface, that can be used either in active or passive mode, a JTAG interface and some additional control and status signals.

The GateMate can be configured in four different ways. The SPI interface allows the FPGA to read configuration data from a flash memory (SPI active mode) or receive configuration data from an external processor (SPI passive mode). In addition, the GateMate also features a JTAG interface, enabling configuration via a JTAG test and programming adapter. The fourth option is the FPGA receiving configuration from previously configured logic within itself, which can, for example, be used for partial reconfiguration in active operation but is obviously not suitable for initial configuration.

The configuration bank with the pins for the JTAG and SPI interfaces is located in the “south-west” corner of the chip. To select the configuration mode to be used, there are four I/O-pins labelled “CFG_MODE”, that are evaluated as a 4-bit input. There are a total of 9 different modes, eight of which are different SPI configurations. The seven remaining possible bit patterns are invalid and should therefore be avoided.

CFG_MD[3:0] * Configuration mode			
0x0	0b 0000	SPI Active Mode	CPOL = 0, CPHA = 0
0x1	0b 0001	SPI Active Mode	CPOL = 0, CPHA = 1
0x2	0b 0010	SPI Active Mode	CPOL = 1, CPHA = 0
0x3	0b 0011	SPI Active Mode	CPOL = 1, CPHA = 1
0x4	0b 0100	SPI Passive Mode	CPOL = 0, CPHA = 0
0x5	0b 0101	SPI Passive Mode	CPOL = 0, CPHA = 1
0x6	0b 0110	SPI Passive Mode	CPOL = 1, CPHA = 0
0x7	0b 0111	SPI Passive Mode	CPOL = 1, CPHA = 1
0xC	0b 1100	JTAG	

* Modes not mentioned in the list may not be selected and lead to a malfunction of the device.

Table 22: Configuration mode setup [17]

Table 22: Configuration mode setup lists all valid configuration mode settings available for the GateMate. An overview of all ports of the configuration bank is shown in Table 23.

By means of a JTAG-SPI bridge in the FPGA, the configuration data can simultaneously be stored in an external flash memory with an SPI interface, when the JTAG interface is used to configure the GateMate.

Pin	Signal name	Reset category	I/O mode	I/O characteristic	Description
R5	CFG_MD0	4	I		Configuration mode bit 0
T5	CFG_MD1	4	I		Configuration mode bit 1
U4	CFG_MD2	4	I		Configuration mode bit 2
V4	CFG_MD3	4	I		Configuration mode bit 3
U1	POR_EN	2	I	pull-down	Enable power-on reset
V3	CFG_DONE	2	O	pull-down, low, driver strength 12 mA	Configuration done signal
V2	CFG_FAILED_N	2	I/O	pull-up, high, open drain, driver strength 12 mA	Configuration failed signal
N3	SPI_CS_N	3	I, I/O	pull-up, driver strength 12 mA	Configuration SPI chip select
N4	SPI_CLK	3	I, I/O	driver strength 12 mA	Configuration SPI clock
P2	SPI_D0	3	I, I/O	driver strength 12 mA	Configuration SPI data bit 0
P1	SPI_D1	4	I		Configuration SPI data bit 1
R2	SPI_D2	4	I		Configuration SPI data bit 2
R1	SPI_D3	4	I		Configuration SPI data bit 3
R4	SPI_FWD	2	O	low, driver strength 12 mA	Configuration SPI data forward
T3	JTAG_TMS	4	I		JTAG test mode select
R3	JTAG_TCK	4	I		Configuration JTAG clock
T2	JTAG_TDI	4	I		JTAG data input
U2	JTAG_TDO	2	O	low, driver strength 12 mA	JTAG data output

Reset categories:

0: No reset (supply or analog input pin)

1: Pin disabled, high-z

2: Pin characteristic already during reset state

3: Pin characteristic depends on the configuration mode (SPI master: I/O, else: input)

4: Input pin

Table 23: Configuration bank signals [17]

3 Evaluation Board

This chapter deals with the design, assembly and initial testing of a new evaluation board, developed for this project. In the first section, the required and optional components and characteristics of the evaluation board are discussed and design constraints formulated. The second section deals with the actual design and assembly, followed by an outline of the initial functional tests and a report of the results and findings from those tests.

3.1 Requirements and design constraints

The first step in designing an evaluation board is to formulate the specific requirements and design constraints. This minimizes the risk of unwelcome surprises later in the design process. The most obvious component is the GateMate FPGA at the centre of the design, which requires additional components to operate, primarily a stable, low-noise power supply. Additionally, a crystal oscillator as a clock source and a flash memory to store configuration data are required. For communication between the GateMate and a PC, there are two options. The first is to place a USB controller IC and the accompanying auxiliary components on the board, the other is to make use of the “GateMate FPGA Programmer”, a small PCB containing the hardware for interfacing the GateMate via USB [19]. The latter option is chosen, because it requires fewer components on the evaluation board, reducing the complexity of the design.

In addition, components specific to the video application, including the camera and display connectors, are required. There are some peculiarities to consider for both of these, when working with the GateMate FPGA. In case of the camera connector, it is crucial that the pin pairs assigned to the MIPI CSI-2 lanes are each connected to four I/O-pins on two different I/O-banks of the GateMate. This is necessary because the two different signalling standards used in the D-PHY interface require different supply voltages at the receiver and each I/O-bank can only have one supply voltage. Also, several resistors need to be connected to the signal lines, as termination resistors and to provide some degree of decoupling between the different I/O-pins. For the video output, an HDMI connector is chosen. This also requires some additional components on the board, which are discussed in the next chapter.

Finally, some GPIOs shall be connected to generic connectors, allowing for any miscellaneous function that may be desired, such as status signals for debugging during development.

Setting some constraints and priorities at the start of the design process can help speed up the development and prevent issues like choice paralysis and scope creep. For the evaluation board developed in this project the highest priority was to guarantee the correct function of the GateMate and to minimize the effects of the PCB layout on the signal integrity of the high-speed communication lines, namely the CSI-2 interface and the HDMI interface.

Other, less rigid design targets were to keep the PCB reasonably small and cheap to fabricate, ensure the components can be mounted reliably and with reasonable effort, without requiring advanced equipment. Therefore, all components should be placed on the top side of the PCB, not be smaller than the (imperial) 0402 package format and have enough space around them to allow for manual placement. Besides that, the PCB should have no more than four layers, as higher layer counts mean increased cost and complexity.

3.2 Schematic Design

The design of both schematic and PCB layout is done using the ECAD software “Altium Designer”, which is a professional tool used extensively in commercial electronics design. This is useful not only for the various advanced features of the software, but also because the design files of the official GateMate evaluation board are freely available in the Altium Designer data format, which serves as a valuable reference [20] [21].

Many elements of the design, both in the schematic and PCB layout, are taken from the official GateMate Evaluation Board by Cologne Chip. This way it can be ensured that all the necessary components for operating the GateMate are present in the design and interconnected and placed in a way that has been proven to work well, reducing development time and risk compared to an entirely new design.

First, all components that aren’t needed from the official evaluation board are removed from the schematic and the new components, such as the connectors for the camera, the HDMI output and the GPIOs, along with smaller components like capacitors and resistors are added and connected appropriately.

The schematics for the whole evaluation board are fairly extensive, so in this section only selected parts of it are discussed in detail.

3.2.1 I/O-Banks

The GateMate’s eight I/O-banks are distributed along the edges of the chip, with the corresponding package pins being located no more than four pins away from the package edge.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	
A	GND	VDD_WC	IO_NA_A8	IO_NA_A1	VDD_NA	IO_NA_A4	GND	IO_NA_A7	IO_NB_B0	GND	IO_NB_B2	IO_NB_B4	GND	IO_NB_B7	IO_EB_B8	VDD_EB	IO_EB_B5	GND	A
B	IO_WC_A8	IO_WC_B8	IO_NA_B0	IO_NA_B1	IO_NA_A2	IO_NA_B4	VDD_NA	IO_NA_B7	IO_NB_A0	VDD_NB	IO_NB_A2	IO_NB_A4	VDD_NB	IO_NB_A7	IO_EB_A8	GND	IO_EB_A5	VDD_EB	B
C	GND	VDD_WC	IO_WC_A7	IO_WC_B7	IO_NA_B2	IO_NA_A3	IO_NA_A5	IO_NA_A6	IO_NA_A8	IO_NB_B1	IO_NB_B3	IO_NB_B5	IO_NB_B6	IO_NB_B8	IO_EB_B7	IO_EB_B6	IO_EB_B4	IO_EB_A4	C
D	IO_WC_A5	IO_WC_B5	IO_WC_A6	IO_WC_B6	VDD_WC	IO_NA_B3	IO_NA_B5	IO_NA_B6	IO_NA_B8	IO_NB_A1	IO_NB_A3	IO_NB_A5	IO_NB_A8	IO_NB_A7	IO_EB_A6	IO_EB_A7	IO_EB_B2	IO_EB_A2	D
E	IO_WC_A3	IO_WC_B3	IO_WC_A4	IO_WC_B4	GND	VDD_NA	GND	VDD_NA	GND	VDD_NB	GND	VDD_NB	GND	VDD_EB	IO_EB_B3	IO_EB_A3	VDD_EB	GND	E
F	GND	VDD_WC	IO_WC_A2	IO_WC_B2	VDD_WC	GND	VDD_NA	GND	VDD	GND	VDD_NB	GND	VDD_EB	GND	IO_EB_B1	IO_EB_A1	IO_EB_B0	IO_EB_A0	F
G	IO_WC_A0	IO_WC_B0	IO_WC_A1	IO_WC_B1	GND	VDD	GND	VDD	GND	VDD	GND	VDD	GND	VDD	IO_EA_B8	IO_EA_A8	IO_EA_B7	IO_EA_A7	G
H	IO_WB_A7	IO_WB_B7	IO_WB_A8	IO_WB_B8	VDD_WB	GND	VDD	GND	VDD	GND	VDD	GND	VDD	GND	IO_EA_B6	IO_EA_A6	IO_EA_B5	IO_EA_A5	H
J	GND	VDD_WB	IO_WB_A6	IO_WB_B6	GND	VDD	GND	VDD	GND	VDD	GND	VDD	GND	VDD	IO_EA_B5	IO_EA_A5	IO_EA_B4	IO_EA_A4	J
K	IO_WB_A5	IO_WB_B5	IO_WB_A4	IO_WB_B4	VDD_WB	GND	VDD	GND	VDD	GND	VDD	GND	VDD	GND	IO_EA_B3	IO_EA_A3	IO_EA_B2	IO_EA_A2	K
L	IO_WB_A3	IO_WB_B3	IO_WB_A2	IO_WB_B2	GND	VDD	GND	VDD	GND	VDD	GND	VDD	GND	VDD	IO_EA_B1	IO_EA_A1	VDD_EA	GND	L
M	GND	VDD_WB	IO_WB_A1	IO_WB_B1	VDD_WB	GND	VDD	GND	VDD	GND	VDD	GND	VDD	IO_EA_B0	IO_EA_A0	GND	IO_SB_A3	IO_SB_B3	M
N	IO_WB_A0	IO_WB_B0	SPI_CS_N	SPI_CLK	VDD_WA	VDD	GND	VDD	GND	VDD	VDD_SB	GND	VDD_SB	IO_SB_A8	IO_SB_B8	N.C.	GND	VDD_SB	N
P	SPI_D1	SPI_D0	VDD_WA	GND	VDD_WA	VDD_SA	GND	VDD_SA	GND	VDD_SA	IO_SB_A4	IO_SB_A7	IO_SB_B7	IO_SB_A6	IO_SB_B6	VDD_PL1	IO_SB_A2	IO_SB_B2	P
R	SPI_D3	SPI_D2	JTAG_TCK	SPI_FWD	CFG_MD0	IO_SA_A1	IO_SA_A2	IO_SA_A4	IO_SA_A6	IO_SA_A7	IO_SB_B4	GND	IO_SB_A5	IO_SB_B5	VDD_SB	GND	IO_SB_A1	IO_SB_B1	R
T	VDD_WA	JTAG_TDI	JTAG_TMS	GND	CFG_MD1	IO_SA_B1	IO_SA_B2	IO_SA_B4	IO_SA_B6	IO_SA_B7	GND	SER_CLK	SER_CLK_N	VDD_CLK	RST_N	VDD_SER_PLL	GND	VDD_SB	T
U	POR_EN	JTAG_TDO	VDD_WA	CFG_MD2	IO_SA_A0	IO_SA_A3	IO_SA_A5	IO_SA_A8	IO_SA_B8	SER_RX_P	VDD_SER	SER_TX_P	GND	GND	GND	GND	IO_SB_A0	IO_SB_B0	U
V	GND	CFG_FAILED_N	CFG_DONE	CFG_MD3	IO_SA_B0	GND	IO_SA_B3	IO_SA_B5	GND	IO_SA_B8	SER_RX_N	SER_RX_TERM	SER_TX_N	GND	POR_ADJ	GND	VDD_SER	GND	V

Figure 35: GateMate A1 BGA pinout [17]

To illustrate, the pinout of the GateMate A1 is shown in Figure 35, as it appears in the PCB footprint, as viewed “through” the chip from above.

The I/O-banks are named after the cardinal directions, north, east, south and west, with two banks for each direction. Each I/O-bank has separate input pins for the supply voltage. In the “south-west” corner of the package there is a special I/O-bank for configuration signals, comprising the pins for the SPI and JTAG interfaces, the “configuration mode” inputs (CFG_MD0 to CFG_MD3), the “power-on reset enable” (POR_EN) as well as the “configuration done” and “configuration failed” status signals. The configuration bank is supplied via the VDD_WA pins.

Beside the I/O-banks, supply-voltage and ground pins, there are also dedicated I/O-pins connected to the integrated SERDES module of the GateMate. These are not intended to be used with the evaluation board designed here, so they are left unconnected, with the exception of the supply voltage, which is recommended to always be connected to avoid interference from floating nets within the chip [17].

The signals assigned to the general purpose I/O-banks will significantly affect the design of the trace layout later on, so this is the primary criterion for this decision. Unnecessary crossing of different signal traces should be avoided, as that could negatively affect the signal integrity and complicate the routing process. Therefore, every functional unit, such as the camera input or the HDMI output and the GPIO connections is assigned to a different I/O-bank. The camera connector is a special case here, because two different supply voltages may be needed to receive both the High-Speed and Low-Power signals of the D-PHY transmissions. This necessitates using two separate I/O-banks for each group of signals. To be able to test different configurations, the supply voltages are designed to be adjustable. This is achieved with jumpers, connecting the I/O-bank’s supply voltage connections to either a 2.5V or a 1.2V supply net.

With the exception of configuration signals, which connect to a dedicated bank, the components can be arbitrarily assigned to any I/O-bank. Ultimately the components are assigned as follows: The camera interface is assigned the north banks, NA and NB, the HDMI connector is assigned to the eastern bank EB, the GPIO connections to the southern banks SA and SB, while the western banks remain unconnected, except for the supply voltage. This determines the general layout, as illustrated in Figure 36.

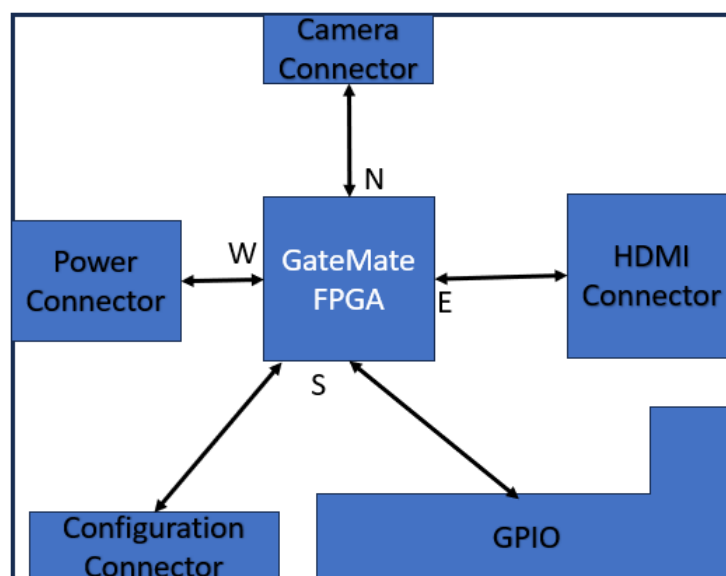


Figure 36: Sketch of connector layout for the evaluation board

3.2.2 Power Supply

A fundamental aspect of designing an evaluation board for an FPGA is ensuring a reliable and steady voltage supply to the chip. Since the chip uses several different supply voltages for different functions, several voltage regulators are required. These supply voltages include the supply for the I/O-banks, which can vary between 1.2V and 2.5V, depending on the application, the “core” voltage, which ideally is adjustable on an evaluation board and can range from nominally 0.85V to 1.15V for the GateMate [17], as well as a 3.3V supply to power external devices that require this supply voltage, such as the camera module. A mini-USB type B connector with the data lines unconnected is used as a power connector, so that a PC or any 5V USB power supply can deliver power to the board. This central supply voltage is primarily used as an input to the voltage regulators delivering the other supply voltages but is also needed for the 5V port of the HDMI connector.

The remaining components on the PCB are powered with one of the other, lower supply voltages, which are generated by MPM3833C switch-mode voltage regulators [22]. These regulators are also used on the official GateMate evaluation board because of their low output noise generation and good regulation capability. Other benefits are high efficiency compared to linear regulators, a small footprint on the PCB and ease of use. Due to the integrated inductor, only capacitors for smoothing input and output voltages and a voltage divider for setting the output voltage are required, as illustrated in Figure 37 and Figure 38.

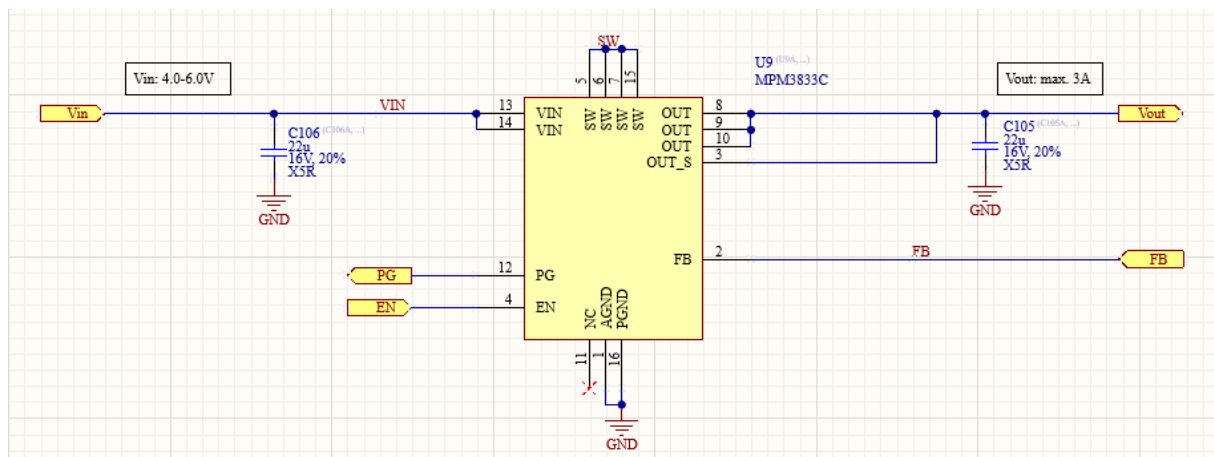


Figure 37: Schematic of a voltage regulator

As can be seen in Figure 38, the regulators are daisy chained with their “enable” inputs (EN) and their “power good” outputs, so that the regulators will start up one after another. The last signal in the chain is used to switch an LED, indicating that all regulators are working.

Another requirement for ensuring a stable supply voltage is sufficient filtering for transient spikes in the power drawn by the FPGA. As the FPGA contains lots of potentially fast switching components, it is crucial to add sufficient decoupling capacitors. Here it is important to keep in mind the parasitic properties of real capacitors, primarily inductance. In general, higher capacitance leads to higher parasitic inductance, which makes using a single large decoupling capacitor ineffective for filtering out high frequency noise. Beside the parasitic inductance of the capacitor itself, the inductance of the traces connecting it to the rest of the circuit must also be considered [23].

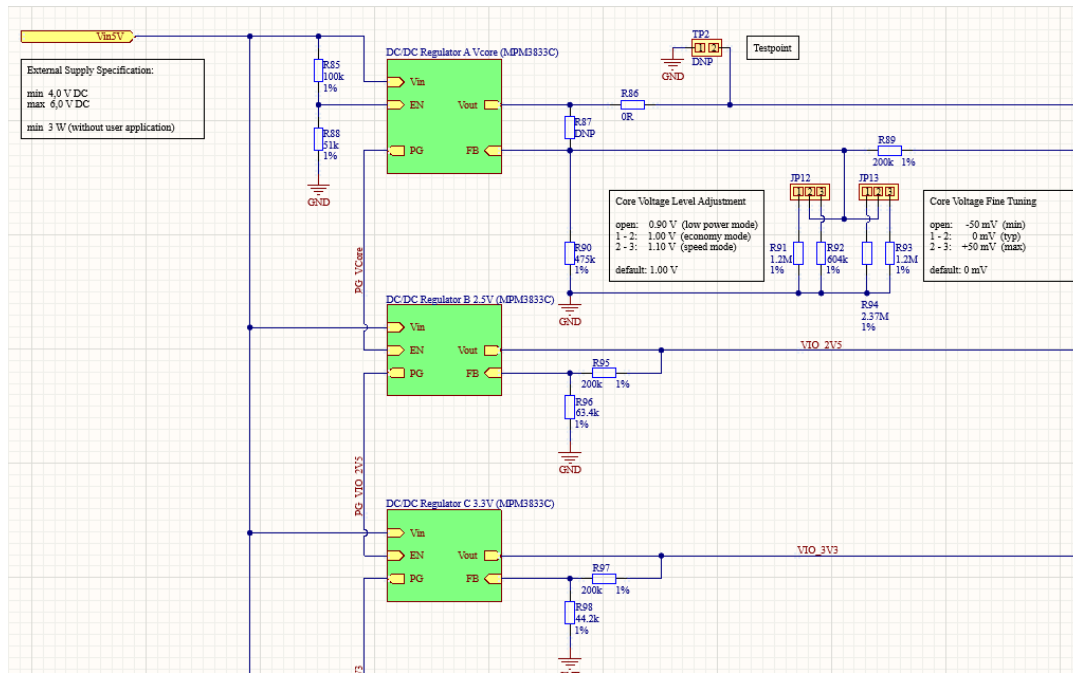


Figure 38: Schematic showing the wiring scheme of the voltage regulators

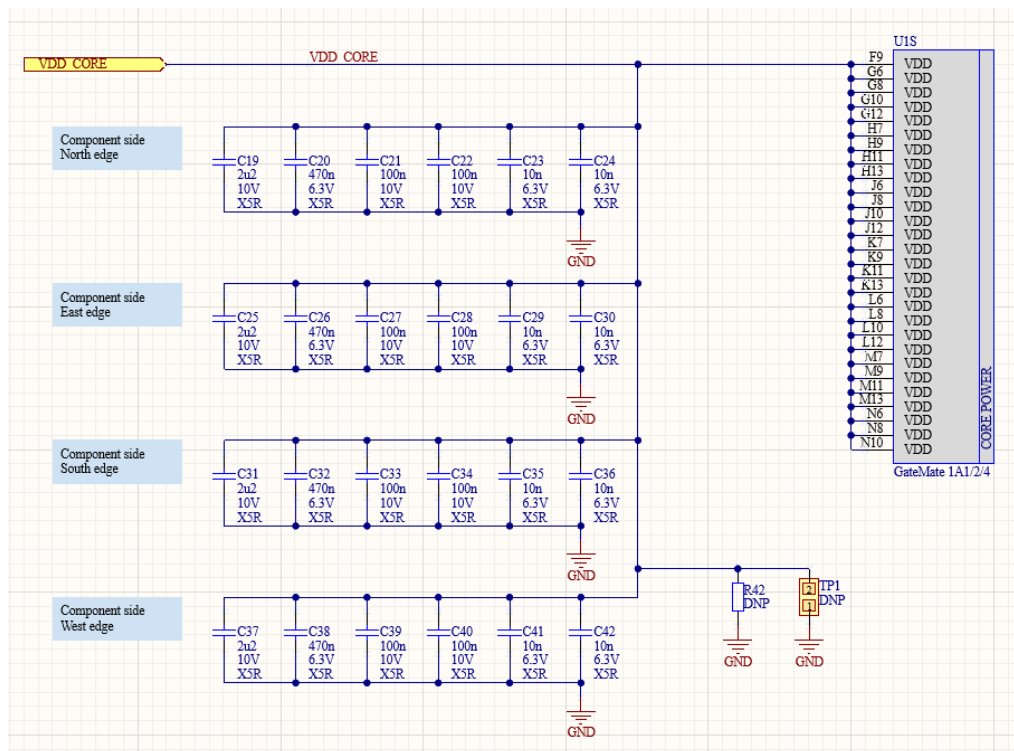


Figure 39: Capacitor banks for core voltage smoothing

Therefore, a common practice is to use several capacitors in parallel, usually of different capacitance values, the smallest being placed closest to the load to minimize the inductance, and thus HF impedance, of the connecting trace. This is also done here, although the choice of components is taken from the official GateMate Evaluation board. To illustrate, Figure 39 shows the capacitors for stabilizing the core supply voltage. Similar constructs are used for every I/O-bank, however, with only four capacitors for each bank. Even though the load on some of the I/O-banks is expected to be minimal, an optimization of this part of the design is not reasonable for a prototype design, as the potential cost and space savings are miniscule.

3.2.3 Camera Connection

The most important component of the new evaluation board is the camera connector that enables the reception of the video input via the CSI-2 interface. Besides its importance for the project, another detail to consider in designing this part is the remaining uncertainty about what is actually required to enable the GateMate to properly receive the signals. There are several reasons for this, as touched on in previous chapters. The LVDS receivers on the GateMate are designed and tested according to the LVDS specification, and although they can receive signals beyond the LVDS specification, this has not been extensively tested and certified. This leaves some uncertainty regarding the exact performance characteristics when working with signals outside the LVDS specification, especially at operating frequencies in the hundreds of MHz, which the GateMate, being a low-cost general-purpose FPGA, is not specifically optimized for.

Another point of uncertainty is how much of the CSI-2/D-PHY functionality is actually required for a functioning basic receiver design. A variety of advanced/optional features are specified for D-PHY, which are not going to be needed here. Crucially, there are also features that are required for compliance with the specification but may not technically be necessary for a functioning receiver, such as the Escape Mode, Low-Power Data Transmission or the ultra-low power state [8]. Many of these features serve to facilitate additional communication between camera and host processor and are not required for simple video transmission. It may even be possible to achieve a reliable receiver without implementing the LP-Mode of the D-PHY interface at all, which would eliminate the need to use two pairs of input pins in different configurations for each lane. In order to be able to test different configurations and compare their performance, the connections on the evaluation board are designed to accommodate an external resistor network, based on [10], for each D-PHY lane, as shown in Figure 40. Beyond the main D-PHY lanes, the CSI-2 interface also includes the “Camera Control Interface” (CCI), which is based on and largely compatible with the I²C-standard. This interface is used to configure the camera module and therefore essential to its operation, so it must be ensured that this works correctly. Since I²C uses pull-up resistors on both signal lines, an adequate voltage must be chosen to supply the circuit. The camera module uses a 1.8V supply on these lines but also features a simple level-shifting circuit, so that higher voltages can be used on the lines outside the module [14]. As the module is designed to be compatible with the 3.3V used by the RaspberryPi GPIOs, it is decided to use the maximum available voltage of 2.5V for the corresponding pins of the GateMate.

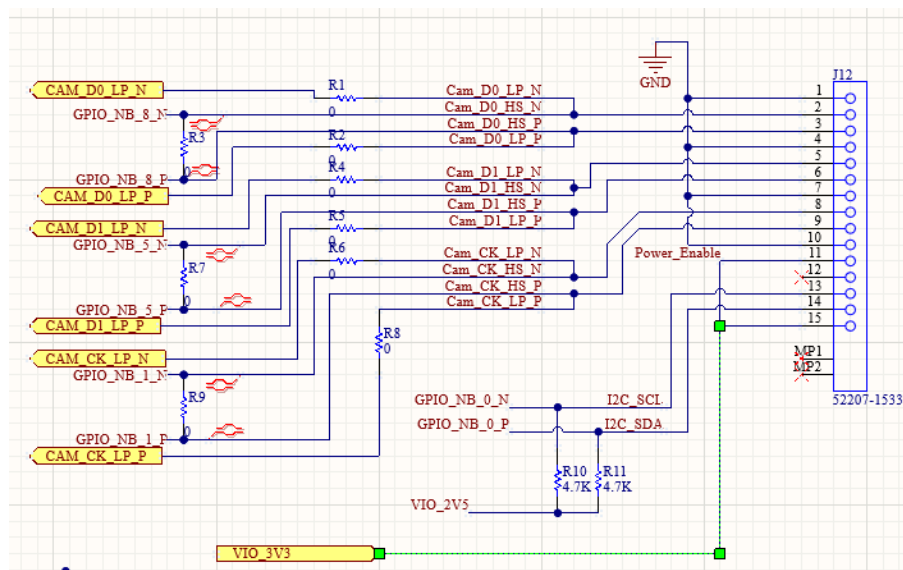


Figure 40: Schematic of camera connector wiring

3.2.4 HDMI Output

A simple and effective way to verify the reception of video data is to display the received data. Therefore, the evaluation board is designed to be able to output video data to a display. As already mentioned in section HDMI and DVI, one way to accomplish that is by using a minimalist implementation of an HDMI connection, utilizing the required compatibility of HDMI-displays with certain video formats, even without any additional communication. This means that an HDMI-connector can be used, of which only the pins for the differential pairs of the TMDS-interface are connected to the FPGA's I/O-ports.

While some recent FPGAs have built-in support for a wide range of I/O-standards, including TMDS [24], the GateMate is quite limited in this regard, only offering compliant transceiver hardware for LVCMOS and LVDS [20]. However, in many cases it is possible to use adapter circuits, like level-shifting networks, to exchange data with devices using different I/O-standards. Such solutions may not be compliant with the exact specifications for a given interface, but if sufficient compatibility can be identified the required performance can be achieved with rather simple solutions.

Since the GateMate lacks dedicated TMDS drivers, an implementation of a TMDS transmitter requires some external circuitry to interface the LVDS output of with a TMDS receiver. A simple solution is to use AC-coupling between the LVDS transmitter and the TMDS receiver. This is achieved by adding series capacitors on both lines of the differential pair, as shown in Figure 41. It is important that the capacitance of these coupling capacitors is high enough to properly transmit the lowest operating frequency. For the evaluation board, MLCCs with a capacitance of 1 μ F are used, according to the recommendation from Cologne Chip. This solution has been successfully tested by Cologne Chip prior to this. Besides that, the ground connection pins must be connected to the ground plane and the "+5V" pin to the 5V input voltage. The resulting network is shown in Figure 41.

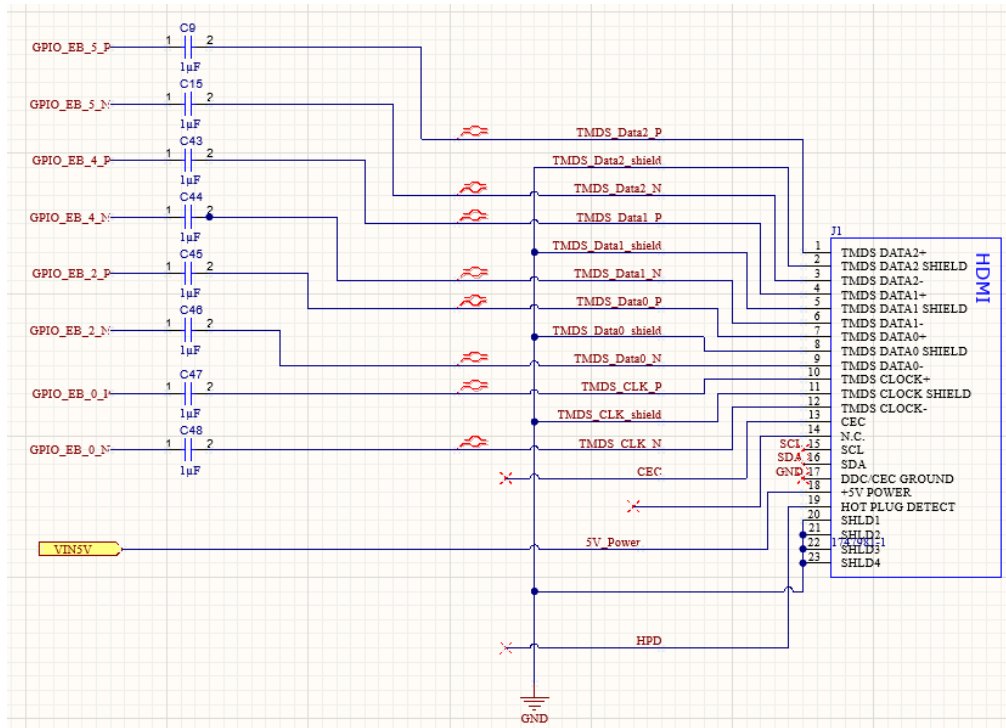


Figure 41: Schematic of HDMI connector wiring

3.2.5 Miscellaneous Components

To allow configuration and proper operation of the GateMate, some additional components are needed. Beside the connector that provides access to the configuration pins, some more components are required and useful, namely an external oscillator that serves as a clock reference, a flash memory chip, to store configuration data, several status LEDs giving information about the availability of supply voltages and the configuration status and lastly, some logic for handling external reset signals.

The GateMate Programmer is connected to the board via a generic 16-pin surface-mounted header, of which eight pins carry signals, while the remaining eight pins are tied to GND, as shown in Figure 42.

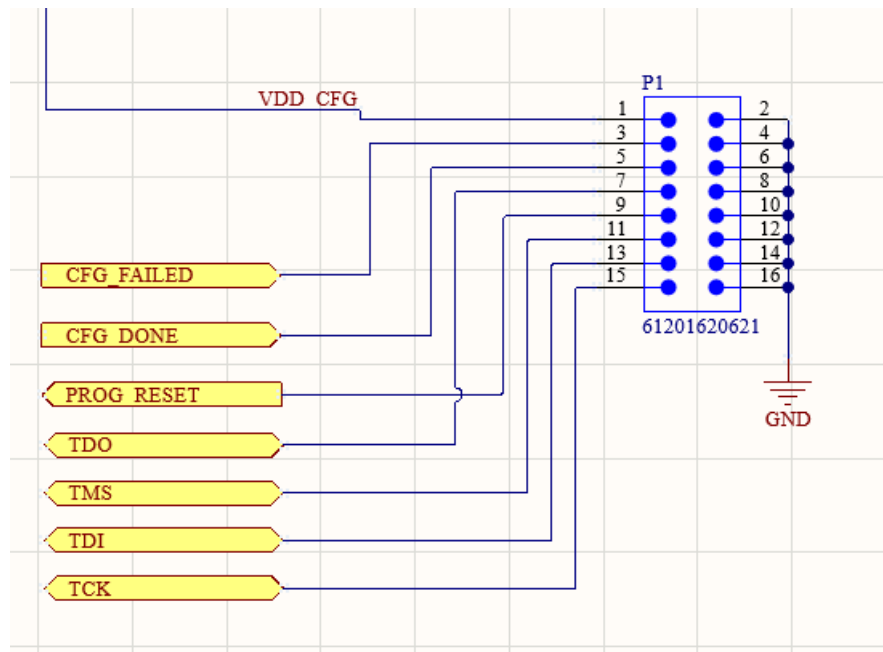


Figure 42: Schematic of programmer connector wiring

For the configuration to work, the correct configuration mode must be set using the “CFG-Mode” inputs of the GateMate. To enable this, an array of jumpers is used, that allows connecting these inputs to either 2.5V supply voltage or GND. Since the programmer is used to configure the FPGA, the mode will almost always be JTAG. For this reason and to use as little board space as possible, small solder jumpers are chosen.

The flash memory for storing configuration data is an MX25V8035FM1I chip from Macronix and is connected directly to the SPI interface of the GateMate.

The GateMate has a dedicated active-low reset input, that can be used to trigger a hardware reset. On the new evaluation board, such a reset shall be triggered by either of three events: The programmer sending a reset signal, the voltage regulators pulling down the “power good” signal (PG) and a push button, so that the user can manually trigger a reset at any time. This is achieved by combining these input signals with NAND-gates, as shown in Figure 44. Note that the SN74LVC08 IC, containing four NAND-gates is oversized for this application, as it was retained from the official GateMate evaluation board, where more input signals were used. The unused inputs are tied to the supply voltage through pull-up resistors.

VIO 2V5

Power Good

PROG RESET

U2E SN74LVC08

R54 200k

R55 10k

R56 10k

R57 10k

U2D SN74LVC08

U2A SN74LVC08

U2C SN74LVC08

U2B SN74LVC08

R53 200R

D15 red

FPGA RESET

FPGA RESET IN

C73 100m 10V XSR

GND

SW2 red

PTS645-SK43

C74 100m 10V XSR

Reset can be triggered by user (SW2), external JTAG or via signal from the voltage regulators.

Finally, for the GPIO pins from the SA and SB banks it is decided to use connectors according to the PMOD specification by Digilent, in order to enable the use of peripheral modules using this interface [25]. This interface uses 12-pin-connectors where two pins are connected to a 3.3V supply, two pins to GND and the remaining eight pins to GPIO ports of the FPGA. However, because the maximum GPIO voltage of the GateMate is 2.5V and many PMOD modules can function with a 2.5V supply voltage, the 3.3V pins are instead connected to 2.5V. This way no level shifters are required. Although some peripheral modules may not function properly with this setup, this is an acceptable compromise between compatibility and design effort.

3.3 PCB Layout

Once the schematic is finished, the layout of the PCB can be designed, considering the design constraints and priorities defined earlier, such as small board size, low cost and good performance of critical components, without making manual mounting of components too difficult. Having the official GateMate evaluation board as a reference greatly simplifies some aspects of the design, such as determining the amount of required copper layers and the minimum/maximum widths of copper traces, especially in the region of BGA footprint.

In the following, key aspects of the design will be presented and the reasoning behind these design choices explained.

3.3.1 Layer Stack and board outline

At the start of the PCB layout design a crucial consideration is the layer stack. This includes defining the number of copper layers and what function they will be assigned. For this project, it is determined, based on the official GateMate evaluation board, that four layers will be sufficient. Of these, the top and bottom layers will be the signal layers, while the inner layers will be used as a ground plane and a dedicated power distribution layer.

The thicknesses of the copper and insulation layers are important for the design and, especially, the manufacture of the PCB. As there are no traces expected to carry currents large enough to cause significant heating and no voltages above 5V are used, the thicknesses of the layers are determined primarily by their impact on manufacturability and cost. For the outer layers, which will necessarily feature trace widths and clearance as small as 0.1mm, a thickness of $18\mu\text{m} + 20\text{-to-}30\mu\text{m}$ is chosen, according to the recommendation of the PCB manufacturer. For the inner layer the standard thickness of $35\mu\text{m}$ is chosen. The chosen dielectric material is the standard FR4-material used by the manufacturer because it's cheapest and no special performance requirements exist for the dielectric.

Finally for the surface finish, an electroless nickel/immersion gold (ENIG) finish is chosen because it is the cheapest option offered by the manufacturer that guarantees sufficiently flat surfaces for soldering fine-pitch BGA's. Other advantages are its decent shelf life and resistance to mechanical damage, which is useful when ordering backup PCBs that may only be required after some time, in case of technical problems [26].

With the layer stack defined, the actual layout of the board can now be designed. For an evaluation board, intended for use mostly by itself, the shape is not very important, so as is common with such boards, a rectangular, almost square shape is chosen. This fits the square shape of the FPGA chip, so that it can be placed relatively close to the centre and the traces can fan out from there to the other components towards the edges of the PCB.

3.3.2 Component Placement

The rough placement of the components has already been sketched in Figure 36 and is decided primarily by the location of the corresponding I/O-banks of the FPGA. Going off the naming scheme for the I/O-banks, in the following, the cardinal directions are used to describe the positions of components on the PCB, with north being on top in the images of the layout. Accordingly, the camera connector is placed on the north edge of the PCB, the HDMI connector on the east edge, the pseudo-PMOD GPIO connectors on the east and south, the programmer connector on the western part of the south edge, and the power connector on the western edge.

The decoupling capacitors for the supply voltages need to be placed as closely as possible to the device they are associated with (for most of them that is the FPGA), to keep the connections short and the associated parasitic resistances and inductances low. Other parts of the power supply, the voltage regulators and jumpers, are not constrained in this way, in part because the dedicated power distribution layer offers plenty of room for routing wide and thus low-impedance traces. Thus, the voltage regulators are placed along the western edge of the board, where there is plenty of space, because the western GPIO-banks of the FPGA are not used. The status LEDs, the reset logic and the flash memory are also placed on the western part of the PCB for this reason. The remaining components, such as the jumpers and the reference oscillator, are placed wherever they fit.

3.3.3 Escape Routing

One of the most critical parts of the PCB layout is the escape routing for the BGA footprint of the GateMate. Due to the small distances between the pads, fine traces and clearances, around 0.1mm are required. To minimize the risk of critical design flaws, the escape routing structure is initially copied from the official Evaluation Board and then adapted to take into account the peculiarities of the new evaluation board, mostly by deleting many connections to I/O-banks that are not used on the new board. The space that is made available this way is used to widen voltage supply lines and optimize the routing of critical signal traces for the video interfaces.

Soldering BGA packages can be difficult without specialized equipment, due to the tight tolerances for placing the component and limited possibilities for inspection, so a special footprint is chosen for the FPGA, that contains additional clearance and mounting holes for the use of a socket. This socket can replace the solder connections by pressing the solder balls of the BGA package to the copper pads. Because of the limited availability and high cost of this particular socket, it is meant as a fallback option in case any problems arise with soldering the BGA package.

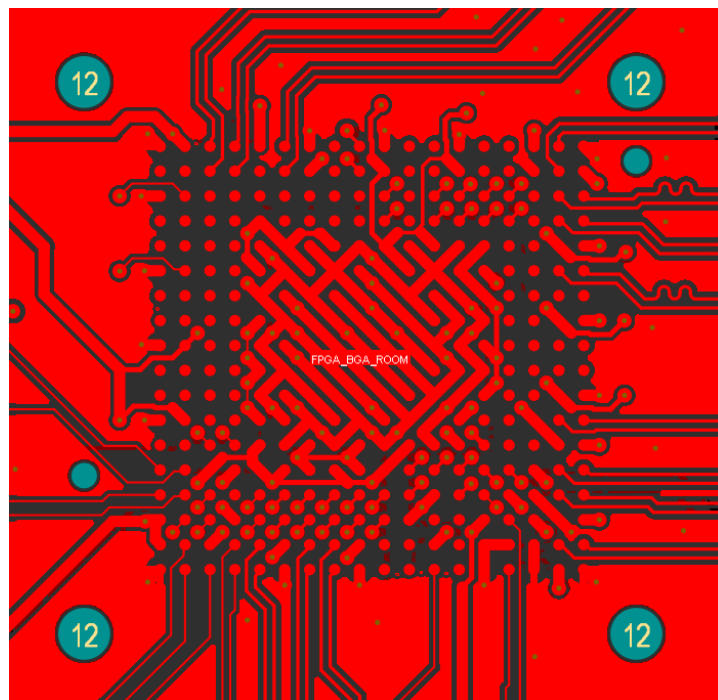


Figure 45: FPGA escape routing on top layer, including copper plane. The holes (marked “12”) are for mounting an optional socket.

3.3.4 Camera and HDMI Connections

The camera connection, and to a lesser extent the HDMI connection, is the most important part of this PCB and is also expected to carry the highest frequency signals of all traces on the board, with clock frequencies well above 100MHz. Thus, optimizing the associated traces with regard to signal integrity is essential. It is also important to implement test points on all the lines, ideally as close to the FPGA as possible, to provide easy access for measurements.

One of the most significant parameters regarding signal integrity is the trace impedance. For mostly straight PCB-traces with a ground plane underneath, the high-frequency impedance is dictated primarily by the geometry, namely the thickness and width of the trace and the thickness of the dielectric between trace and ground plane. For differential pairs the distance between the two traces in the pair is another crucial factor [27]. Altium designer offers several features that help with designing the traces correctly and routing differential pairs with a constant distance between the traces and tightly matched lengths. With these tools the optimal values for trace width and intra-pair distance for a given layer stack can be easily calculated and input to the routing tool. The software also calculates the expected signal delay for a given trace length, so that the intra- and inter-pair delay can be easily matched, which becomes increasingly important with higher frequencies.

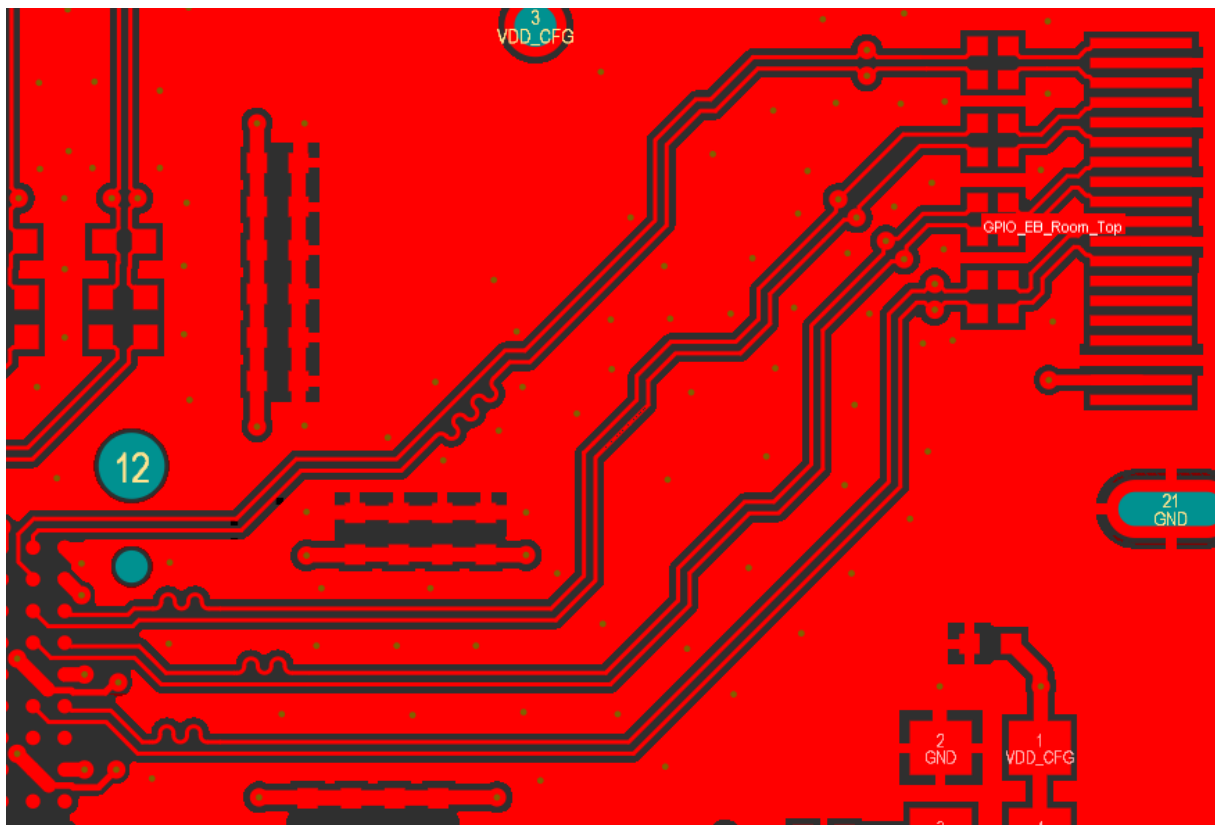


Figure 46: Differential pairs connecting FPGA and HDMI connector

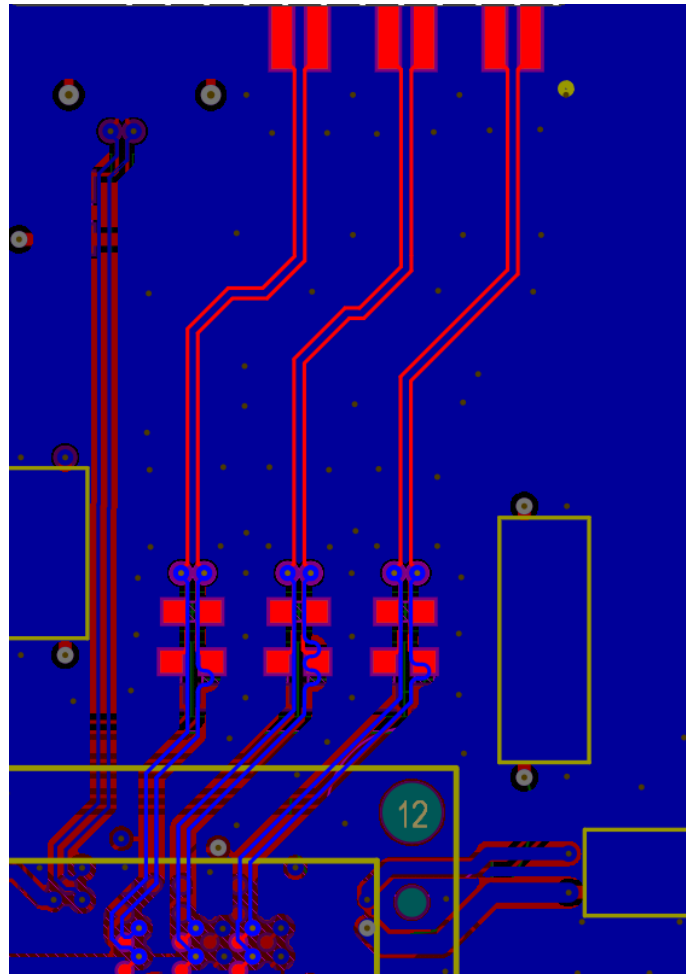


Figure 47: Differential pairs connecting FPGA and camera connector

Figure 46 and Figure 47 depict the differential pairs connecting the FPGA to the HDMI connector and the camera connector, respectively. The trace and gap widths for the differential pairs are calculated using Altium Designer's integrated impedance calculator, so that the trace impedance for the differential pairs is around 100Ω . Great care is taken to match the signal delays on the differential pairs. That entails matching the lengths of the traces within a pair, which is done by careful routing and adding "accordion patterns" in one of the traces, to lengthen it. This way, the intra-pair delays (as calculated by Altium designer) differ less than 2ps. Inter-pair matching is only required for traces connecting to the same components, so all TMDS pairs and all D-PHY pairs. Here too, the difference between the lowest and highest calculated delay within one group is kept below 2ps. It is also important to make sure that the copper surrounding the differential pairs is at ground potential. This is ensured by not only surrounding the differential pairs with ground planes on the signal layers but also connecting these to the central ground plane on the inner layer. For this purpose, many so-called "stitching" vias are placed around the differential pairs.

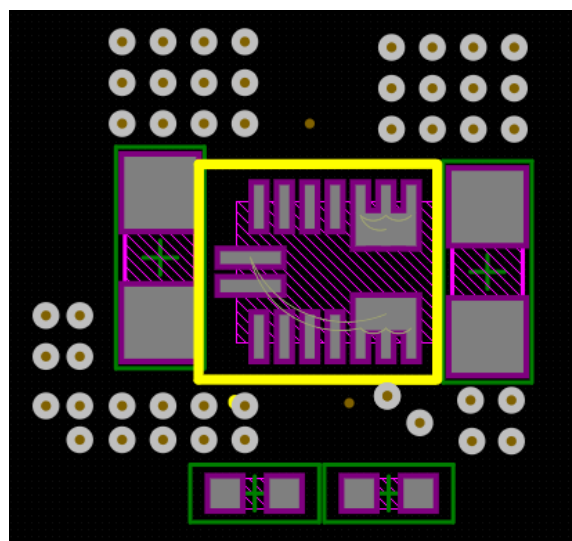
Each differential pair also features a pair of vias, to serve as test points, and in the case of the D-PHY connections also serve to split the traces, so that each trace can be connected to two I/O-pins.

3.3.5 Power Supply

The main components for the power supply are the switching voltage regulators. Due to the high-speed switching the footprint and surrounding copper, as well as the filter capacitors on the input and output side, should be designed to minimize electromagnetic interference while also

Figure 3: Recommended Layout

A better illustration of the placement of the components, that is the regulators with the associated capacitors and resistors, is shown in Figure 49.



- 64 -

3.3.6 GPIO and Other

The remaining traces are not expected to carry any “delicate” signals like the D-PHY and HDMI connections, making the routing of these rather trivial. Other aspects worth mentioning are the test points placed on the lines for the JTAG and SPI connections to the GateMate to enable debugging, if the need should arise, and another test point on the trace connecting the reference clock oscillator to the FPGA.

The GPIO connections between the south banks of the FPGA and the pseudo-PMOD connectors are not expected to carry any high frequency or particularly important signals and are thus routed as differential pairs with only roughly matched intra- and inter-pair delays, some with slight deficiencies, such as tight curves that were accepted for the benefit of keeping development time reasonably short.

To aid with placing and mark key components for later use, a silkscreen is designed, which is shown, along with the component placement, in Figure 50.

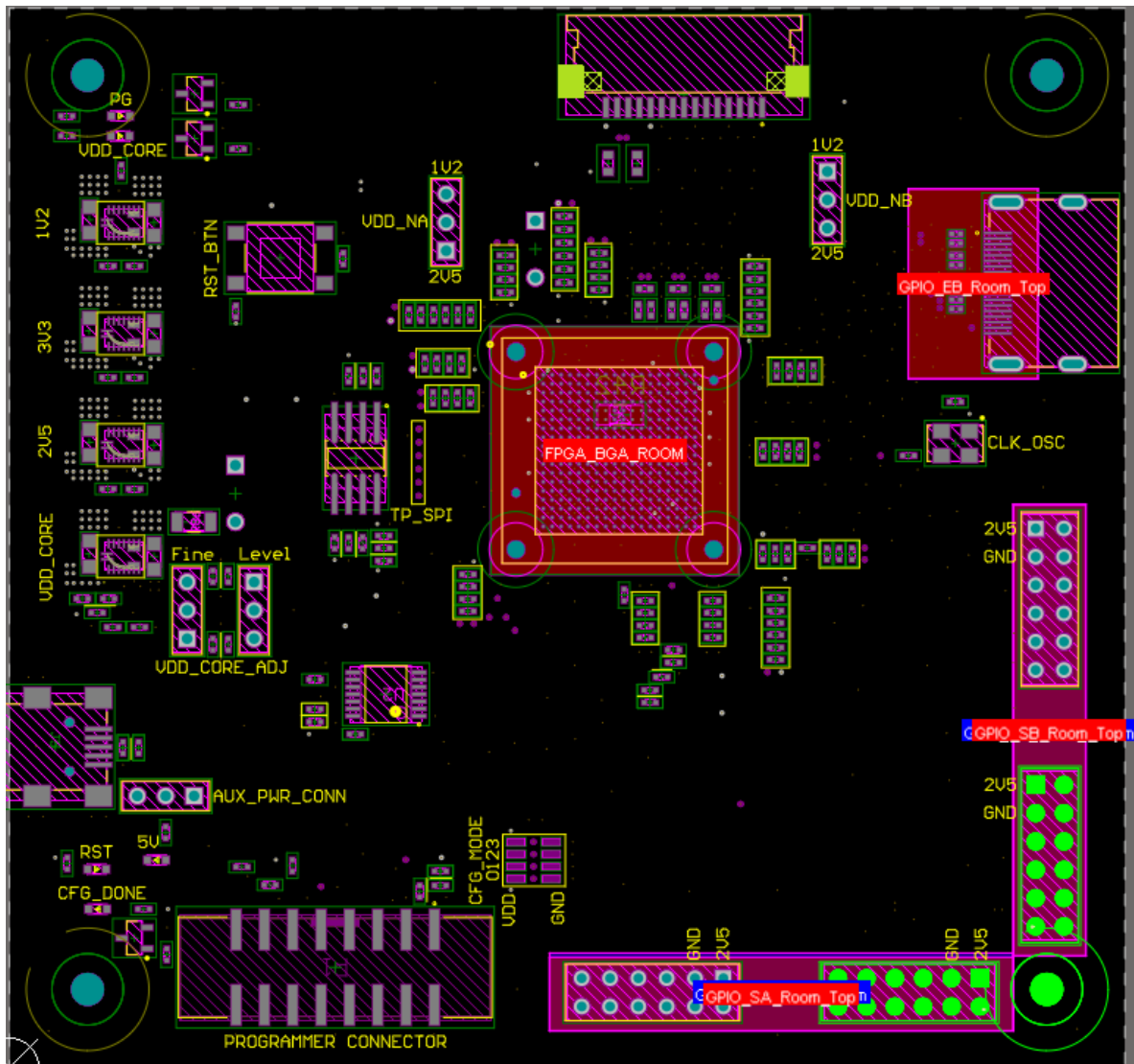


Figure 50: Placement of all components, including silkscreen

3.4 Assembly

The PCB itself was manufactured by a contract manufacturer for PCBs and PCBAs. A total of four samples were ordered, so that there are backup boards, in case one takes damage or more than one sample is needed or desired.

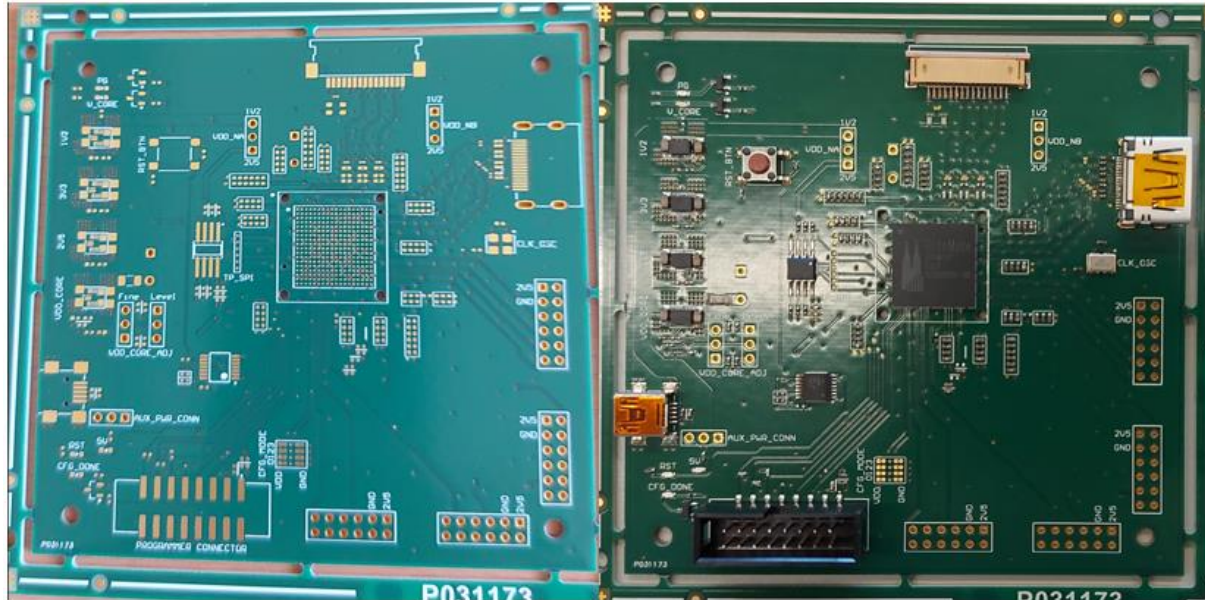


Figure 51: Top view of the PCB before (left) and after (right) mounting of surface-mounted components

The surface mount components are placed manually, with the aid of a mechanical pick-and-place tool, after applying lead-free $\text{Sn}_{96.5}\text{Ag}_3\text{Cu}_{0.5}$ (SAC305) solder paste with a T4 grade particle size (20-38 μm). The assembly is then placed in an oven with a controlled temperature ramp-up and kept around 230°C until obvious outgassing of flux and liquification of the solder are observed and the assembly is slowly cooled back down. The through-hole components are manually soldered onto the PCB afterwards.

3.5 Initial Testing

After soldering the components, the PCBA was first inspected visually, which revealed that one decoupling capacitor was missing. Either it was overlooked during placement or fell off during the baking process. After mounting the missing capacitor, visual inspection showed no obvious defects.

Next, the PCBA was checked for short circuits, primarily between the supply voltage nets and GND, but also between various signal lines and signal lines and GND. No short circuits were found during this.

After checking for major defects, the first test with voltage supply was done using a laboratory power supply. To minimize potential damage due to previously undetected faults, the output current was initially limited to 1mA and was slowly increased to 800mA, which would mean a total power dissipation of 4W, which is almost twice the maximum power consumption of around 2W of the GateMate as given in the datasheet. This current limit was not exceeded, and no signs of localized overheating were observed.

Subsequently the different supply voltages were measured, revealing a problem with the voltage regulator for the 1.2V supply. After initially delivering around 1.3V, after a short time the voltage on the output side increased to 5V, indicating a short circuit. A repair was attempted, but ultimately

unsuccessful. It was decided to remove the defective regulator, effectively separating the 1.2V net from all other supply voltage nets. Instead, one of the jumpers connecting to the 1.2V net, would be used to connect a laboratory power supply to deliver the 1.2V if needed. It should be noted however, that the “power good” daisy chain of the voltage regulators is interrupted in this configuration, although due to the pullup resistor on the net that also connects to the reset circuitry, this does not cause any problems in the power-up sequence of the FPGA.

With the safe operation of the PCBA verified, the functionality of the FPGA could be tested. For this purpose, simple testbenches were designed using Verilog. The code was synthesized and implemented using the Cologne Chip toolchain and uploaded via the GateMate Programmer. The first testbench designates all but one of the connected GPIOs in the southern banks as outputs, with one of them serving as an input. This way the GPIOs can be tested one by one.

Function	Test
GPIO connectivity and basic function	One input, remaining GPIOs as output. Assign to input value. Check if output matches input for all ports.
Clock input and PLL	Verify reference oscillator with oscilloscope. Configure “Blink” example using PLL. Check if output frequency matches expectation.
Connection of each D-PHY trace to two pins	HS pins are configured as input, LP as output, both powered with 2.5V. Separate output pin indicates whether signal is received at the inputs.

Table 24: Functional test for FPGA

The test described in Table 24 did not reveal any defects or unexpected behaviours, so the evaluation board was deemed to be functional and ready to be used for the development of a CSI-2 receiver.

4 Design and Implementation

In this section the design of the D-PHY receiver in Verilog is explained in detail and the results of initial tests are presented and discussed.

4.1 Demonstrator Concept

A simple and effective way to verify the reception of video data is to display the received data. Especially in the context of an interface implementation on an FPGA it makes sense to demonstrate not only that the data transmission works, but also that the transmitted data can be utilized in a meaningful way. Therefore, in this project, beyond the implementation of the CSI-2 compatible receiver, a demonstrator is developed that allows a direct feed-through of the received data to a display via a DVI connection using HDMI hardware. Aside from the obviously required implementation of the DVI output, this also requires some degree of data processing and synchronization. Processing is required because the camera outputs raw sensor data that, as already touched on in Chapter 2.4.1, needs to be interpreted and converted to a standard format to be intelligible for a display, such as a PC monitor. The synchronization is necessary because both CSI-2 and DVI expect a continuous stream of data with clearly defined timings and no provisions for halting the transmission intermittently. Consequently, some form of buffering and processing is required between the input and output side of the system. A simplified block diagram of the system is shown in Figure 52, where the “dual-clock frame buffer” handles most of this functionality.

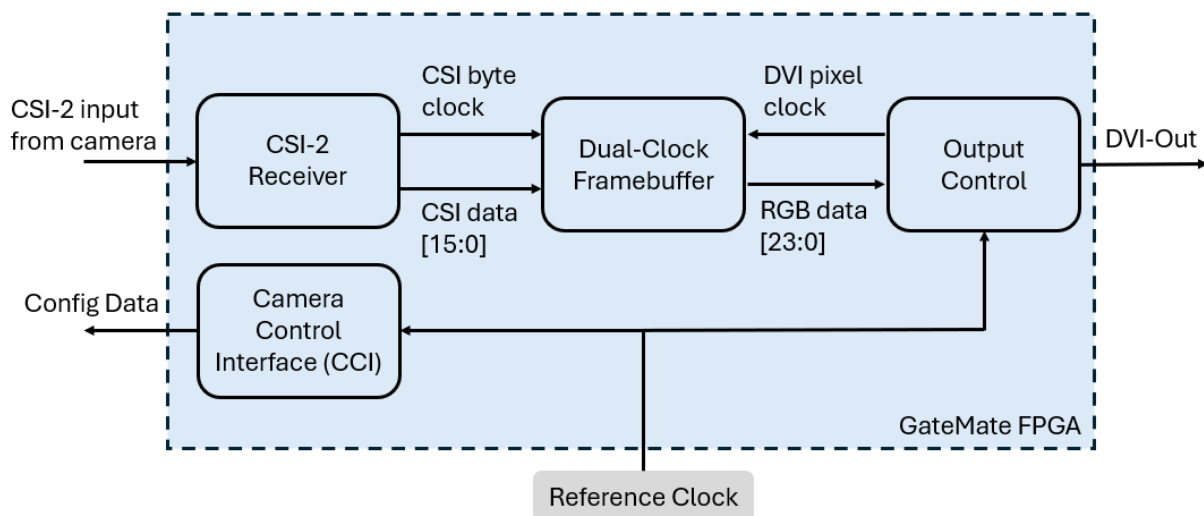


Figure 52: Simplified block diagram of the demonstrator system

In Figure 52, the demonstrator is split into four subsystems: The CSI-2 receiver, the camera control interface, the dual-clock framebuffer and the output control module. Although the CCI is also part of the CSI-2 specification, it operates fully independently from the video receiver, uses a different clock and is therefore displayed as a separate subsystem. The CSI-2 receiver uses only the CSI clock received from the camera module and the byte clock derived from it. The latter also works as the write side clock for the framebuffer. On the other hand, the CCI module, which generates a comparatively very slow 100kHz output clock is clocked with the external reference clock on the evaluation board, running at 10MHz. The reference clock is also used by the “output control” module, where it is used to generate the output pixel- and bit-clocks used for the TMDS output. Of these, the pixel clock is used for most of the output control logic, and it also serves as the read side clock of the framebuffer.

Due to limited on-chip memory and to limit complexity, the framebuffer shall be kept as simple and compact as possible. As the video input and output occur with the same framerate it is sufficient to store no more than one frame at a time and only very limited synchronization logic is required compared to a general-purpose, asynchronous FIFO buffer. The buffer shall also enable an easy conversion of the raw image data to the 24-bit RGB format with almost no processing. The easiest way to achieve this is to set up the camera module's output format to "RAW8", meaning that every sub-pixel value is given as an 8-bit integer number. This way three of the four sub-pixels of each "full colour" pixel can be combined into one 24-bit pixel, effectively reducing the image resolution by a factor of 4. To simulate an image with the original sub-pixel resolution for the output, this value is then used four times, twice per line over two lines. By designing the memory access so that three sub-pixel values can be read simultaneously the read operations can be clocked with the output pixel clock. A more detailed explanation of the buffer stage is given in section 4.4.

Finally, the output control module handles the output timing, the TMDS encoding and the generation of related clocks. It also performs rudimentary colour balancing on the image data, which is necessary because the camera chip senses light differently than the human eye, which leads to colours in the unprocessed image looking unnatural, depending on the lighting. This colour-balancing shall be static in the demonstrator, which is sufficient if the lighting in the operating environment is adequately consistent.

The following sections give a detailed overview of the design of the system components, roughly following the main data path beginning with the CSI-2 receiver and ending with the TMDS output.

4.2 D-PHY / CSI-2 Receiver Design

The D-PHY receiver developed in this project is meant to provide basic video reception functionality, which means that the more advanced features of the CSI-2/D-PHY interface are not implemented. This includes the Escape Mode functionalities of D-PHY and any reverse communication on the D-PHY lanes, as well the entire LP functionality.

The primary goal is to enable the reception of video data via the CSI-2 interface, specifically on the GateMate FPGA. Thus, the Verilog design is tailored to this particular FPGA by utilizing specific device primitives and inline attributes, that help achieve optimal performance. By omitting the LP signal processing entirely, the interface is greatly simplified.

Much of the receiver design is based on a similar design made for Xilinx FPGAs in [28] and adapted for this project. The most notable changes are the removal of SystemVerilog constructs used in [28] that the Yosys synthesis software may not handle very well; the removal, and where necessary replacement, of signals for LP functionality, and replacing Xilinx primitives, like the input SERDES, with constructs suitable for the GateMate FPGA. Additionally, the CSI-2 protocol handling was simplified to optimize the performance of the core functionality required for this project.

4.2.1 Module Structure

For complex HDL designs the code is usually split into several smaller functional units, referred to as modules. This not only simplifies understanding and working with the code, but also facilitates code reuse, debugging and enables more dynamic, modular designs. To leverage these potential benefits, the module structure, that is which functionality goes into which module and how these interact must be chosen properly. A bad module structure can complicate the design process by causing confusion and introducing unnecessary dependencies. Therefore, it makes sense to develop a basic module structure in the beginning, to serve as a guideline during development.

A simplified version of the module structure for the CSI-2 receiver in a configuration with two data lanes is shown in Figure 53. Note that for clarity, only the primary data path and the clock distribution are represented, while any auxiliary signals, like status flags, are omitted.

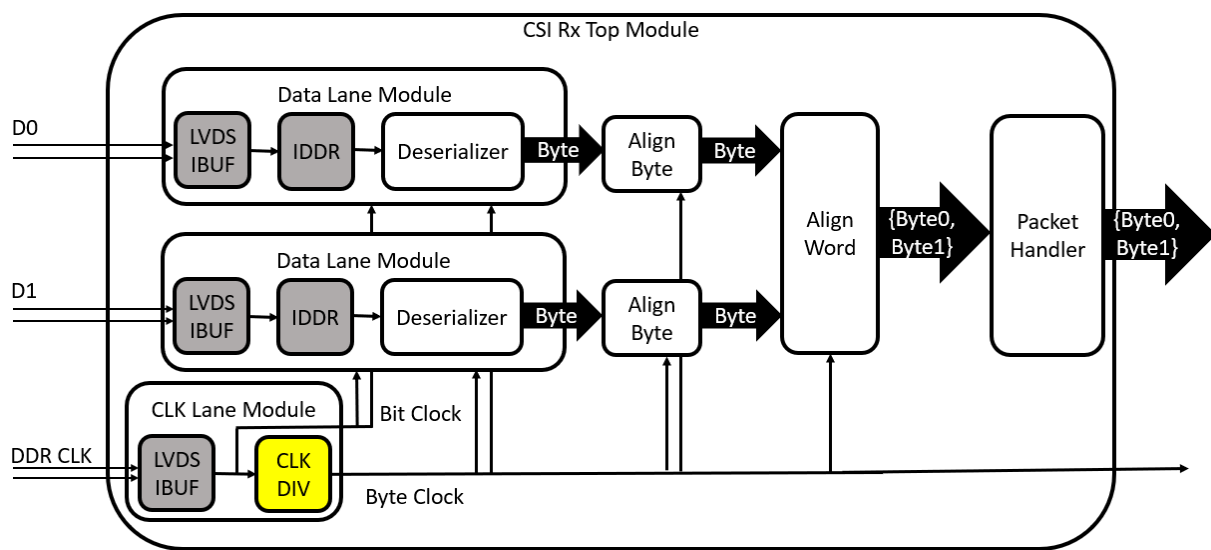


Figure 53: Module structure of the CSI-2 receiver

Each block in Figure 53 represents a Verilog module instance, except the “CLK DIV” block marked in yellow, with the grey-filled blocks being device primitives.

The initial reception and deserialization of the incoming clock and data signals is handled by so-called “lane modules”, based on the terminology from the D-PHY specification. There are two types of lane modules, the clock lane module that handles reception of the DDR clock signal and generation of a byte clock and the data lane module that handles the reception and deserialization of the incoming data streams. The deserialized bytes are then forwarded to a module called “align byte”, which checks the incoming data for the synchronization sequence “10111000” (0xB8) and aligns the byte accordingly. Although this could reasonably be integrated into the data lane modules, this was not implemented to keep the device-specific primitives within small and easy to replace modules.

The aligned byte streams are then passed on to a module named “align word”, which bundles and synchronizes the byte streams to forward them to the “packet handler”, that processes the low-level protocol of the CSI-2 interface. The “align word” module also filters out the synchronisation sequence from the incoming data stream and detects “false triggers”, which describes an event where the synchronisation sequence is detected on only one of several active data lanes.

4.2.2 Clock Lane Module

The clock lane module serves to convert the incoming differential clock signal into a single ended signal, that can be used as a clock within the FPGA and derive a byte clock, which will be the clock signal for most of the synchronous receiver logic. The first part is handled by the LVDS input buffer, the latter is achieved by using a simple clock divider.

```
module csi_rx_ck #(
    parameter ONCHIP_RTERM_EN = 0,
    parameter IDELAY = 4'h0
)(
    input wire [1:0] dphy_ck_hs,
    input wire rst,

    output hs_bit_clk, // ddr bit clock = dphy_ck_hs
    output reg hs_byte_clk // sdr byte clock = dphy_ck_hs/4
);

(* clkbuf_inhibit *)
wire ddr_ck;

CC_LVDS_IBUF #(
    .LVDS_RTERM(ONCHIP_RTERM_EN),
    .DELAY_IBF(IDELAY)
) dphy_ck_ibuf (
    .I_P (dphy_ck_hs[0]),
    .I_N (dphy_ck_hs[1]),
    .Y   (ddr_ck)
);
```

Listing 1: Code snippet from clock lane module, including interface definition

The interface definition of the clock lane module is shown in Listing 1, including parameters that can be used to configure the LVDS input buffer. “ONCHIP_RTERM_EN” is normally set to 0 to deactivate the on-chip termination resistor. “IDELAY” can be used to configure a delay for the received signal and is normally also set to 0.

To ensure a short path for the incoming clock signal from the LVDS buffer, the corresponding wire “ddr_ck” is assigned the inline attribute “(*clkbuf_inhibit*)”, which prevents the synthesis and place and route tools from assigning it to a dedicated clock distribution network in the FPGA. An additional benefit of this is that none of the four available clock meshes is used up by this low-fanout clock. While this is not yet very relevant in this project, it is beneficial if the receiver is integrated into a more extensive design.

The LP functionality, which is limited for the clock lane even in a fully featured CSI-2 receiver, is not implemented, which simplifies the design and removes the need for additional input pins.


```

assign hs_bit_clk = ddr_ck; // enable ? ddr_ck : 0;
reg [1:0] div = 2'b00;

always @(posedge hs_bit_clk) begin
    if (rst) begin
        div <= 0;
        hs_byte_clk <= 0;
    end
    else begin
        div <= div + 1'b1;
        if (div == 2) begin
            hs_byte_clk <= ~hs_byte_clk;
            div <= 1;
        end
    end
end
end

```

Listing 2: Code snippet from clock lane module showing byte clock generation

Listing 2 shows the Verilog code for the byte clock generation. A 2-bit counter is incremented with the rising edges of the incoming HS clock signal. The byte clock signal is toggled after every two edges. This means that the byte clock will always be generated whenever there is an HS clock signal. Because it is not guaranteed that the number of clock transitions preceding the first bit of the synchronisation sequence is always the same (only that the sequence begins on a rising clock edge), this necessitates a routine, that aligns the deserialized data bytes with the byte clock.

The frequency of the byte clock is a quarter of the incoming HS clock frequency, because the HS clock is used as a DDR clock, so that with every clock cycle, two bits of data are transferred.

4.2.3 Data Lane Module

The data lane modules are more complex than the clock lane module, although most functionality is contained in the “csi_rx_iserdes” sub-module. Beside the LVDS input buffers, they contain DDR input registers and the deserializer sub-module. Although the LP functionality defined for data lanes in the D-PHY specification is a lot more extensive than for clock lanes, this implementation neglects this component of the interface. Similarly to the clock lane module, this allows to simplify the design, and most importantly reduce the required number of GPIOs by half. This comes at the cost of losing potential functionality that is, however, not required here.

Listing 3 shows the interface definition of the data lane module. In addition to the parameters for the LVDS input buffer, there is a parameter related to the DDR input register. The “INV_CK” parameter can be used to invert the allocation of the rising and falling edge of the DDR input register and is normally set to 0.

The inputs of the module are the HS connections for the data lane as well as the bit and byte clock signals. Note that the LP inputs are not used in the current implementation. The only output of the module is the stream of received data bytes.

```

module csi_rx_dn #(
    parameter ONCHIP_RTERM_EN = 0,
    parameter IDELAY = 4'h0,
    parameter SWAP_D = 1'b0,
    parameter INV_CK = 1'b0
)(
    input wire rst,
    input wire [1:0] dphy_dn_hs,
    input wire bit_clk,
    input wire byte_clk,

    output wire [7:0] recvd_byte
);

```

Listing 3: Interface definition of data lane module

The data lane module is essentially just a wrapper to have a unified interface for the device primitives (LVDS buffer and DDR input register) and the “csi_rx_iserdes” sub-module. The latter’s module definition is shown in Listing 4.

```

module csi_rx_iserdes (
    input wire bit_clk,
    input wire byte_clk,
    input wire rst,
    input wire [1:0] ddr_i,
    output reg [7:0] byte_o
);
    reg [7:0] s0 = 0, s1 = 0, s2 = 0;
    (*keep*)
    always @(posedge bit_clk or posedge rst) begin
        if (rst) begin
            s0 <= 0;
            s1 <= 0;
            s2 <= 0;
        end else begin
            s0 <= {ddr_i[1], ddr_i[0], s0[7:2]};
            s1 <= s0;
            s2 <= s1;
        end
    end
    always @(posedge byte_clk or posedge rst) begin
        if (rst)
            byte_o <= 0;
        else
            byte_o <= s2;
        end
    end
endmodule

```

Listing 4: Module definition of deserializer

The module consists of three 8-bit wide registers, with an additional 8-bit output register. Unless the reset signal is HIGH, the register “s0” operates as a shift register, clocked with the D-PHY bit clock. The current data from the DDR input register is written to the most significant bits, the old data is shifted towards the least significant bits. This way the incoming data, that is transmitted LSB first, is immediately stored in the correct bit order. The registers “s1” and “s2” are used to “double-flop” the data to reduce the risk of metastable data being passed on, when transitioning from the bit clock domain to the slower byte clock domain [29]. That means with each cycle, the data from s0 is written to s1 and the data from s1 written to s2. With each positive edge of the byte clock, the data from s2 is written to the output register “byte_o”.

4.2.4 Byte Alignment

As mentioned previously, a routine to properly align the deserialized data bytes with the byte clock edges is required, because the byte clock generation is completely independent from the signals on the data lanes. It can also compensate for misalignment caused by skew on the transmission lines. The alignment has to be performed for every HS data burst.

The alignment is achieved by evaluating the combination of the current incoming byte and the previous one. Regardless of the offset between the byte clock and the data byte, at least one full valid byte will always be contained within these 16 bits. To determine the offset between byte clock and the limits of the data bytes, the module looks for the synchronization sequence at the start of the transmission by comparing the known sequence to the contents of each set of eight consecutive bits within the two bytes, as shown in Listing 6.

Once the correct alignment for the current burst is found, the corresponding position of the input data vector is assigned to the output, and the “data_valid” flag is set to indicate the validity of the output data.

Listing 5 shows the interface definition of the byte alignment module. Beside the clock, reset, enable and data inputs/outputs, there are several control signals coming in from and going out to other modules. The incoming signals “wait_for_sync” and “packet_done” are important control signals determining when to start the alignment. If wait_for_sync is not set, the module will not change the currently applied offset to the output data, which prevents false realignment in case the synchronisation sequence is detected at a time when it is not expected. The “packet_done” signal indicates that a full packet has been received, and the alignment status must be reevaluated for the next incoming packet. This signal is passed to the module from the “Align Word” module and is also used to suppress “false triggers”, where the synchronisation sequence is only detected on one data lane, which should not occur during normal operation. If a false trigger is detected the packet_done signal is immediately asserted to reset the synchronisation status.

The outgoing status signals are “found”, which is an asynchronously calculated signal that is asserted for one clock cycle, when the synchronisation sequence is found in the input data, and “data_vld”, which is high when the data currently being put out by this module is valid.

```

module csi_rx_align_byte (
    input      clock,          // byte clock in
    input      reset,          // active-1 reset
    input      enable,         // active-1 enable
    input  [7:0] deser_in,     // raw data from ISERDES
    input      wait_for_sync,  // when 1, if sync not already found, look for it
    input      packet_done,    // drive 1 to reset synchronisation status

    output reg    found,
    output reg  [7:0] data_out, // aligned data out, typically delayed by 2 cycles
    output reg    data_vld     // goes 1 as soon as sync pattern is found
);
    // so data_out on next cycle contains header

```

Listing 5: Interface definition for align byte module

```

always@* begin
    //---find SYNC pattern and its bit-offset
    found = 1'b0;
    offset = 3'd0;
    if ({curr_byte[0], last_byte} == {SYNC, 1'd0}) begin
        found = 1'b1;
        offset = 3'd0;
    end
    if ({curr_byte[1:0], last_byte} == {SYNC, 2'd0}) begin
        found = 1'b1;
        offset = 3'd1;
    end
    if ({curr_byte[2:0], last_byte} == {SYNC, 3'd0}) begin
        found = 1'b1;
        offset = 3'd2;
    end
    if ({curr_byte[3:0], last_byte} == {SYNC, 4'd0}) begin
        found = 1'b1;
        offset = 3'd3;
    end
    if ({curr_byte[4:0], last_byte} == {SYNC, 5'd0}) begin
        found = 1'b1;
        offset = 3'd4;
    end
    if ({curr_byte[5:0], last_byte} == {SYNC, 6'd0}) begin
        found = 1'b1;
        offset = 3'd5;
    end
end

```

Listing 6: Code snippet showing logic for byte alignment

4.2.5 Align Word Module

The last module in the data path of the D-PHY receiver combines the data streams from the data lanes and compensates for skew between the lanes, while also filtering out the synchronisation sequence from the input data. The interface definition of the module is shown in Listing 7. Via the parameter “NUM_LANES” the module can be configured for a variable number of lanes, by sizing the width of the input and output bus as well as the internal registers for data buffering accordingly. Note that the current implementation of the CSI-2 protocol handling assumes a 2-lane configuration and would have to be adapted manually.

```
module csi_rx_align_word #(
    parameter NUM_LANES = 2
)()
    input      byte_clock,      // byte clock in
    input      reset,           // active-1 reset

    input      enable,          // active-1 enable
    input      packet_done,     // packet done input from packet handler entity
    input      wait_for_sync,   // whether or not to be looking for an alignment
    input [(NUM_LANES*8)-1:0] word_in,      // unaligned word from the byte aligners
    input [NUM_LANES-1:0] valid_in,        // valid flags from byte aligners

    output reg packet_done_out, // 'packet_done' output to byte aligners
    output reg [(NUM_LANES*8)-1:0] word_out, // aligned word out to packet handler
    output reg valid_out        // goes high once alignment is valid: First word
                                // with 'valid_out=1' is the CSI packet header,
                                // i.e. the 0xB8 Sync byte is filtered out
endmodule;
```

Listing 7: Interface definition for align word module

To perform the alignment the incoming data from all lanes, with their associated “data valid” signals are retained for two clock cycles. By checking the AND combination of the data valid signals from all lanes, the moment when all lanes are receiving data is determined. In that moment, if the module’s “valid_out” signal is not yet set, the stored values of each lane’s data valid signal is evaluated. In case any lane had presented valid data within the previous two clock cycles, the corresponding register is linked to the output vector. This alignment is then latched until a global reset or a “packet done” signal from the module handling the CSI-2 low-level protocol is set.

The false trigger recognition works by checking for three consecutive cycles with a HIGH data valid signal on any lane and checking if the current data valid signal is HIGH on all lanes. Unless all lanes are showing valid data, the “packet_done_out” signal is sent to the byte aligners.

Contrary to the CSI-2 or D-PHY specification the “align word” module does not restore a single byte-stream from the incoming data but instead parallelizes the data. The output width is determined by the number of lanes. By parallelizing the data stream within the FPGA, the clock speeds can be kept low, which significantly reduces timing issues and helps to leverage the inherent advantages of FPGAs for parallel data processing.

```

else begin
    word_dly_1 <= word_in;
    word_dly_2 <= word_dly_1;
    valid_dly_1 <= valid_in;
    valid_dly_2 <= valid_dly_1;
    valid_out <= valid;
    if (enable == 1'b1) begin
        if ({valid_in_all, valid, wait_for_sync} == 3'b101) begin
            valid <= 1'b1;
            for ( i = 0; i <= NUM_LANES-1; i = i + 1) begin
                if (valid_dly_2[i] == 1'b1) begin
                    taps[i] <= 2'd2;
                end
                else if (valid_dly_1[i] == 1'b1) begin
                    taps[i] <= 2'd1;
                end
                else begin
                    taps[i] <= 2'd0;
                end
            end
        end
        else if (packet_done == 1'b1) begin
            valid <= 1'b0;
        end
    end
end

```

Listing 8: Code snippet showing logic for determining validity of synchronization signals

4.2.6 CSI-2 Protocol Handling

The full CSI-2 protocol is quite extensive, offering a lot of functionality. However, much of it is not necessary for many applications, so it can be omitted here, to achieve a lightweight implementation of a receiver capable of receiving data sent from a known CSI-2 source. Essentially all functionality that is not directly related to the reception of video data has been left out to reduce the complexity of the system, save chip area and thus also reduce the number of potential failure points.

One consequence of this, however, is that some parts of the receiver are tailored specifically for the application and are not able to handle things like different data types used in the transmission. Instead, the receiver is designed to only accept the expected data type via a 2-lane CSI-2 link, as utilized by the camera module. Configuring the receiver for a different application requires changes in the HDL code, beyond changing a few parameter values. The interface definition of the module is given in Listing 9.

```

module csi_rx_packet_handler #(
    parameter NUM_LANES = 2
)(
    input clock,
    input reset,
    input enable,
    input [15:0] data,
    input data_valid,
    output sync_wait,           //signal readiness to receive next packet
    output packet_done,         //reset alignment status after packet
    output [15:0] payload_out,
    output payload_valid,
    output [7:0] good_checksum_cnt, //for verification only
    output [7:0] bad_checksum_cnt  //for verification only
);

```

Listing 9: Interface definition of packet handler module

As discussed in section 2.2.2, the CSI-2 protocol is based around two types of packets: long packets, consisting of a four-byte header, the payload of arbitrary length (usually the length of one line of image data), and a two-byte footer, containing a checksum, and short packets the size of a long packet header.

The packet handler in the receiver is based on a finite state machine with eight states handling the control flow. To facilitate the handling of the incoming data, most importantly the headers, a four-byte (32 bit) shift-register called “packet_data” is used to temporarily store the incoming data. By evaluating defined sections of the register content at the right time, the header can be evaluated with minimal delays. Listing 10 shows the logic for evaluating the packet headers. The “is_allowed_type” signal is asserted, if the received packet type code is listed as an accepted type. In this configuration only RAW8 data is accepted as it is the only relevant data type sent by the camera module, and packets of any other type are discarded. Other packet types could be added to the list of accepted types by adding the corresponding type-code to the concatenation in the second line shown in Listing 10. To make use of this capability, some additional logic for handling different data types would be required, however.

```

wire is_allowed_type;
assign is_allowed_type = ({packet_type == 6'h2A}); //RAW8 data
wire [7:0] expected_ecc;
always @(*) begin
    is_hdr = ({data_valid, state} == 4'hD);
    packet_type = packet_data[5:0];
    packet_len = packet_data[23:8];
    packet_for_ecc = packet_data[23:0];
    valid_packet = ((packet_data[31:24] == expected_ecc) & is_allowed_type) & (packet_data[7:6] == 2'd0);
    long_packet = (packet_type > 6'h0F) & valid_packet;
end
csi_rx_hdr_ecc u_ecc(
    .data(packet_for_ecc),
    .ecc(expected_ecc)
);

```

Listing 10: Logic for evaluation of packet headers

Other parts of the header are the word count (`packet_len`), which indicates the number of payload bytes in the packet, and the error correction code (ECC) for the header. While the word count can simply be stored for later comparison to the number of received bytes, the ECC is compared to a locally generated ECC for the same header data. Although the ECC allows the correction of single bit errors in the header, this functionality is not implemented here, as it would require significantly more logic than a simple check for equality for a questionable benefit. Instead, the goal of keeping the hardware requirements for the receiver as small as possible is prioritized and the ECC only serves to confirm the integrity of correctly received headers. In Figure 54 a graph of the state machine that controls the receiver is presented to illustrate the control flow.

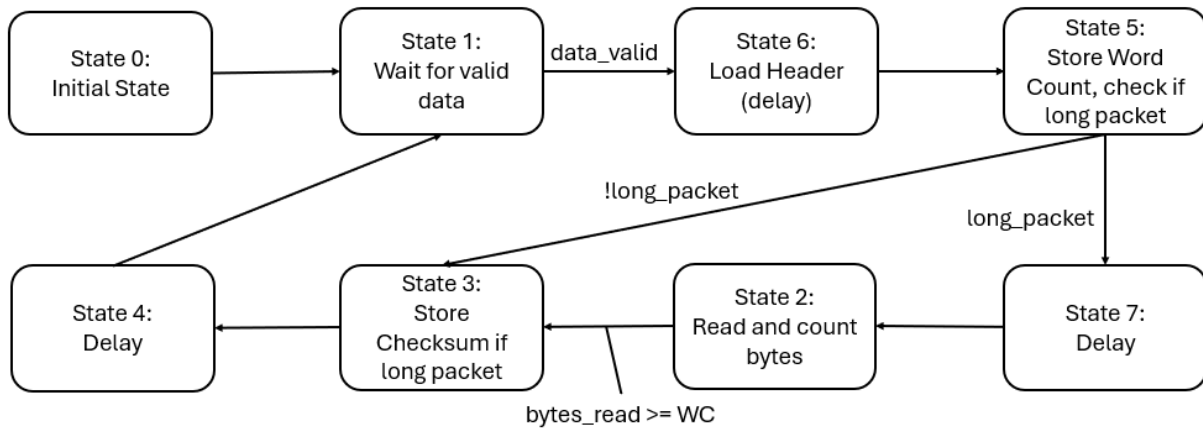


Figure 54: Flowchart of the state-machine for packet handling

State 0 is the startup state that is never reached again during operation. At the first rising edge of the byte clock, the state-machine transitions to state 1. In this state the state-machine waits until the “`data_valid`” signal from the word align module is asserted and state 6 is reached. This state serves to create a delay of two clock cycles, during which the packet header is read and processed, so that in the following state 5 the further steps can be determined, based on the content of the header. Note that the required delay depends on the number of data lanes used in the CSI-2 interface, which determines the number of bytes read in one clock cycle. In a single lane configuration, for example, the delay would need to be twice as long to read the full header.

In state 5 the word count bytes from the header (which contain the payload in short packets) are stored in the “`packet_len_q`” register, so that by comparison with the number of received payload bytes, the end of the payload can be determined. This happens regardless of whether the packet is a long packet or short packet and in the latter case, the register will contain the short packet’s payload, which is not used in this design. For short packets and invalid packets, the state machine goes directly to state 3, in which the “`packet_done`” signal is asserted, signalling the end of the packet transmission to the other components of the receiver.

If the packet was determined to be a long packet, the next state is state 7, which is another delay state, during which the first payload bytes are shifted through the “`packet_data`” register to the section (15 to 0) that will be assigned to the payload data output port in the following state 2. State 2 is then retained until the entire payload has been read and state 3 is reached. In this state, the checksums, both the received one and the locally calculated one, are stored in dedicated registers for comparison. Upon the next rising clock edge, the state machine transitions to state 4, which delays the transition back to state 1 by one clock cycle to give the other receiver components time to properly reset for the next transmission. In this state the signal “`lfsr_rst`” is

set, which serves not only to reset the linear feedback shift register (LFSR) module for the next packet but also functions as the clock signal for the checksum evaluation logic.

In addition to the ECC for the header, the CSI-2 protocol employs checksums to verify the integrity of the payload data. These checksums are generated in the transmitter by shifting the payload through an LFSR, as described in section 2.2.4. Although this functionality is not required for data reception and requires a non-trivial amount of additional hardware, it is included in the receiver to facilitate thorough verification of the receiver performance. To verify the checksum, the generation process is repeated in the receiver, using the received payload data, and the two checksums are compared. If they are equal, the data was most likely transmitted without any error. To automate the checksum evaluation to some degree, two counters, which are clocked with the previously mentioned “lfsr_rst” signal, are employed: One counts the number of matching checksums, the other counts the number of non-matching checksums. With each rising edge of this signal the received and locally calculated checksums are compared, and the appropriate counter is incremented. This redundancy helps to identify potential errors in the evaluation logic because if, at the end of a packet, both or none of the counters are incremented, an error must have occurred in the evaluation.

The checksum generation in the receiver is complicated somewhat by limited range of available clock frequencies. The LFSR used in the CSI-2 protocol is designed for serial operation, that is all payload data bits are shifted through the LFSR sequentially, which would not be practical here. Therefore, a parallelized implementation for the LFSR is required. For this, a highly configurable Verilog implementation of such a parallelized LFSR, which can be found at [30], is used and configured as shown in Listing 11. The module used to generate the checksums is comprised of two components. The core component is a generic, configurable module that implements a parallelized LFSR in combinatorial logic, based on the set parameters. It computes bitmasks to calculate the state of an LFSR of arbitrary width (LFSR_WIDTH) with a given polynomial (LFSR_POLY) after an arbitrary number (DATA_WIDTH) of shift operations.

```
lfsr_crc#(  
    .LFSR_WIDTH(16),  
    .LFSR_POLY(16'h1021),  
    .LFSR_INIT(16'hFFFF),  
    .LFSR_CONFIG("GALOIS"),  
    .REVERSE(1),  
    .INVERT(0),  
    .DATA_WIDTH(16),  
    .STYLE("AUTO")  
) crc_inst (  
    .clk(clock),  
    .rst(lfsr_rst),  
    .data_in(payload_out),  
    .data_in_valid(payload_valid),  
    .crc_out(checksum_calc)  
);
```

Listing 11: Instantiation of the "lfsr_crc" module configured for the CSI-2 CRC

The other component is a wrapper module, called “lfsr_crc” (CRC for Cyclic Redundancy Check) that instantiates an LFSR with user-defined parameters, defines a specialized interface for the use of the LFSR for checksum generation and updates the inputs to the combinatorial module with each cycle of the byte clock.

4.2.7 CSI-2 Receiver Module

The modules discussed so far are bundled together in a module called “csi_rx_top”. In this module the modules that make up the receiver are instantiated and connected together, so that a full CSI-2 receiver can be instantiated as one module. The interface definition of this top module is shown in Listing 12.

```
module csi_rx_top #(
    parameter ONCHIP_RTERM_EN = 0,
    parameter IDELAY_CK = 4'h0,    //clock input delay
    parameter IDELAY_DN = 4'h0,    //data input delay
    parameter NUM_LANES = 2, // only 2 lanes supported
    parameter NUM_RAW = 8 // only RAW8 supported
)(
    input enable,
    input reset,
    input [1:0] dphy_ck_hs,
    input [1:0] dphy_d0_hs,
    input [1:0] dphy_d1_hs,

    output f0, f1, //show when sync sequence is found (for debugging)

    output csi_byte_clk,
    output [(NUM_LANES*NUM_RAW)-1:0] csi_raw_data,
    output csi_raw_valid,
    output [7:0] csi_checksum_good, //good checksum count for debug
    output [7:0] csi_checksum_err  //bad checksum count for debug
);
```

Listing 12: CSI-2 receiver top module interface definition

Contained in the receiver top module are one clock lane module, two instances each of the data lane module and byte aligner, the word align module and the packet handler. This way the whole receiver can be instantiated as one module, that provides the unprocessed video data from the camera.

4.3 CCI-Handler

One component of the CSI-2 interface is relatively independent from the rest. That is the “Camera Control Interface” (CCI), an I²C-derived interface for transmitting configuration data and miscellaneous commands to the camera. Because it is largely compatible with the I²C specification, a regular I²C-master can be used to implement this. A module called “CCI handler” is used to initialize the camera by sending the appropriate configuration data to the camera’s control registers via the CCI interface. The module’s interface definition is shown in Listing 13

```
module CCI_Handler (  
    input    clk,  
    input    RST_N,  
    inout    I2C_SCL,  
    inout    I2C_SDA,  
    output    init_done,  
    output    init_error  
);
```

Listing 13: CCI handler interface definition

The CCI handler module consists of a state-machine and two sub-modules. One is a configurable I²C master, originally written in System Verilog, taken from [31], converted to Verilog using a System-Verilog-to-Verilog converter [32] and configured for this particular application. Specifically, the fixed slave device address (7'b0010000) [9] is entered, the register address width and data width are set to 8 bits, while the register size is set to 16. Additionally, to ensure proper communication an error flag was added, that indicates a failed transmission attempt. This error flag is set, when the I²C master receives a negative acknowledge. This allows easy error handling in the CCI handler module. The clock for the I²C communication is created from the external reference clock. This is achieved by using a clock counter that toggles at a user defined value. The logic of the module is clocked with the reference clock, with the divider output functioning as an enable signal. This way, issues from bad routing of the clock signal can be prevented [31]. Formula (1) shows the I²C clock frequency f_{SCL} as a function of the system’s master clock frequency f_{CLK} and the divider value. For given frequency values, the divider value can be calculated using formula (2).

$$f_{SCL} = \frac{f_{CLK}}{4(divider + 1)} \quad (1)$$

$$divider = \frac{f_{CLK}}{4 * f_{SCL}} - 1 \quad (2)$$

For the testbench the external reference clock is used, so f_{CLK} is 10MHz and f_{SCL} shall be 100kHz, which is the usual I²C clock frequency for standard mode [32]. Therefore, the resulting divider value is 24.

4.3.1 Initialization Module

The Raspberry Pi camera module V2, contains a Sony IMX219 camera chip. To configure the camera chip for the intended operation, the configuration data is loaded to the chips’ control registers via the CCI. The IMX219 uses a 16-Bit address space, with every register containing eight Bits of data. Not all of these registers can be written to, many are set as read-only and contain information such as the chip-ID, lot number or values for fixed settings, that cannot be changed

by the user. The datasheet distinguishes between “configuration registers”, including “control registers” and “frame bank control” for settings limited to one of the two frame banks and the read-only “parameter limit registers”, containing information on the possible range of parameter settings.

For this project, only a subset of the control registers is relevant. Specifically, the image resolution and the frame rate as well as the settings for the CSI-2 output are crucial. Additionally, the camera needs to receive a “start” command to begin operation and for testing purposes, various pre-defined test images can be put out by the camera [9].

To minimize the difficulty of handling real image data, the resolution, framerate and the clock frequency of the data transmission should be kept as low as possible, while retaining realistic testing conditions. Therefore the 640*480@60Hz video format is chosen, as it is the easiest to handle format that is supported by most modern displays, with the lowest pixel frequency of only 25MHz (see chapter 2.1). The clock frequency for the D-PHY link shall be set to 200MHz, which is the minimum value for the camera module [9]. This way, development is not impaired by issues of handling very large amounts of data, which may necessitate handling external memory, and high clock frequencies, for which the FPGA used in this project has not been extensively characterized. Once a working system for low resolutions is achieved, it may be adapted for higher resolutions and clock frequencies and the limits of its capability explored.

The initialization sequence with the values to write into the configuration registers and the order of the write operations, is based on [33]. These values are stored in two look-up tables. The correct sequence is achieved by assigning the register addresses and corresponding values to successive addresses in the memory blocks. An address counter, connected to the address input of the memory blocks is initialized with 0 and increments whenever the module receives a dedicated signal “step_increment” from the CCI handler module, which is described in detail in the next chapter. A reset signal from the CCI handler module resets the address counter. When the final configuration value has been transmitted, the initializer module sets a flag indicating the completion of the configuration procedure to the CCI handler module. All further control functionality is handled in the CCI handler module. An overview of the full initialization sequence is given in Table 26. All remaining registers remain at the default setting. Note that the order in which the data is written to the control registers does not matter, as long as the camera is in the standby state. Thus, only the initial standby command and final “start stream” command are fixed, while the other may be freely arranged.

While the settings for digital or analogue gain, test image output and data format are fairly straightforward, the setup for the timing, i.e. the framerate of the captured video and the clock for the CSI-2 transmission are more complex, since they depend on several factors. The effective framerate of the video capture is a function of the clock frequency, the value in the “line_length” register and the value in the “frame_length_lines” or “coarse_integration time registers, as described Figure 55.

$$\text{Frame_Rate[frame/s]} = \frac{1}{\text{Time_per_Line[sec]} \times (\text{Frame_Length})}$$

$$\text{Time_Per_Line_[sec]} = \frac{\text{Line_Length_pck[pix]}}{2 \times \text{Pix_Clock_Freq[MHz]}}$$

[In the case of (frame_length_lines - 4 > coarse_integration_time)]: Frame_Length = frame_length_lines
 [In the case of (frame_length_lines - 4 < coarse_integration_time)]: Frame_Length = coarse_integration_time + 4

Figure 55: Relationship between frame rate and register values [9]

The “line_length” and “frame_length” values determine the maximum size of the captured image for timing purposes, whereas the actual active data for the output is determined by different registers, namely “x_out_size” and “y_out_size”, the start and end pixel address registers “x_add_sta”, “x_add_end”, “y_add_sta” and “y_add_end” that determine which pixels are read out, as well as the “x_odd_inc” and “y_odd_inc” registers, that may be set to 1 or 3, depending on whether every other pair of pixels shall be skipped or not. Additionally, there are registers to determine the size of the data output in x- and y-direction “x_out_size” and “y_out_size”, although the available documentation does not go into detail regarding their exact function [9].

The pixel clock frequency in Figure 55 depends on the external clock frequency fed to the camera chip, which for the Raspberry Pi Camera module is 24MHz, and the values set for the clock dividers and multipliers illustrated in Figure 56. As can be seen, the IMX219 uses two PLLs to generate the clock signals for internal processes and the data output respectively. The internal clock frequency is determined by the desired image size and framerate (or exposure time) and the output clock frequency is determined by the desired DDR clock frequency for the CSI-2 transmission. Additionally, an output pixel clock is derived from this output clock that, depending on the output data format (RAW8 or RAW10) has one 8th or one 10th of the frequency.

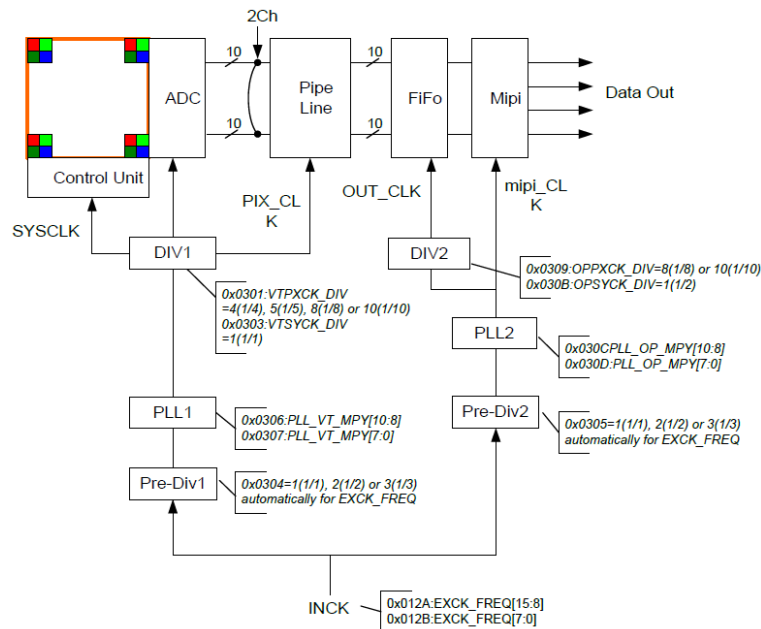


Figure 56: Clock system in the IMX219 [9]

The framerate desired in this project is nominally 60Hz. However, because of the limitations of the (simplified) PLL primitive in the GateMate, the actual output framerate is slightly lower, because the pixel clock runs at 25MHz rather than the nominal 25.175MHz. With 25MHz pixel clock frequency, the resulting framerate is 59.52 Hz. To match the incoming and outgoing framerates as closely as possible, the camera is set up accordingly. The clock setup is shown in Table 25, the corresponding setup register values can be found in Table 26.

Register	Value	Unit
External Clock Frequency	24	MHz
PLL clk in	8	MHz
PLL1 clk out	368	MHz
VT Pixel Clock	92	MHz
VT System Clock	368	MHz
PLL2 clk out	400	MHz
OP Pixel Clock	50	MHz
OP System Clock	200	MHz

Table 25: Chosen clock settings for the IMX219 in this project

With a VT Pixel Clock of 92MHz, a frame length of 862 (0x035E) and a line length of 3586 (0x0E02) a framerate of 59.92Hz results, according to the formula from Figure 55.

As shown in Table 26, the start and end addresses in the X-direction are 1000 and 1640 respectively for a line length of 640 pixels, which is also the value for the x_out_size. In the Y-direction, the start and end addresses are set to 750 and 1230, for an output size of 480 pixels, as is also written to the y_out_size register. The “odd_increment” for both directions is set to 1. The exact start and end addresses are arbitrarily chosen to be somewhere near the middle of the pixel array. Choosing odd address values may change the alignment of the Bayer-pattern, although this has not been tested. Binning is deactivated, the test pattern colours and offset, as well as the test pattern mode are set to 0. The test pattern width and height are set to 640 and 480 respectively to match the size of the active image so that activating a test pattern is as simple as changing the value for the test pattern mode.

Index	Name	Address	Value	Info
0	REG_MODE_SEL	16'h0100	8'h00	Mode Select: 0 Standby, 1 Streaming
1	REG_CSI_LANE	16'h0114	8'h01	CSI_lane_mode 1: 2-Lane, 3: 4-Lane (must be 1 for the camera module)
2	REG_DPHY_CTRL	16'h0128	8'h00	MIPI Global timing setting 0: auto mode, 1: manual mode
3	REG_EXCK_FREQ_MSB	16'h012A	8'h18	Reference clock frequency [MHz]
4	REG_EXCK_FREQ_LSB	16'h012B	8'h00	
5	REG_FRAME_LEN_MSB	16'h0160	8'h03	frame length in lines BINNING_MODE = 0,1,2 Unit: 1Lines BINNING_MODE = 3 Units: 2Lines
6	REG_FRAME_LEN_LSB	16'h0161	8'h5E	
7	REG_LINE_LEN_MSB	16'h0162	8'h0E	Line length in pixels
8	REG_LINE_LEN_LSB	16'h0163	8'h02	
9	REG_X_ADD_STA_MSB	16'h0164	8'h03	x_addr_start X-address of the top left corner of the visible pixel data Units: Pixels
10	REG_X_ADD_STA_LSB	16'h0165	8'hE8	
11	REG_X_ADD_END_MSB	16'h0166	8'h06	x_addr_end X-address of the bottom right corner of the visible pixel data Units: Pixels
12	REG_X_ADD_END_LSB	16'h0167	8'h68	
13	REG_Y_ADD_STA_MSB	16'h0168	8'h02	y_addr_start Y-address of the top left corner of the visible pixel data Units: Lines
14	REG_Y_ADD_STA_LSB	16'h0169	8'hEE	
15	REG_Y_ADD_END_MSB	16'h016A	8'h04	y_addr_end X-address of the bottom right corner of the visible pixel data Units: Pixels
16	REG_Y_ADD_END_LSB	16'h016B	8'hCE	
17	REG_X_OUT_SIZE_MSB	16'h016C	8'h02	output image size (X-direction) Width of image data output from the sensor module Units: Pixels
18	REG_X_OUT_SIZE_LSB	16'h016D	8'h80	
19	REG_Y_OUT_SIZE_MSB	16'h016E	8'h01	output image size (Y-direction) Height of image data output from the sensor module Units: Lines
20	REG_Y_OUT_SIZE_LSB	16'h016F	8'hE0	
21	REG_X_ODD_INC	16'h0170	8'h01	x_odd_inc Increment for odd pixels 1, 3
22	REG_Y_ODD_INC	16'h0171	8'h01	y_odd_inc Increment for odd pixels 1, 3
23	REG_BINNING_H	16'h0174	8'h00	defines binning mode (H-direction). 0: no-binning, 1: x2-binning, 2: x4-binning, 3: x2-analog (special) binning

24	REG_BINNING_V	16'h0175	8'h00	defines binning mode (V-direction). 0: no-binning, 1: x2-binning, 2: x4-binning, 3: x2-analog (special) binning
25	REG_CSI_FORMAT_C	16'h018C	8'h08	CSI-2 data format
26	REG_CSI_FORMAT_D	16'h018D	8'h08	CSI-2 data format
27	REG_VTPXCK_DIV	16'h0301	8'h04	vt_pix_clk_div Video Timing Pixel Clock Divider Value
28	REG_VTSYCK_DIV	16'h0303	8'h01	vt_sys_clk_div Video Timing System Clock Divider Value
29	REG_PLL_VT_MPY_MSB	16'h0306	8'h00	pll_vt_multiplier PLL Video Timing System multiplier Value
30	REG_PLL_VT_MPY_LSB	16'h0307	8'h2E	
31	REG_OPPXCK_DIV	16'h0309	8'h08	op_pix_clk_div Output Pixel Clock Divider Value
32	REG_OPSYCK_DIV	16'h030B	8'h01	op_sys_clk_div Output System Clock Divider Value
33	REG_PLL_OP_MPY_MSB	16'h030C	8'h00	pll_op_multiplier PLL Output System multiplier Value
34	REG_PLL_OP_MPY_LSB	16'h030D	8'h32	
35	REG_TP_RED_MSB	16'h0602	8'h00	Solid color test pattern Red component
36	REG_TP_RED_LSB	16'h0603	8'h00	
37	REG_TP_GREEN_MSB	16'h0604	8'h00	Solid color test pattern Green(R) component
38	REG_TP_GREEN_LSB	16'h0605	8'h00	
39	REG_TP_BLUE_MSB	16'h0606	8'h00	Solid color test pattern Blue component
40	REG_TP_BLUE_LSB	16'h0607	8'h00	
41	REG_TP_GREEN_MSB	16'h0608	8'h00	Solid color test pattern Green(B) component
42	REG_TP_GREEN_LSB	16'h0609	8'h00	
43	REG_TEST_PATTERN_MSB	16'h0600	8'h00	0000h – no pattern (default) 0001h – solid color 0002h – 100 % color bars 0003h – fade to grey color bar 0004h – PN9 0005h – 16 split color bar 0006h – 16 split inverted color bar 0007h – column counter 0008h – inverted column counter 0009h – PN31
44	REG_TEST_PATTERN_LSB	16'h0601	8'h00	
45	REG_TP_X_OFFSET_MSB	16'h0620	8'h00	test_pattern_window_x_offset
46	REG_TP_X_OFFSET_LSB	16'h0621	8'h00	
47	REG_TP_Y_OFFSET_MSB	16'h0622	8'h00	test_pattern_window_y_offset
48	REG_TP_Y_OFFSET_LSB	16'h0623	8'h00	
49	REG_TP_WIDTH_MSB	16'h0624	8'h02	test_pattern_window_width
50	REG_TP_WIDTH_LSB	16'h0625	8'h80	
51	REG_TP_HEIGHT_MSB	16'h0626	8'h01	test_pattern_window_height
52	REG_TP_HEIGHT_LSB	16'h0627	8'hE0	
53	REG_DIG_GAIN_GLOBAL_MSB	16'h0158	8'h01	digital gain
54	REG_DIG_GAIN_GLOBAL_LSB	16'h0159	8'h06	
55	REG_ANA_GAIN_GLOBAL	16'h0157	8'h90	analog gain
56	REG_INTEGRATION_TIME_MSB	16'h015A	8'h03	coarse_integration_time
57	REG_INTEGRATION_TIME_LSB	16'h015B	8'h5A	
58	REG_MODE_SEL	16'h0100	8'h01	Mode Select: 0 Standby, 1 Streaming

Table 26: Initialization sequence for the camera module

4.3.2 Initialization Control State-Machine

The state-machine handles the initialization of the camera module via the I²C bus. The state-machine has a total of seven states, as shown in Figure 57:

State 0: In the initial state the system is reset. The reset signal for the I²C master is asserted and the output register values as well as the register counting the number of failed write operations are set to 0. State 1 is entered automatically with the next clock cycle.

State 1: In this state the initializer is activated and the I²C write operation is prepared. The reset signal for the I²C master is de-asserted and the “run_init” and “init_read_EN” flags for the “INIT_IMX219” module are set. State 2 is entered automatically with the next clock cycle.

State 2: The register address and corresponding data is sent to the I²C master. State 3 is entered automatically with the next clock cycle.

State 3: In this step the transmission is started by setting the “i2c_enable” signal to 1. If the I²C master sets the “i2c_master_busy” flag, i2c_enable is set to 0 and state 4 is entered with the next clock cycle.

State 4: This state is responsible for error handling and detecting the end of the initialization. If the I²C master has asserted the “i2c_master_error” flag, the failed attempt counter (i2c_iterations) is incremented and state 1 is entered with the next clock cycle to repeat the attempt. If “i2c_iterations” reaches 4, the state-machine enters the error state (state 6). If no error is detected, the “step_increment” signal is set to 1, the attempt counter is reset and state 1 is entered with the next clock cycle, unless the “init_complete” flag has been set by the initializer. In the latter case, state 5 is entered with the next clock cycle.

State 5: This is the successful end-state. The “init_done” flag is set to indicate the successful initialization.

State 6: This is the error state. The “init_error” flag is set to indicate that an error has occurred, and the camera could not be initialized.

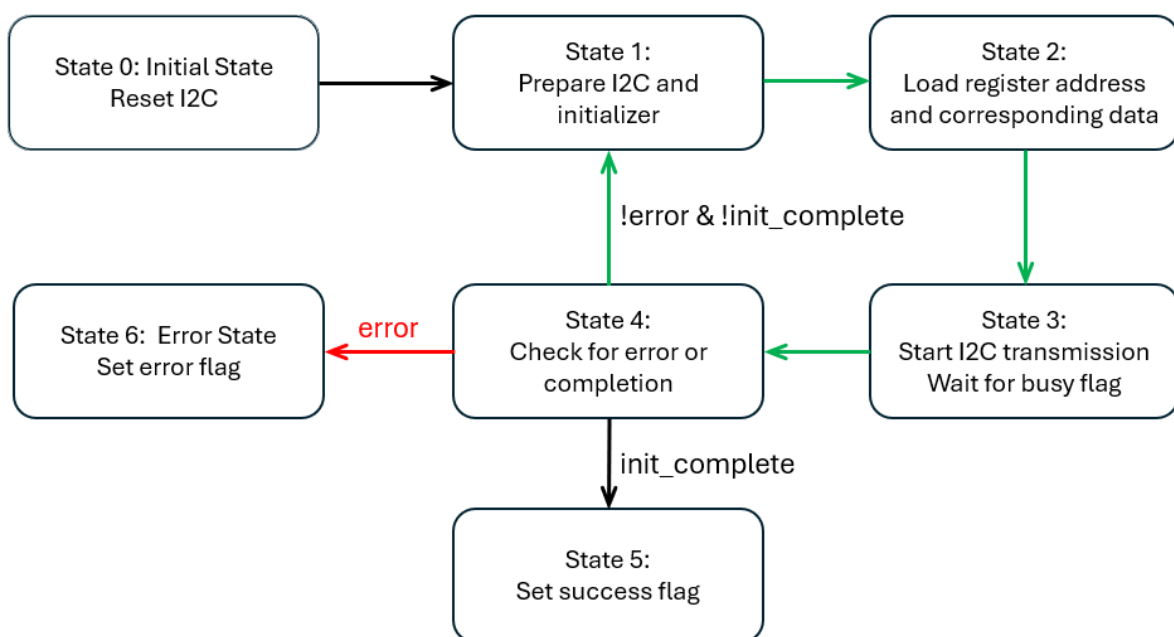


Figure 57: Flowchart of the state-machine controlling the initialization of the camera

4.4 Framebuffer

The framebuffer is used to store a video frame and plays a central role in synchronizing input and output of the demonstrator. It operates similarly to an asynchronous FIFO, using two independent clocks for writing and reading. However, compared to a generic asynchronous FIFO the overhead is reduced significantly. This can be achieved because the input and output occur at the same framerate (60Hz) with the differences being only the manner of transmission. The video output from the camera module uses a 200MHz clock, which is divided down to a 50MHz byte clock in the receiver that is used as the write-side clock in the buffer. The output side is clocked with the 25 MHz pixel clock that is used for the HDMI/DVI output. Additionally, the framebuffer is designed such that potential data loss due to asynchronous input and output is ignored without causing major image errors. The interfaces of the buffer with the other main components of the demonstrator are illustrated in Figure 58.

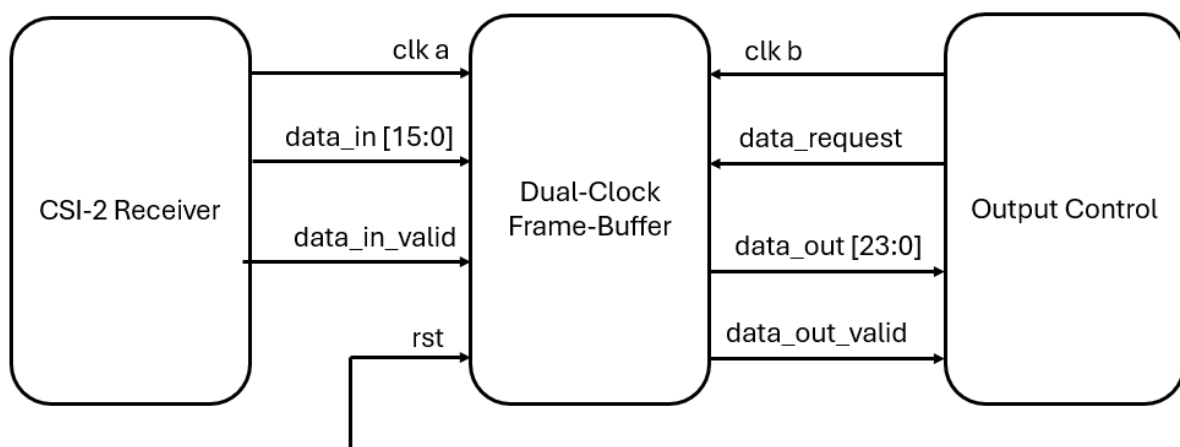


Figure 58: Simplified illustration of the framebuffer interface

Internally, the buffer uses the GateMate's embedded dual-port RAM blocks to store the image data. The memory is organized in a matrix with the dimensions of the incoming image, which is 640*480 in this project. Therefore, both for writing and reading, two counters are used respectively to track the current memory address. The details of the memory structure are defined in part by the physical structure of the RAM blocks in the FPGA and by the needs of the application.

As already mentioned in section 4.1, the on-chip RAM of the GateMate is not sufficient to store a full frame. The raw data for a single frame requires $640 \times 480 = 307200$ bytes but the GateMate only contains 32 RAM-blocks of 5120 bytes each (163840 bytes total). Therefore, the amount data to store needs to be reduced. This can be achieved by either reducing the resolution or the colour-depth of the image. In this project, it was decided to reduce the resolution of the stored image by a factor of four to reduce the required amount of memory to well below the capacity available in the GateMate. This was preferred to reducing the colour-depth because of the internal addressing scheme of the RAM blocks.

Table 2.2: 20K configurations				Table 2.3: 40K configurations			
Configuration	TDP	SDP	ECC	Configuration	TDP	SDP	ECC
(RAM size per 20K block)				32K × 1 bit	✓	✓	✗
16K × 1 bit	✓	✓	✗	16K × 2 bit	✓	✓	✗
8K × 2 bit	✓	✓	✗	8K × 5 bit	✓	✓	✗
4K × 5 bit	✓	✓	✗	4K × 10 bit	✓	✓	✗
2K × 10 bit	✓	✓	✗	2K × 20 bit	✓	✓	✗
1K × 20 bit	✓	✓	✗	1K × 40 bit	✓	✓	✓
512 × 40 bit	✗	✓	✓	512 × 80 bit	✗	✓	✓

Figure 59: Configuration options for block-RAM in GateMate [17]

As illustrated in Figure 59, the data width cannot be configured to be 8 or 16 bits, which is the width of the incoming data. This means that either, the memory blocks will not be fully utilized, which in turn means that either the colour-depth would have to be reduced to a potentially impracticable three bits, or disproportionately complex memory allocation would be required.

To configure the RAM-blocks the Verilog inference example for dual-port RAM from [34] is used. This means that an interface module, called “bram_dp_no_change” with parameters for address width and data width, is declared, which can then be instantiated in the framebuffer module. The module interface definition for the RAM submodule is shown in Listing 14. Aside from the two clock ports, the module has a “write enable” (wea, web), “data in” (dia, dib), address (addra, addrb) and “data out” port for both clock domains.

```

module bram_dp_no_change #(
    parameter DATA_WIDTH=18,
    parameter ADDR_WIDTH=9
)
    input wire wea,
    input wire web,
    input wire clka,
    input wire clk b,
    input wire [DATA_WIDTH-1:0] dia,
    input wire [DATA_WIDTH-1:0] dib,
    input wire [ADDR_WIDTH-1:0] addra,
    input wire [ADDR_WIDTH-1:0] addrb,
    output reg [DATA_WIDTH-1:0] doa,
    output reg [DATA_WIDTH-1:0] dob
);

```

Listing 14: Dual-clock RAM module interface definition

This module is used as the basic building block for the buffer around which the rest of the module is constructed. Here, two arrays of this module are instantiated, with “DATA_WIDTH” set to 16 to match the width of the input data and “ADDR_WIDTH” set to 10, so that each block has 1024 addresses. The reasoning behind this is given in the following.

The input counter increments with each rising edge of the write-side clock (CSI byte clock) when the “data_in_valid” signal is HIGH, counting the number of incoming byte-pairs. This allows a

simple implementation of a “save two, drop two” system, that is used for both lines and columns of pixels, effectively reducing the resolution of the stored image by a factor of four, relative to the incoming image. The saving of pairs is required due to the Bayer-filter used on the sensor. Each even line (starting at index 0) contains alternating red and green values and each odd line contains alternating green and blue values. To obtain a sensible full colour value, three neighbouring colour components (red, green, blue) are required.

Whereas the data input comes line by line, on the output side it is desirable to be able to read from two lines (one even and one odd) simultaneously, as it allows reading the full colour data for one pixel in one clock cycle. Therefore, two arrays of the RAM submodule are instantiated, one for even and one for odd lines.

Input Image Column Index		0, 1	4, 5	8, 9	12, 13	...	636, 637
Line Memory Address		0	1	2	3	...	159

Address → in Block	0-159	160-319	320-479	480-639	640-799	800-959
Blocks ↓	Line indices from input image					
Even 0	0	4	8	12	16	20
Odd 0	1	5	9	13	17	21
Even 1	24	28	32	36	40	44
Odd 1	25	29	33	37	41	45
...	...	...	...	...	...	...
Even 19	456	460	464	468	472	476
Odd 19	457	461	465	469	473	477

Figure 60: Mapping of incoming image data to the memory addresses

Normally the synthesis and implementation software allocate one 20K RAM-block for each instance of the RAM submodule. This complicates the line addressing, because it means that using one instance per line is not feasible, as it would require 240 RAM blocks and there are only 64 individually configurable blocks (of 20480 Bits) available. Therefore, several lines have to be stored in one block. With a data width of 16 Bits, and 2560 Bits in total for one line, it is possible to fit six lines of image data into each RAM-block. This requires some additional logic to map the pixel coordinates to the memory addresses. The resulting structure is illustrated in Figure 60. The upper table illustrates how the original column indices of the pixels that are stored map to the addresses in the RAM-blocks, using the first even line as an example. In the lower table, the mapping of incoming image lines to the individual RAM-blocks is shown. Each line represents a RAM-block, each column an address region within the RAM-blocks and the numbers in the table are the line indices from the incoming image.

The total number of blocks is calculated as follows: Half of the 480 lines, that is 240, are to be stored. Divided by six lines per block the result is 40 blocks. Of these, half are even lines and half are odd, which use the same indices, so that there are 20 “block addresses” in total. From each line 160 pairs of values are stored, the other 160 are discarded.

In Listing 15 the most important write-side signals in the Verilog implementation of this addressing scheme are shown. The style is deliberately verbose to make the code easier to understand. Each

RAM-block has separate enable (en_a, en_b) and write-enable (we_a, we_b) signals, both of which need to be set HIGH to write to the memory with the respective clock. Here, the signals “en_a” and “we_a” are used for the write-side, while en_b controls the readout.

```
//write control
    wire we_a;
    wire en_a, en_b;
//combinatorial
    wire store_pixel;
    assign store_pixel = (col_cnt_i[0] == 0 & lin_cnt_i[1] == 0);
    assign en_b = (data_available & data_request);
    assign data_out_valid = data_available;
    assign we_a = data_in_valid & en_a; //always write when valid data available
    assign en_a = store_pixel;
//signals for rams, write side
    wire[15:0] wr_data_even, wr_data_odd;
    wire[9:0] wr_addr_even, wr_addr_odd;
    reg [2:0] block_lin_cnt = 0; //this counts the lines written to the current block
    reg[6:0] wr_sel = 0; //this denotes which pair of RAM blocks is currently used
    reg [9:0] wr_offset = 0;

//write control
    wire wr_odd_line;
    assign wr_odd_line = lin_cnt_i[0];
    assign wr_addr_even = (col_cnt_i >> 1) + wr_offset;
    assign wr_addr_odd = wr_addr_even; //always same addresses for either line
    assign wr_data_even = data_in;
    assign wr_data_odd = data_in;
```

Listing 15: Write-side signals and asynchronous logic in the framebuffer

The state of the enable signals depends on the current counter values “col_cnt_i” and “lin_cnt_i” and the “data_in_valid” signal, that comes from the CSI receiver module. Writing is enabled, when the column index is even, half the line index is even, and “data_in_valid” is HIGH. This results in the desired “save two, drop two” operation for both lines and columns.

The full address of any memory location consists of three components: The pair of RAM-blocks, denoted by “wr_sel”, the column address “wr_addr_even” and “wr_addr_odd” (these are effectively the same signal) and the odd/even indicator “wr_odd_line”, that is equal to the 0-bit of the line counter. Because several lines are stored in each block, an additional counter “block_lin_cnt” to keep track of the number of lines written to the current block and an offset register “wr_offset” are required. After each line, “wr_offset” is incremented by the number of stored columns per line.

Listing 16 depicts a code snippet that shows how the counters cooperate. While “data_in_valid” is HIGH, the column counter is incremented with every rising edge of the write clock until it reaches a set number. It is then reset to 0 and the line counter is incremented. Once the second line (index 1) is written, the “data_available” flag is set to indicate that data can now be read and

the line count for the current block and the write offset are incremented. Once the block line count reaches 5, it is reset along with the write offset, and “wr_sel” is incremented.

```
//write side counter(s)
always@(posedge clk_a or negedge write_rst_n) begin
    if(write_rst_n) begin
        if(data_in_valid) begin
            col_cnt_i <= (col_cnt_i >= COLUMNS - 1) ? 0 : col_cnt_i + 1; //increment until COLUMNS-1, then reset to 0
            if (col_cnt_i >= COLUMNS-1) begin
                lin_cnt_i <= (lin_cnt_i == (12*LINES-1)) ? 0 : lin_cnt_i + 1; //incrment to LINES-1, then reset to 0
                if (wr_odd_line & (lin_cnt_i[1] == 0)) begin
                    data_available <= 1; //set data_available when 2 lines are written
                    block_lin_cnt <= block_lin_cnt + 1; //increment lines in current ram block
                    wr_offset <= wr_offset + OS_INCREMENT; //increment addr offset after every odd line
                    if (block_lin_cnt == 5) begin
                        wr_sel <= (wr_sel == LINES/2-1) ? 0 : wr_sel + 1;
                        wr_offset <= 0; //reset wr_addr offset
                        block_lin_cnt <= 0;
                    end
                end
            end
        end
    end
end
```

Listing 16: Write-side address counter operation in the framebuffer

At the end of each frame the write-side counters are reset to their initial values. This is important because the timing on the write-side depends entirely on the CSI-2 receiver and there is no logic to react to errors in the receiver. Consequently, if individual packets (which contain one line of image data) are not received correctly, the counter values will no longer match the correct pixel indices, leading to distortions in the output image. By resetting the counters after each frame, such distortions will not be carried over to the following frames. Important to note here is that this reset only works correctly if the CSI clock runs continuously during frame transmission and stops in between frames, as is the case with the IMX219 camera output.

The read logic works similarly, although there are some key differences. While the addressing scheme is simplified by the fact that odd and even lines are read simultaneously, every line must be read out four times before moving on to the next one. An overview of the signals for controlling the readout is given in Listing 17. Note that, in this configuration, “data_out” only accesses three quarters of the data written to the memory, as only one of the two green colour values is used per full-colour pixel.

```
//read control
wire[15:0] rd_data_even [(LINES/2)-1:0];
wire[15:0] rd_data_odd [(LINES/2)-1:0];
wire [9:0] rd_addr_even, rd_addr_odd;
reg[1:0] hold_lin = 0; //counter to enable reading each line 4 times
reg [6:0] rd_sel = 0;
reg [3:0] ram_pos = 0; //which of the 6 lines is read from current block
reg [9:0] rd_offset = 0;
assign data_out = {rd_data_even[rd_sel][7:0], //red
                  rd_data_odd[rd_sel][7:0], //green
                  rd_data_odd[rd_sel][15:8]}; //blue
assign rd_addr_odd = (col_cnt_o >> 2) + rd_offset; //rd_offset is multiple of LINELENGTH/2
assign rd_addr_even = rd_addr_odd;
```

Listing 17: Read-side signals and asynchronous logic in the framebuffer

In contrast to the write-side, the read side counters for columns and lines (col_cnt_o and lin_cnt_o) go through the full range of 640 columns and 480 lines for the output frame. Therefore,

the read address is determined by the column count shifted two bits to the right (divided by 4) plus the read offset. Similarly to “block_lin_cnt” on the write side, the counter “ram_pos” counts the number of lines that have been read from the current block, so the address offset can be adjusted accordingly.

The counter operation on the read-side is depicted in Listing 18. While “en_b” is HIGH, i.e. when data is available to read and the output module sets the “data_request” signal, the column counter is incremented with each rising edge of the read clock. When it reaches the final index of a line (here 639), the total line counter and the “hold_lin” counter are incremented. Once the latter reaches three, it is reset and “ram_pos” is incremented, along with the read offset. This means that each stored line is read four times to output a full 640*480 image. Once all six lines have been read from the current pair of blocks, the cycle repeats until the full frame has been read and the next frame begins.

```

always@(posedge clk_b) begin
    if(rst_n) begin
        if (en_b) begin
            col_cnt_o <= (col_cnt_o == LINELENGTH-1) ? 0 : col_cnt_o + 1;
            if (col_cnt_o == LINELENGTH-1) begin
                lin_cnt_o <= (lin_cnt_o >= 12*LINES-1) ? 0 : lin_cnt_o + 1;
                hold_lin <= hold_lin + 1;
                if (hold_lin == 2'd3) begin
                    hold_lin <= 0;
                    ram_pos <= ram_pos + 1;
                    rd_offset <= rd_offset + OS_INCREMENT; //increment addr offset
                    if (ram_pos == 5) begin
                        ram_pos <= 0;
                        rd_sel <= (rd_sel == (LINES>>1)-1) ? 0 : rd_sel + 1;
                        rd_offset <= 0;
                    end
                end
            end
        end
    end
end
end

```

Listing 18: Read-side counter operation in the framebuffer

An important feature of this buffer architecture is that the read and write operation are essentially completely independent from one another. This may lead to some data being lost if the framerates on the input and output side do not match, but it achieves the required tasks of compensating for the mismatch in the read and write speeds and preventing catastrophic errors on either input or output side, both of which do not allow for any pauses in transmission outside the dedicated blanking periods.

4.5 DVI/HDMI Output

To display the video data received from the camera, a pseudo-HDMI output is used, employing an HDMI port and cable, but using the DVI protocol for the video transmission, which any HDMI device is required to accept [1]. This way the video output is easily implemented and accepted by all HDMI displays. The transmitter used in this project is based on an existing design for a DVI output from [35] and adapted for the demonstrator. The main components are a module called “vga_core” that handles the generation of the control signals “data enable” (de), “Hsync” and “Vsync”, and another module called “dvi_core” that is responsible for encoding the data and control signals for the TMDS transmission, as well as the serialization of the data streams. The bit clock and pixel clock are generated using a PLL for the 125MHz bit clock and a clock divider that divides this clock down to the 25MHz for the pixel clock. In addition, a simple colour balancing module is used. All these components are bundled into an output module called “video_output_ctrl”, the structure of which is sketched in Figure 61.

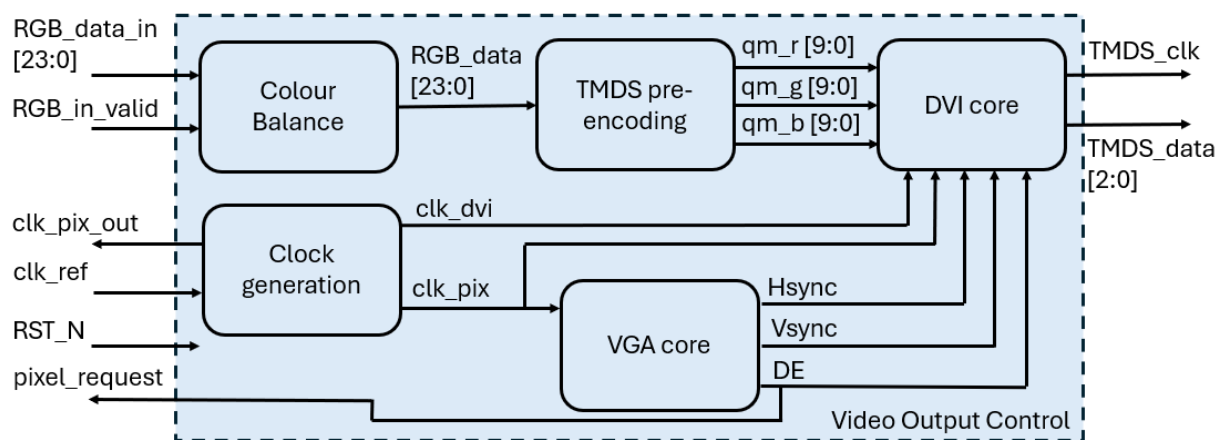


Figure 61: Block diagram of the internal structure of the output module

Beside the sub-modules instantiated, the output control module contains logic to synchronize the output to the input. This is implemented by holding the components that determine the output timing (VGA core, DVI core) in reset until the “RGB_in_valid” signal is HIGH, indicating that data is available to read from the framebuffer. Additionally, all non-clock inputs to the DVI core are fed through registers to compensate for the delay imposed by the colour balancing and ensure correct timing.

The output control module does not contain the output buffer instantiation for the TMDS-link. Instead, they are instantiated in the overall top module. Since the LVDS output buffers are the only type of differential output buffer available in the GateMate FPGA, these are used for the TMDS-link, although they do not meet the specification for TMDS shown in section 2.3.4. AC-coupling is used between the output buffers and the TMDS receiver of the display (see section 3.2.4) to prevent issues arising from the mismatch in common-mode voltage between the two sides of the interface, with around 1.25V at the LVDS output buffer and around 3.3V at the TMDS receiver. With the LVDS buffers configured in “boost” mode (meaning the buffer drives an output current of 6.4mA), this causes a 640mV voltage drop across the termination resistance of $2 \times 50\Omega$ in the TMDS receiver, which is well within the receivers’ specified range of between 120mV and 1200mV.

4.5.1 Colour Balance

As already mentioned in section 4.1, raw data from a camera chip produces “unnatural” looking images, depending on the ambient lighting. In order to create a more pleasant image, the intensity of the individual colour components (here red, green and blue) is adjusted, according to the lighting situation. A simple way to achieve this is by multiplying the different components with different modifiers. Usually, rational modifiers between 0 and 2 are used to keep the colour values within the available range (0 to 255 for byte size integer values) and not distort the colour too much. Unfortunately, rational multiplication requires relatively complicated hardware, so this approach is not really suited for this demonstrator. However, by first dividing the values by a power of 2, using bit-shift operations and subsequently multiplying them with an integer value it is possible to achieve effectively rational factors. For example, a shift to the right by 2 bits equals (approximately) a division by 4, so that any integer multiple of quarters (rounded to the next integer) of the initial value can be obtained through multiplication with an integer value. The interface definition of the colour balance module is shown in Listing 19.

```
module color_balance #(
    parameter INT_GAIN_RED = 4'd4,
    parameter INT_GAIN_GREEN = 4'd4,
    parameter INT_GAIN_BLUE = 4'd4,
    parameter SHIFT_DIV = 3'd2
)(
    input clk,
    input [23:0] rgb_in,
    input rgb_in_valid,
    output reg [23:0] rgb_out,
    output reg rgb_out_valid
);
```

Listing 19: Colour balance module interface definition

The module uses a registered output to prevent any timing issues due to long combinatorial paths. By this means a static delay of one clock cycle is introduced. To prevent overflow of the colour values, the multiplication results are nine bits wide. If the MSB is 1 in the result, that colour component will be assigned the value 0xFF instead. The full code of the module body is shown in Listing 20.


```

reg valid_in_reg; //this is used to delay the incoming valid signal
wire[8:0] pre_red, pre_green, pre_blue;
wire [7:0] red, green, blue;

assign pre_red = (rgb_in[23:16] >> SHIFT_DIV) * INT_GAIN_RED;
assign pre_green = (rgb_in[15:8] >> SHIFT_DIV) * INT_GAIN_GREEN;
assign pre_blue = (rgb_in[7:0] >> SHIFT_DIV) * INT_GAIN_BLUE;

assign red = (pre_red[8]) ? 8'hFF : pre_red;
assign green = (pre_green[8]) ? 8'hFF : pre_green;
assign blue = (pre_blue[8]) ? 8'hFF : pre_blue;

always@(posedge clk) begin
    valid_in_reg <= rgb_in_valid;
    if (valid_in_reg) begin
        rgb_out <= {red, green, blue};
        rgb_out_valid <= 1;
    end
    else begin
        rgb_out <= 24'd0;
        rgb_out_valid <= 0;
    end
end
endmodule

```

Listing 20: Verilog code for colour balance module

4.5.2 VGA Core

This module generates the timing signals for the video output, namely “data enable”, “Hsync” and “Vsync”. Data enable is high when image data is transmitted and low during blanking periods. Hsync (short for horizontal synchronization) denotes the transition between image lines, whereas Vsync (short for vertical synchronization) denotes the transition between frames. The exact timings and the polarity of the Hsync and Vsync pulses depend on the video format and are specified in [3].

The VGA core uses two counters, clocked with the pixel frequency of the output video format, representing the pixel coordinates in the virtual frame (the active image plus the “blanking pixels”), one for the horizontal and one for the vertical axis. The timings are hardcoded as local parameters in the Verilog code and are by default set up for the 640*480@60Hz format. As in [3], the timing parameters are given in numbers of pixels at a given pixel frequency, which is approximately 25MHz for the format used here. The pixel frequency of 25MHz deviates slightly from the 25.175MHz specified in [3], because the (simplified) PLL primitive of the GateMate does

not allow the required fine grain control of the output frequency. In practice however, this deviation does not appear to cause any issues.

```

wire vsync_s      = ~(vcount_s >= V_START_SYNC && vcount_s < V_END_SYNC);
wire clear_vcount_s = (vcount_s == V_MAX_COUNT-1);

wire de_s        = (hcount_s <= H_RES-1 && vcount_s <= V_RES-1);

```

Listing 21: Excerpt from Verilog code of VGA core showing how signals are determined based on counters

The state of the output signals “data enable”, “Hsync” and “Vsync” is determined by the counter values and the timing parameters, as illustrated in Listing 21. The result of the combinatorial logic is output synchronously using registers.

4.5.3 DVI Core

The DVI Core module bundles the instances of the TMDS encoder and serializer modules for the four TMDS channels, with the data channels being almost entirely identical. Only the blue channel receives the Hsync and Vsync signal to transmit the corresponding codes during blanking intervals. The clock channel only transmits a continuous clock signal but uses the same serializer module as the data channels as a simple way to ensure synchronicity with the data.

Because the TMDS encoding scheme (described in detail in section 2.3.2) comprises some long combinatorial paths, most notably the step of XORing (or XNORing) each data bit with the result of the previous XOR/XNOR operation and the disparity calculation, the encoder module from [35] was split into two parts with a register stage in between. This way the most time-consuming steps of the encoding process can be executed in successive clock cycles, reducing the risk of timing violations. The first part of the encoding process, the XOR/XNOR operations and the counting of “1” bits in the input byte, was moved outside of the DVI core for timing purposes. The interface of the DVI core was modified to reflect that the module now receives 10-bit “pre-encoded” colour values, instead of 8-bit integer colour values. Listing 22 shows part of the “pre-encoder” module, including the module interface. The output of this purely combinatorial module is tied to a register in the output control module.

```

module tmds_pre_encoder(
    input [7:0] i_data,
    output [8:0] qm_o
);
    /* generate xor and xnor representation */
    wire [8:0] dat_xor_s, dat_xnor_s;
    assign dat_xor_s[0] = i_data[0];
    assign dat_xnor_s[0] = i_data[0];
    assign dat_xor_s[8] = 1'b1;
    assign dat_xnor_s[8] = 1'b0;
    genvar l_inst;
    generate
    for (l_inst = 1; l_inst < 8; l_inst = l_inst + 1) begin
        assign dat_xor_s[l_inst] = i_data[l_inst] ^ dat_xor_s[l_inst-1];
        assign dat_xnor_s[l_inst] = ~(i_data[l_inst] ^ dat_xnor_s[l_inst-1]);
    end
endgenerate

```

Listing 22: TMDS pre-encoder

The serializer module was also modified slightly, due to issues with clock timing. “Optimizations” during synthesis and implementation caused malfunctions in the clock serializer, because the output shift register was implemented incorrectly, which led to a timing mismatch between the clock and data signals. To remedy this, the HDL code was adapted to better reflect the actual hardware implementation. By splitting the output shift register, whose positions 0 and 1 are tied to the inputs of a DDR output register and that shifts the data by 2 bits every clock cycle, into two separate registers with their “0” position tied to the DDR register and shifting only by one bit per clock cycle. Additionally, a *(*keep*)* inline attribute is used to minimize changes to this construct during synthesis.

<pre> reg [9:0] dat_pos; always @(posedge fast_clk_i) begin if (ref_clk_i_edge) dat_pos <= dat_i; else dat_pos <= {2'b0, dat_pos[9:2]}; end CC_ODDR #(.CLK_INV(1'b0)) ddr_inst (.CLK(fast_clk_i), .DDR(fast_clk_i), .D0(~dat_pos[1]), .D1(~dat_pos[0]), .Q(dat_o)); </pre>	<pre> reg [4:0] dat_pos_even; reg [4:0] dat_pos_odd; wire[4:0] evens; wire[4:0] odds; assign evens = {dat_i[8], dat_i[6], dat_i[4], dat_i[2], dat_i[0]}; assign odds = {dat_i[9], dat_i[7], dat_i[5], dat_i[3], dat_i[1]}; (*keep*) always @(posedge fast_clk_i) begin if (ref_clk_i_edge) begin dat_pos_even <= evens; dat_pos_odd <= odds; end else begin dat_pos_even <= {0, dat_pos_even[4:1]}; dat_pos_odd <= {0, dat_pos_odd[4:1]}; end end CC_ODDR #(.CLK_INV(1'b0)) ddr_inst (.CLK(fast_clk_i), .DDR(fast_clk_i), .D0(dat_pos_odd[0]), .D1(dat_pos_even[0]), .Q(dat_o)); </pre>
---	--

Listing 23: Comparison between old serializer code (left) and new serializer code (right)

In Listing 23 the differences between the initial serializer (left) and the edited version (right) are shown. Note that, while the code is significantly more verbose, the shift register structure is functionally identical. Another change can be seen in the assignment to the output DDR register, where an unnecessary inversion was removed, that required an additional inversion of the image data when passing it to the DVI output module.

4.6 Top Module

The modules presented in the previous section are tied together into the full demonstrator in a top module. Here, the connections between the different sub-modules and the interfaces of the system with the outside world are defined. For the demonstrator this top module is called “csi_top”. In this module, the components “csi_rx_top”, “video_output_ctrl”, “CCI_handler” and “RAM_buffer” are instantiated and linked together, along with the LVDS output buffers for the TMDS output, which are set to “LVDS boost” mode. Additionally, a simple module is included, which holds the whole system in reset for 64 cycles of the external reference clock after startup. Beside the necessary inputs and outputs, additional GPIO ports are defined, that may be used for additional functions, such as outputting signals for debugging and verification. In the version for final testing, these signals include the CSI byte clock, the “done” signal of the CCI module, the good and bad checksum counters, a register that latches to 1 once a bad checksum is detected and the “csi_raw_valid” signal from the CSI-2 receiver.

5 Initial Testing

The functionality of the design is verified first in a simulation and then with the real hardware. In the simulation, only the components in the video data path are tested, whereas the hardware test necessarily also requires the CCI component to function, so that the camera can be initialized. The CCI module, as well as the VGA and DVI modules for the output, have been successfully used in almost identical form in a different project and are therefore expected to function. Consequently, it was decided to spare the effort of simulation tests for these modules, unless problems occur, that would justify them.

5.1 RTL-Simulation

The simulation is performed using IcarusVerilog, an open-source Verilog simulator. To view the resulting .vcd-files, the open-source waveform viewer GTKWave is used. For the simulation, the system is split into two parts. The first is the D-PHY/CSI-2 receiver, from the input buffers to the internal data output (“csi_raw_data”), and the latter comprises the framebuffer and a slightly modified version of the video output module.

In this project, simulation was used primarily to verify the basic functionality and get a better understanding of how the individual components of the system behave. Post-synthesis or post-implementation simulations were not performed. Extensive verification by simulation requires not only more powerful simulation tools than IcarusVerilog, but also very carefully designed testbenches that accurately represent the behaviour of the real components. While this kind of verification is a requirement for integrated circuit designs, where prototyping is very expensive, this is not necessarily the case for FPGA designs. Loading a different configuration into the FPGA is cheap and fast and has the benefit of operating in real hardware, albeit with severely limited observability compared to simulation data. Therefore, throughout the design process, simulations were also used to identify the causes of issues that were observed in the physical hardware test. In the following sections a selection of important simulation results is presented.

5.1.1 D-PHY/CSI-2 Receiver

The simulation requires a testbench, written in Verilog, that simulates the incoming signals from the camera. As the camera uses a D-PHY link with two data lanes, this is also what is simulated. The testbench makes use of nested Verilog tasks to simplify the process of configuring the simulated transmission. The fundamental building block is “send_mipi_byte” which generates the corresponding waveforms in clock and data lanes for an arbitrary input byte. This is then used in the “send_mipi_clock” task, that assigns an all 0 byte to the send_mipi_byte task. These can then be combined into “send_mipi_line” and “send_mipi_frame” tasks to construct a realistic input data stream with relatively little effort, while the code is also relatively easy to understand for humans. The testbench is configured to send a simulated “frame” consisting of one frame start short packet and ten long packets (or lines) with a length of 24 payload bytes plus the four-byte header and a two-byte footer. For the data lines, an example sequence with a known checksum is used in the simulation, so that the local checksum generation can be easily tested.

Table 27 shows the structure of the simulated frames, how the bytes are distributed on the data lanes and what the expected input for the align word module looks like. The parts marked as “Filler” emulate the times in between HS bursts, where the D-PHY clock continues running, but no data is transmitted. The data stream is deliberately misaligned with the byte clock to test the byte alignment.

Packet	Component	Lane0	Lane1	{Lane1, Lane0}	
Frame Start	Filler	0x00	0x00	0x0000	
	Short Packet	0xB8	0xB8	0xB8B8	
		0x12	0x00	0x0012	
		0xFF	0x00	0x00FF	
	Filler	0x00	0x00	0x0000	
Repeat Filler once					
Long Packet	Filler	Repeat Filler 7 times			
	Long Packet Header	0xB8	0xB8	0xB8B8	
		0x2A	0x18	0x182A	
		0x00	0x13	0x1300	
	Payload	0xFF	0x00	0x00FF	
		0x00	0x02	0x0200	
		0xB9	0xDC	0xDCB9	
		0xF3	0x72	0x72F3	
		0xBB	0xD4	0xD4BB	
		0xB8	0x5A	0x5AB8	
		0xC8	0x75	0x75C8	
		0xC2	0x7C	0x7CC2	
		0x81	0xF8	0xF881	
		0x05	0xDF	0xDF05	
		0xFF	0x00	0x00FF	
		0x00	0x01	0x0100	
		Footer	0xF0	0x00	0x00F0
		Filler	0x00	0x00	0x0000
	Repeat Filler 8 times				
	Repeat Long Packet 9 times				

Table 27: Structure of the test input for simulation

Figure 63 shows a selection of waveforms from the simulation. It depicts the DDR clock, byte clock, one of the deserializer sub-modules and the data registers of the align byte modules for both data lanes. It can be seen that the byte clock generation, deserialization and the byte alignment work as intended, with one small exception: The synchronisation byte is not aligned properly, because the correct alignment is only set up with the next rising edge of the byte clock. This is not a problem however, because the following bytes are properly aligned, and the synchronisation byte is discarded by the word align module anyway, regardless of its content. The header and the first payload data bytes of one of the long packets are shown to also be received as expected in Figure 62.

word_in[15:0] =	0000	2EAE	182A	1300	00FF
word_dly_1[15:0] =	0000		2EAE	182A	1300
word_dly_2[15:0] =	0000			2EAE	182A
word_out[15:0] =	0000			182A	1300
valid_out =					

Figure 62: Simulation results showing the header bytes being the first valid output of the align word module

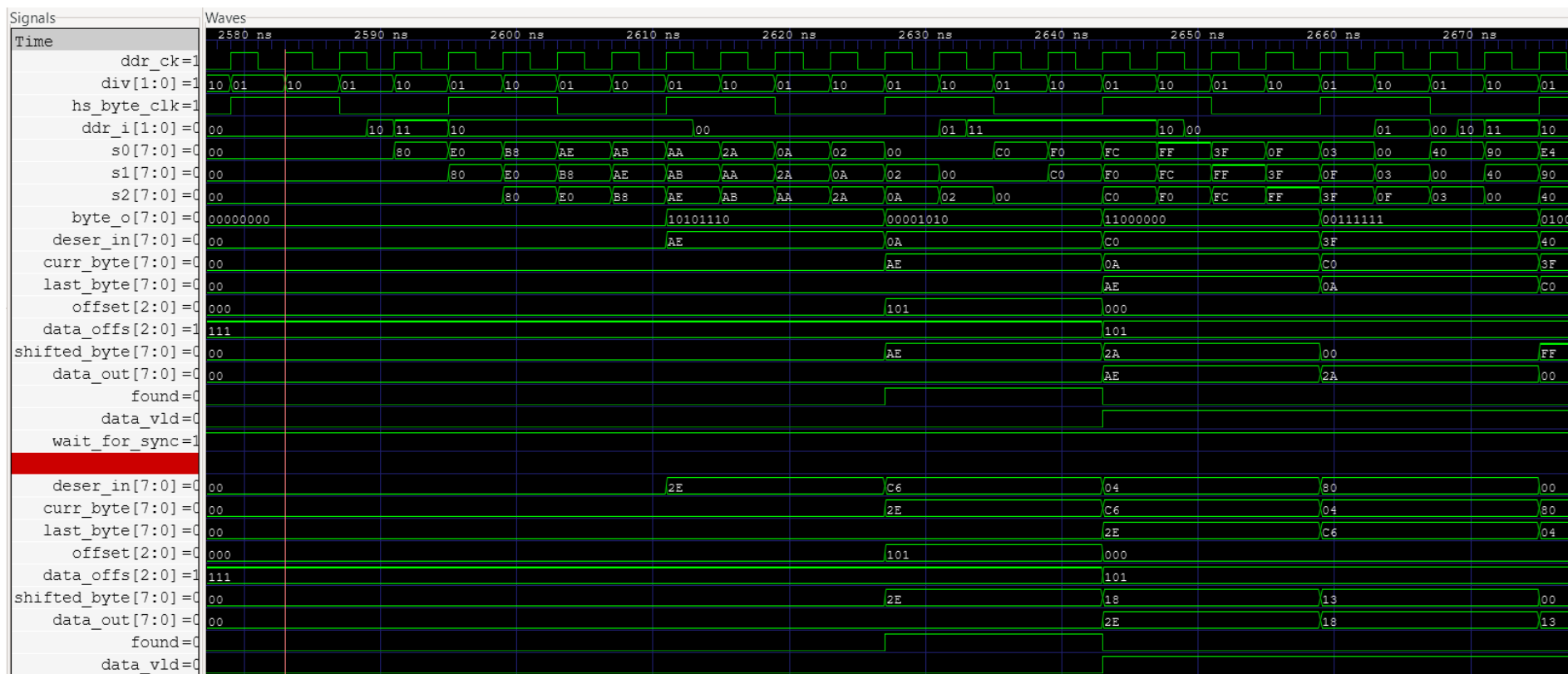


Figure 63: Selection of signals from simulation. Depicted are the registers of one of the deserializers and the byte aligners from both data lanes at the start of transmission.

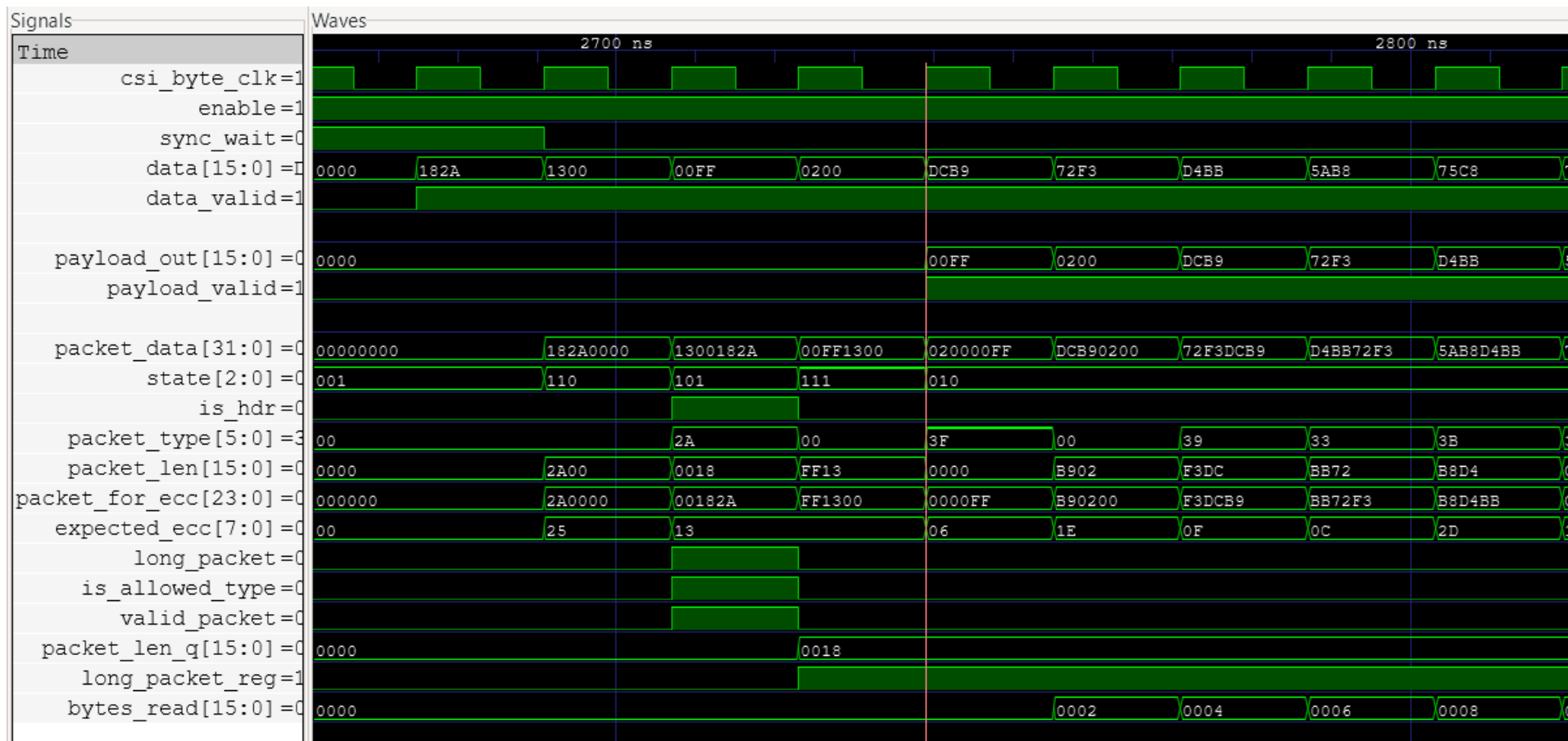


Figure 64: Selected simulated waveforms from the “packet handler” module at the start of a long packet

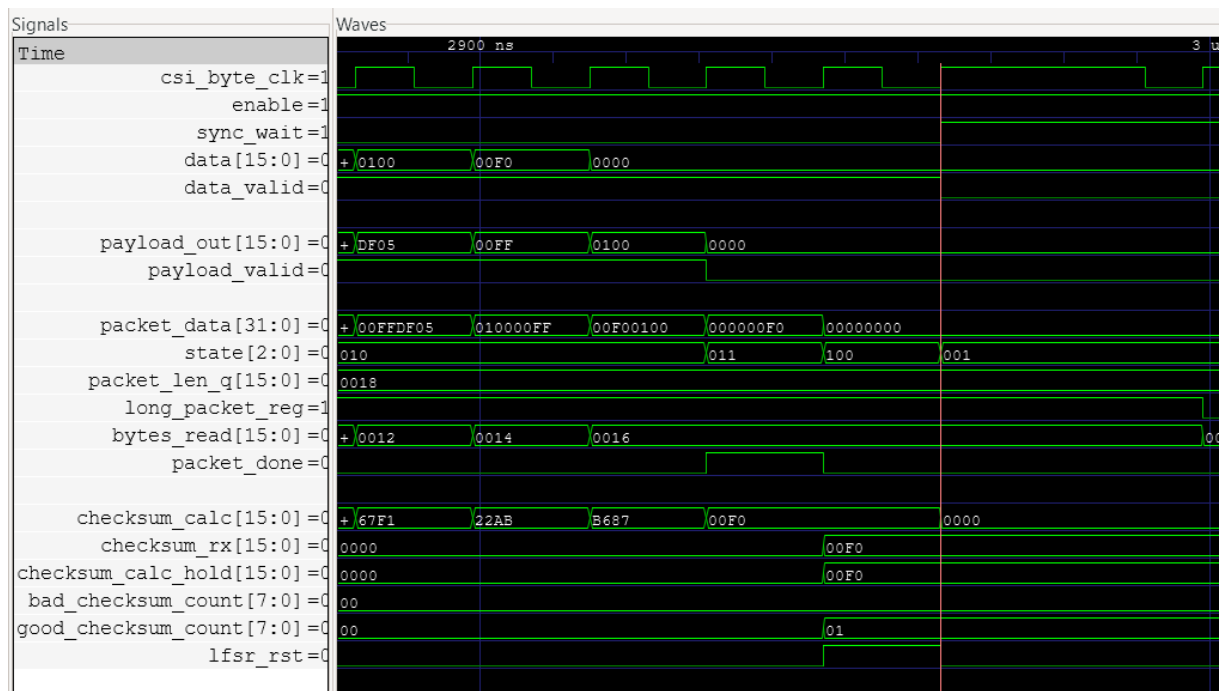


Figure 65: Selected simulated waveforms from “packet handler” at the end of a long packet

The word alignment stage is followed by the packet handler that deals with the CSI-2 protocol, effectively separating the payload data from the protocol overhead. Figure 64 shows the start of a simulated long packet reception. The header is correctly interpreted, and the first payload byte is forwarded to the output along with the “payload_valid” signal. It can be seen how the state-machine cycles through the expected states (1 – 6 – 5 – 7 – 2) and the output bytes are counted. In an interesting coincidence the ECC generated from the ID and word count bytes of the header is the same as the one generated from the header’s ECC and the first two payload bytes of this test sequence.

The end of the packet, from the last payload bytes to the end of transmission, is shown in Figure 65. It shows that the end of the payload data is correctly identified, and a correct checksum has been generated by the LFSR, the appropriate “good_checksum_count” is incremented and the LFSR is reset at the intended time. Finally, the state machine returns to the waiting state “1” and the system is ready for the next packet. Note that there is a discontinuity in the simulated clock signal, which however does not affect the operation.

With this simulation, it is confirmed that the system works as intended in ideal circumstances. The next step of the verification process is to test the design under more realistic conditions.

Overall, the simulation shows the write side of the buffer to work as intended, although this is predicated on the assumption that the simulation mimics the behaviour of the actual hardware with sufficient accuracy.

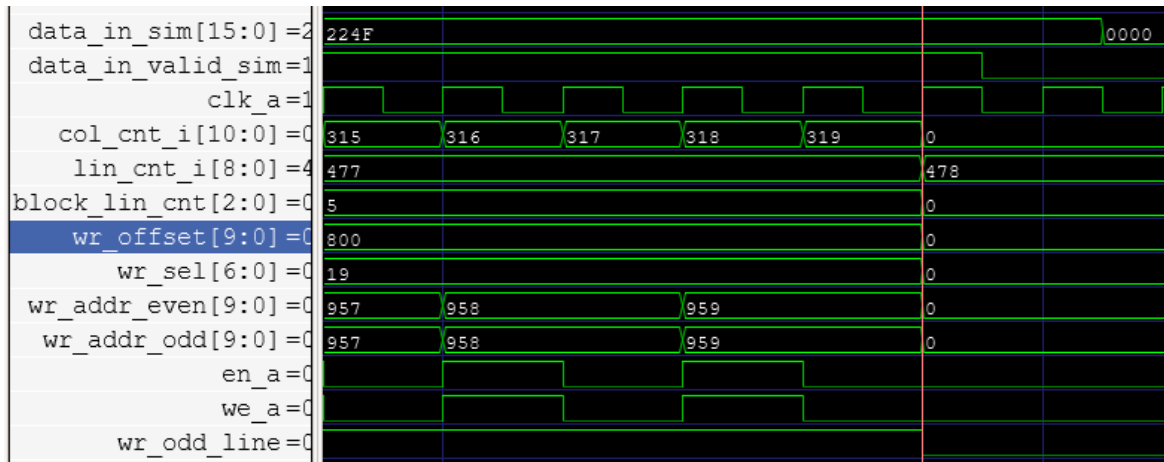


Figure 67: Simulated waveforms from framebuffer write side at the end of a frame write operation

With the write-side operation verified, now the read side must be examined in the same way. The important signals on the read side at the end of a frame read are shown in Figure 68. As expected, the column counter resets to 0 after reaching the final value of 639, along with all the other counters and address registers for the read side. It can also be seen that the next frame read begins immediately, as long as the “data_request” signal is set, which is fixed for this simulation but normally controlled by the output control module.

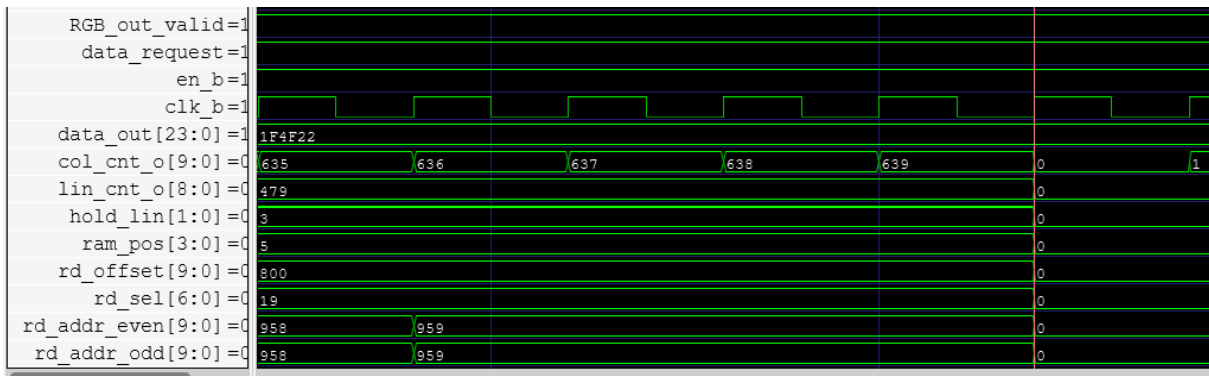


Figure 68: Simulated waveforms from framebuffer read side at the end of a frame read operation

In addition to the main functionality, the write-side reset, which shall occur after every incoming frame, is tested. For this, input clock “clk_a” is paused for a short time, to simulate the pause in the CSI byte clock that occurs between each frame transmission. Figure 69 provides a selection of relevant signals, including the write-side column counter and address registers, as well as the read-side counters and address. As intended, only the write-side is affected, while the read side continues to operate uninterrupted.

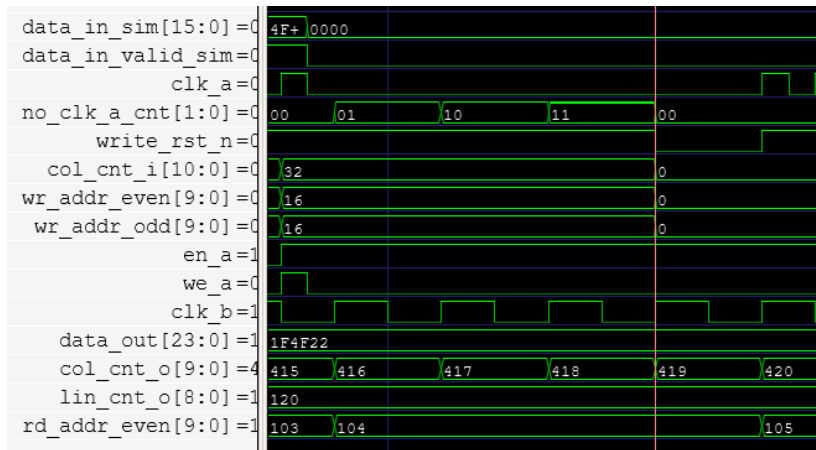


Figure 69: Simulated waveforms of read and write counters during write-side reset

The second simulation serves to verify the output logic, most crucially the horizontal and vertical pixel counters in the “vga core” that determine the output timing, and the serializer. Incorrect timing of the output signals or errors in the serializer can lead to distortions of the displayed video or even make the signals unintelligible for the display.

Figure 70 shows the counters and output signals of the “vga core”, as well as the format parameters used for the simulation. The values for active resolution and blanking intervals have been set to small numbers to limit the resource requirement for simulating an entire frame. The counters reset to 0 after reaching the maximal values of H_MAX_COUNT - 1 (here 17) and V_MAX_COUNT - 1 (here 14) and the negative polarity “hsync” and “vsync” pulses have the desired length.

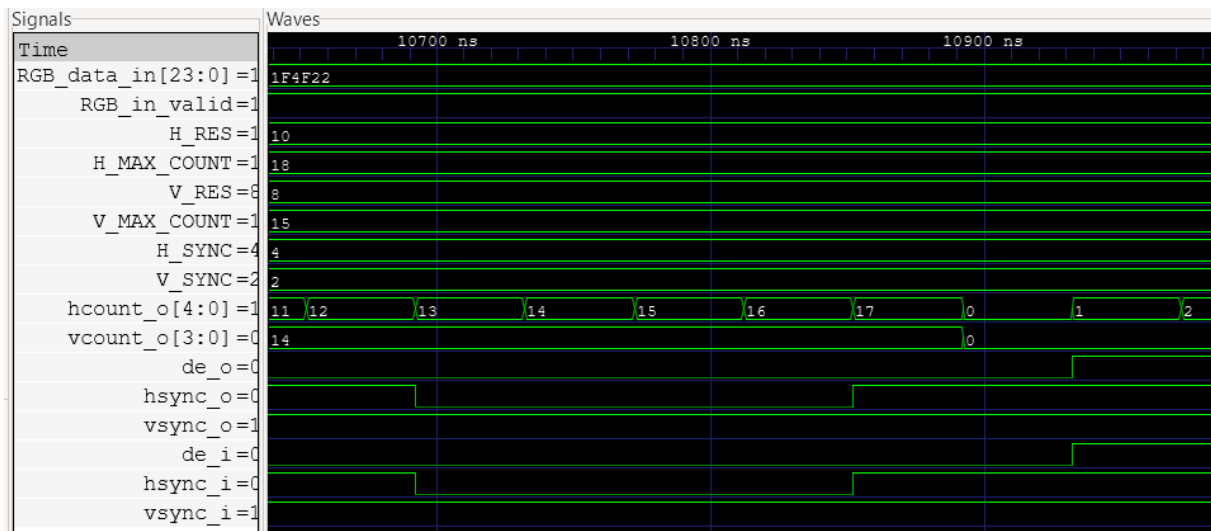


Figure 70: Simulated waveforms from output simulation at the end of a frame

Another crucial part is the operation of the serializer sub-module that is used for both the data and clock transmission via the TMDS-link, with the “clock serializer” receiving a fixed 10-bit input word (10'b0000011111). The waveforms of the most important registers in the serializers for the clock (top group) and one data lane (bottom group) are presented in Figure 71. The serializer is shown to work correctly in both cases. It should be noted that dat_o is the output of a DDR register and updates its value with both edges of “fast_clk_i”, which is the DVI bit clock. Between the incoming pixel clock and the outgoing pixel clock, a 180° phase-shift can be observed. The delay between these clock signals can be controlled via the fixed input word for the clock serializer.

However, this is usually not an issue because the pixel clock edges do not need to be aligned with the first data bit of the data words.

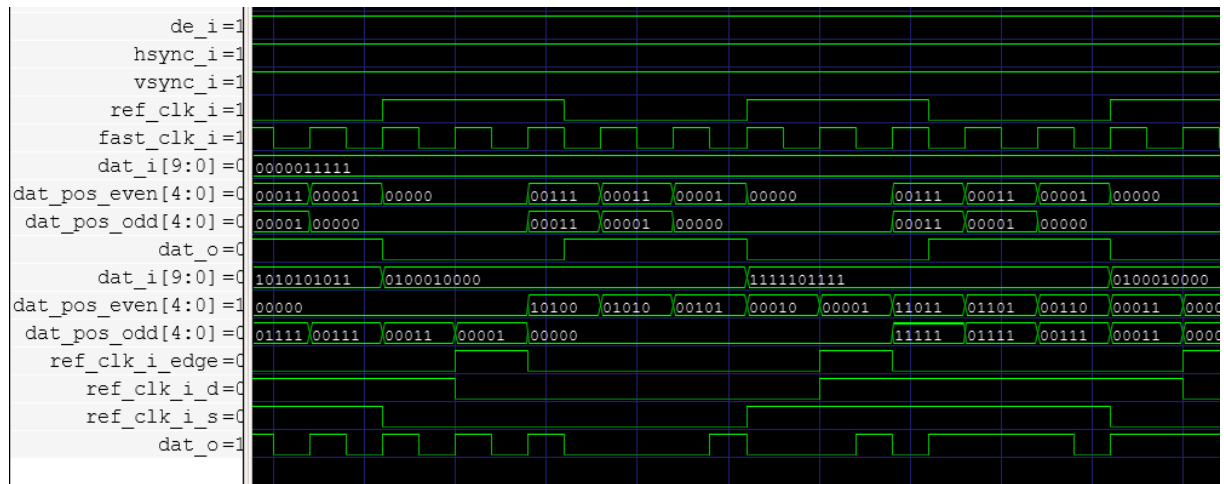


Figure 71: Simulated waveforms of serializer operation

Finally, the buffer and output modules are simulated together in a single testbench to verify that no timing errors are introduced by the interface between them. To limit the amount of generated simulation data, the simulated timeframe is kept short. The simulation ends when about half of the second line of image data has been transferred because the start of transmission is the only critical point that has not been covered by the previous simulations.

The most critical part of the interaction between the two modules is the timing that might be thrown off by the logic for holding the output until enough data is available and the delay introduced by the colour correction module, whose function can also be evaluated based on this simulation.

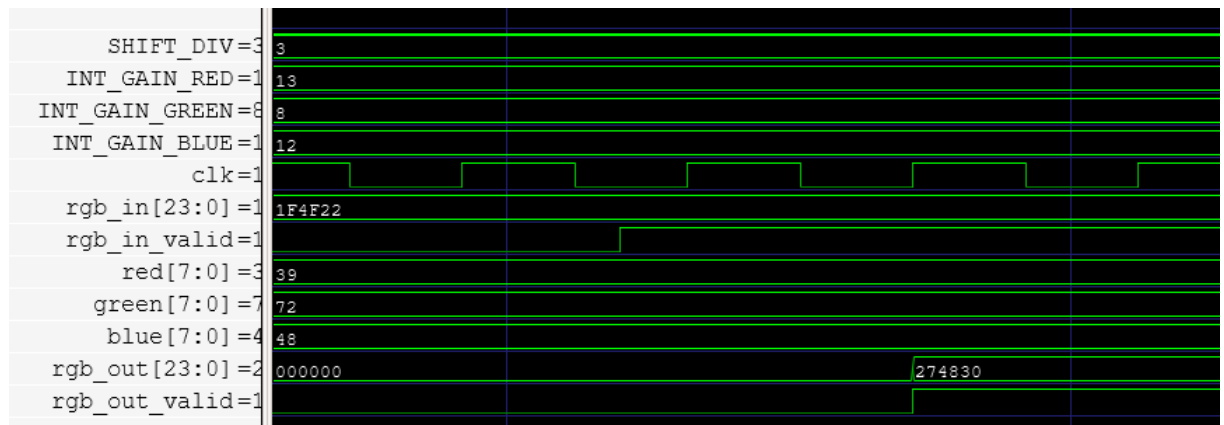


Figure 72: Simulated waveforms of colour balance module

Figure 72 depicts waveforms from the colour correction module in the simulation. On the top are the configuration parameters used in the simulation, as decimal numbers. “SHIFT_DIV” is set to 3, so the input colour values are shifted to the right by 3 bits, which is equivalent to an integer division by 8. The results of this operation are then multiplied by the “INT_GAIN” values to determine the output colour values red, green and blue. An analysis of the resulting output values reveals some weaknesses of the concept. The actual ratio of output to input values depends on the input values and may lead to colour distortions and noise in the output. This is because the division by bit-shifting yields inaccurate results for values that are not integer multiples of 8. For

example, all input values between 24 and 31 will yield 3 as a result. To illustrate, a comparison between the ideal ratios and the real ratios seen with the data used in the simulation is given in Table 28. While this is not a problem for this demonstrator, the architecture is unsuitable for applications where image quality is important, as it reduces the effective colour depth.

Colour	Multiplier	Input Value	Ideal Output	Ideal Ratio	Actual Output	Actual Ratio
RED	13	31	50,375	1,625	39	1,258
GREEN	8	79	79	1	72	0,911
BLUE	12	34	51	1,5	48	1,412

Table 28: Comparison of intended and actual ratios between input and output of colour balance module

A more extensive selection of signals is given in Figure 73. The excerpt shows the simulation states where the framebuffer asserts the “data_valid” signal to the output control and the video output is started. Here, it is crucial that the data is correctly aligned with the control signals “de”, “hsync” and “vsync”. The signals “de_s”, “hsync_s” and “vsync_s” are generated by the “vga core” module, and are synchronously forwarded to the dvi module via the registers “de_reg”, “h_sync” and “v_sync”. The data output “RGB_data_cor” from the colour correction module is asynchronously processed by the TMDS “pre_encoder” that puts out the 9-bit XOR or XNOR representations of the input bytes, depending on the number of “1s” in them, as defined in the DVI specification. The pre-encoder outputs are fed via the registers “qm_r”, “qm_g” and “qm_b” to the DVI module.

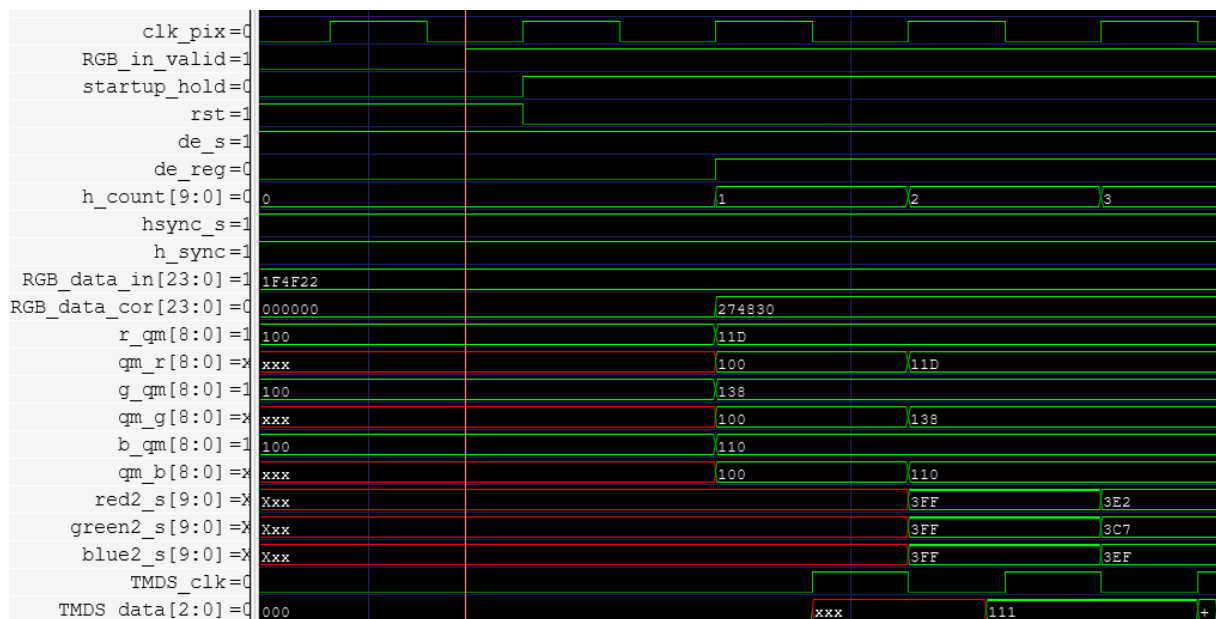


Figure 73: Simulated waveforms from output module during start of transmission

In Figure 73 the start of the video output is shown. Once the “RGB_in_valid” signal at the output control module is asserted by the framebuffer, the reset holding the operation of the output is lifted with the next rising edge of the pixel clock. One clock cycle later, the data and control signals for the DVI module are set. It can be observed that the colour information for the first pixel is not the actual data, but the default “0” output of the colour correction module. This is not a major problem however, because the delay only occurs at the start of transmission, when “RGB_in_valid” signal is set, which happens only once at startup, and only the initial data bytes are lost and the mapping of the pixels to the output image is not impaired beyond this first pixel.

The issue could possibly be addressed by using the “RGB_out_valid” signal from the colour correction module but it is very likely that the first pixels, likely even the entire first frame are not displayed anyway, as the receiving display needs some time to synchronize with the incoming data stream and the DVI specification gives 100ms after the start of the clock transmission as the upper time limit for clock synchronization [4].

Based on the RTL simulations, the buffer and output system function as intended under ideal circumstances and are ready for testing in real hardware.

5.2 Hardware Tests

After verifying the design of the receiver logic in simulation where signal delays, switching times and other complicating factors that occur in real hardware are ignored, it must be ensured that the design also works in the FPGA. The easiest way to test is to simply load the design into the FPGA with the camera module and display connected to the board and to monitor the setup's operation. Before testing the full system, however, the input and output side of the demonstrator are tested separately.

5.2.1 D-PHY Input Test

Some monitoring is required to verify the operation of the D-PHY receiver and CCI module. For this purpose, the integrated logic analyser (ILA) from Cologne Chip is used to monitor the state of the output signals of the word aligner module. This minimalistic approach is chosen because the ILA requires additional hardware to be embedded into the design, which can be problematic, if the timing is critical, as it is with this design. For this same reason, the CSI-2 protocol handling, which includes the LFSR for the CRC was left out of this test setup, as it caused problems with the ILA. If the initialisation of the camera works and the video data is received properly, the first four bytes received in each burst transmission must be the expected header bytes. As the camera is set up for an output format of 640*480p@60Hz in an 8-bit RAW format, the headers for the payload data bursts (excluding synchronisation packets and generic data packets) are expected to look as shown in Table 29.

Position	Data Lane	Meaning	Expected Value (bin)	Expected Value (hex)
0	0	Virtual channel ID and data type	0010_1010	2A
1	1	Word count LSByte	1000_0000	80
2	0	Word count MSByte	0000_0010	02
3	1	ECC	0000_1110	0E

Table 29: Expected header and distribution on the data lanes

The virtual channel feature for CSI-2 is not used in this implementation, so the first two bits of the ID-byte are 0, the remaining bits are 0x2A, being the data type code for RAW8. Since in the RAW8 format the word count is equal to the horizontal resolution (number of pixels per line), the expected value is 0x0280 (640 in decimal). It is important to take into account that the least significant byte is transmitted first. The error correction code for this header can be calculated by the method described in [7] and is expected to be 0x0E. The ILA captures the state of pre-selected signals within the FPGA fabric and sends them to a computer via the GateMate Programmer's USB-interface. This data is stored in a .vcd file which can then be viewed using GTKWave or any other suitable waveform viewer.

Additional monitoring of the CCI module's operation is not necessary, if the camera puts out data in the expected format. If the camera module is left unpowered for some time before starting the test, the correct operation is implied in this case because the camera stores configuration data only in volatile memory and the default settings differ significantly from the settings used here.

The GateMate ILA is a system that combines a python-based software component with a hardware component that is embedded into the design under test on the FPGA [36]. The python program handles the configuration of the ILA, the communication of the host computer with the GateMate and the generation of the necessary hardware. The hardware part is responsible for collecting the actual data samples. To be able to use the ILA for the intended test of the D-PHY

receiver, some modifications are necessary. Primarily, a PLL must be instantiated in the design, to provide a continuous free-running clock for the ILA hardware, that is sufficiently fast to correctly sample the data, without exceeding the recommended maximum clock frequency for the ILA of 160MHz. Thus, the clock for the ILA is set to run at 100MHz, allowing two samples for each cycle of the byte clock. The other modification compared to the simulation is that the maximum value for the counter imitating the function of the low-level protocol module is increased to 484, which equals the number of expected payload bytes plus the packet header. This is not strictly necessary for this test, since a lower counter limit would only lead to a premature reset of the synchronisation status, so that the receiver would begin looking for the next synchronisation sequence before the end of the packet transmission. However, realistic packet lengths make it easier to spot errors, like unexpectedly long downtime between two packets that would indicate that some packets are not received.

The ILA is set up using a python script that guides the user through the configuration steps, including selection of a clock and the signals to monitor, the number of samples to collect at a time and more. An important part of this setup is selecting the trigger, that marks the starting point of the data collection. In the test done here, the trigger is set to be the rising edge of the word_valid signal, as this marks the start of the output data stream of the receiver. As a compromise between the amount of additional logic required for the ILA and the size of a continuous sample, the ILA is set up to gather 16384 samples in one run. Figure 74 depicts a screenshot showing the part of the setup where the sample count is selected.

```
Select signals to be analyzed (0 = finish, f = filter): 0

!!!!!!!!!!!!!!!!!!!!!! Note !!!!!!!!!!!!!!!!!!!!!!!
!
! The capture duration must be defined.
! The maximum duration depends on:
! - available ram
! - width of the sample
! - sampling frequency
! FIFO Cascade (Width x Depth)
! FIFO (Input Width x Depth)
!
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!

-----Please choose one of the following durations: -----
+-----+-----+-----+-----+-----+
| # | smp_cnt | duration [us] | FIFO Cascade | FIFO |
+-----+-----+-----+-----+-----+
| 1 | 4096 | 40.96 | 2 x 1 | 10 x 4096 |
| 2 | 8192 | 81.92 | 4 x 1 | 5 x 8192 |
| 3 | 12288 | 122.88 | 2 x 3 | 10 x 4096 |
| 4 | 16384 | 163.84 | 4 x 2 | 5 x 8192 |
| 5 | 20480 | 204.8 | 2 x 5 | 10 x 4096 |
| 6 | 24576 | 245.76 | 4 x 3 | 5 x 8192 |
| 7 | 32768 | 327.68 | 4 x 4 | 5 x 8192 |
| 8 | 40960 | 409.6 | 4 x 5 | 5 x 8192 |
| 9 | 49152 | 491.52 | 4 x 6 | 5 x 8192 |
+-----+-----+-----+-----+-----+

Total Capture duration (choose between 1 and 9):
```

Figure 74: Excerpt from ILA configuration showing the selection of sample count

The hardware test with the settings described above yielded the waveforms shown in Figure 75. In this zoomed out view, nine equally spaced patterns are visible, with the word_valid signal being high and the output of the align word module changing, indicating that data is being received and put out by the align word module. The equal length and spacing of “packets” in the waveform indicate that the reception works consistently. A look at the timescale reveals that the reception

of each packet, including the blanking interval, takes roughly 19µs, so that 480 packets would take 9120µs or 9.12ms. As this is well below the period time of a frame at 60 frames per second of about 16.67ms. There is strong indication that all packets that were sent within the monitored timeframe were properly received.

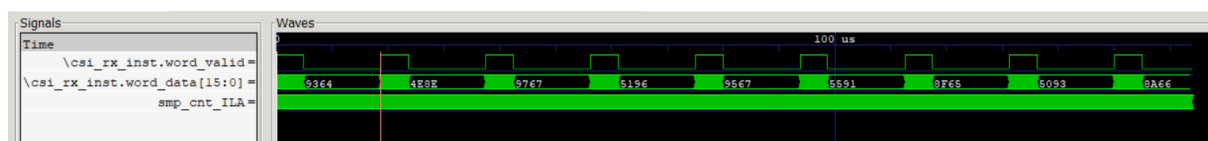


Figure 75: Zoomed out view of data stream captured by ILA

A closer look at the waveforms is shown in Figure 76. In this closeup view, the actual data, formatted in hexadecimal numbers, can be read. The first two samples contain the first four bytes of the transmission, which contain the packet header. The data word is the concatenation of the bytes from the two data lanes {lane1, lane0}, so the expected header as described in Table 29 should appear as 0x802A, then 0xE02, as seen in the waveform diagram.

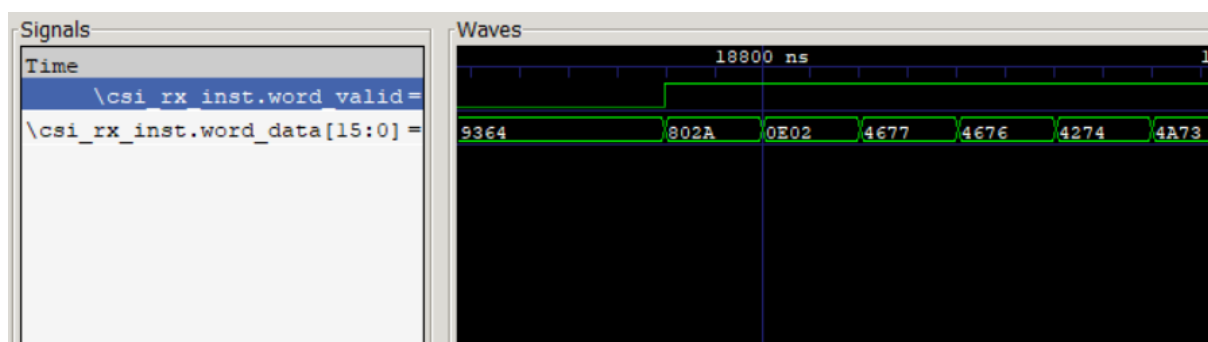


Figure 76: Zoomed in view of start of a packet, showing the expected header bytes

An examination of all headers in the sample shows that all headers are received according to the expectation. Repeated tests with this setup showed consistently good results. Changes in the ILA setup however, especially when data in the lane modules was to be sampled, lead to malfunctions, such as an apparent inability of the receiver to detect the synchronisation sequence on both data lanes simultaneously. It could not be verified whether the reception was actually not working or the data gathered by the ILA was corrupted. In either case, the likely reason is that the additional hardware required for the ILA causes suboptimal signal routing that does not meet the timing requirements, either in the design under test, the ILA hardware or both.

The results of the hardware test demonstrate that the camera has been initialized properly, and the receiver works in the physical hardware. Combined with the results of the simulation this demonstrates that the design is functional and can be used as a basis for a full CSI-2 receiver implementation.

5.2.2 Output Test

A functional test of the output system, beginning at the framebuffer, was done using a dedicated testbench design. The top module instantiates only the framebuffer and the output components of the demonstrator. An additional module generates a test image in the FPGA, which is then written to the framebuffer and then output to a display. The test image consists of four equally sized parts with distinct colours, which makes potential errors in the output easy to identify.

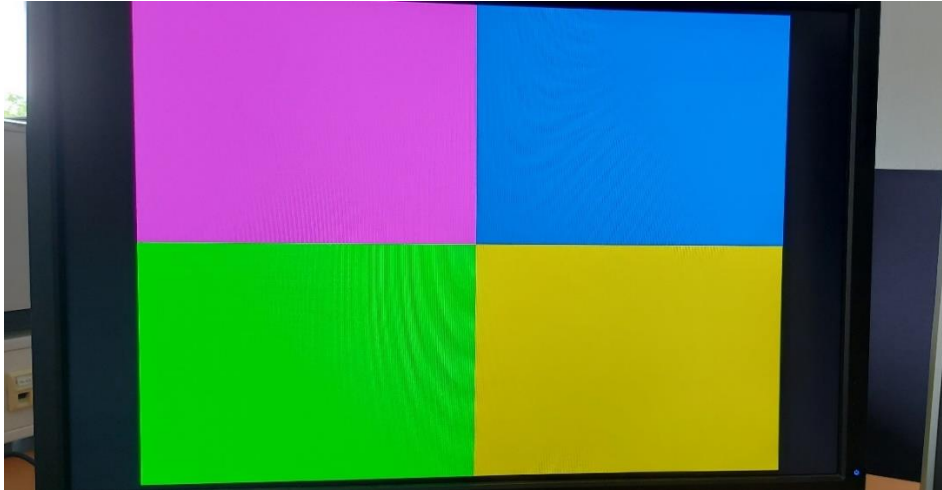


Figure 77: A photo of the test frame on a PC monitor

The resulting test image output on a PC monitor is shown in Figure 77. Note that the image is a photograph of a screen and therefore contains some artifacts, most notably the chromatic aberration at the border between the magenta and green sections, which does not appear like this on the monitor.

5.2.3 Full System Test

Once the input and output components are verified to work as intended, they are combined into the full demonstrator, synthesized and implemented on the GateMate FPGA for a final test of the entire system. The most important performance indicator is the video output seen on the test monitor. A photograph of the system in action is given in Figure 78.

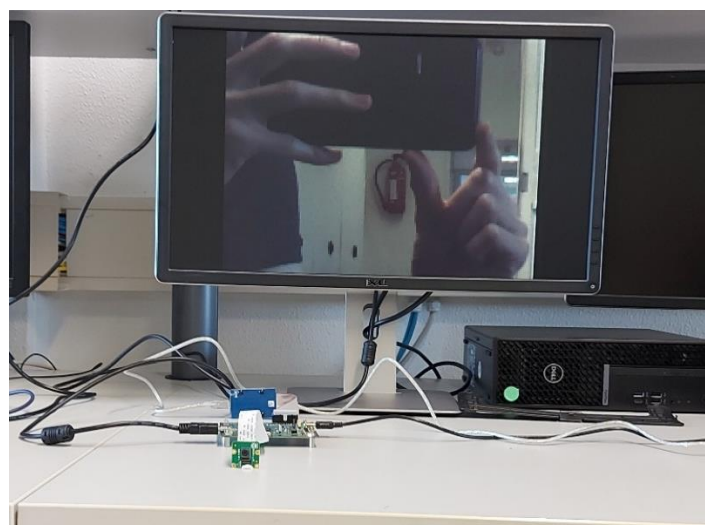


Figure 78: A photo of the demonstrator filming the author taking the picture

The system performs the intended function of receiving and immediately displaying video data. For a more in-depth characterization several signals were analysed, using an oscilloscope. All signals were measured using single-ended, active probes with a bandwidth of up to 1GHz.

First, the incoming signals are measured to check if the camera operates as expected. The CSI-2/D-PHY clock signal was observed to have the expected frequency of 200Mhz and the byte clock derived from it was observed running at the correct frequency of 50MHz, exactly one quarter of the incoming DDR clock. Figure 79 and Figure 80 show the activity on the clock lane and one data lane of the CSI-2 link on a 10ms/division timescale. At this timescale the actual clock and data signals cannot be displayed, but instead the individual frame transmissions can be identified. The clock lane enters the stop state between each frame transmission, so the signal level rises to 1.2V. From the distance between the rising edges of the clock lane signal, the time per frame can be estimated to be around 16ms.

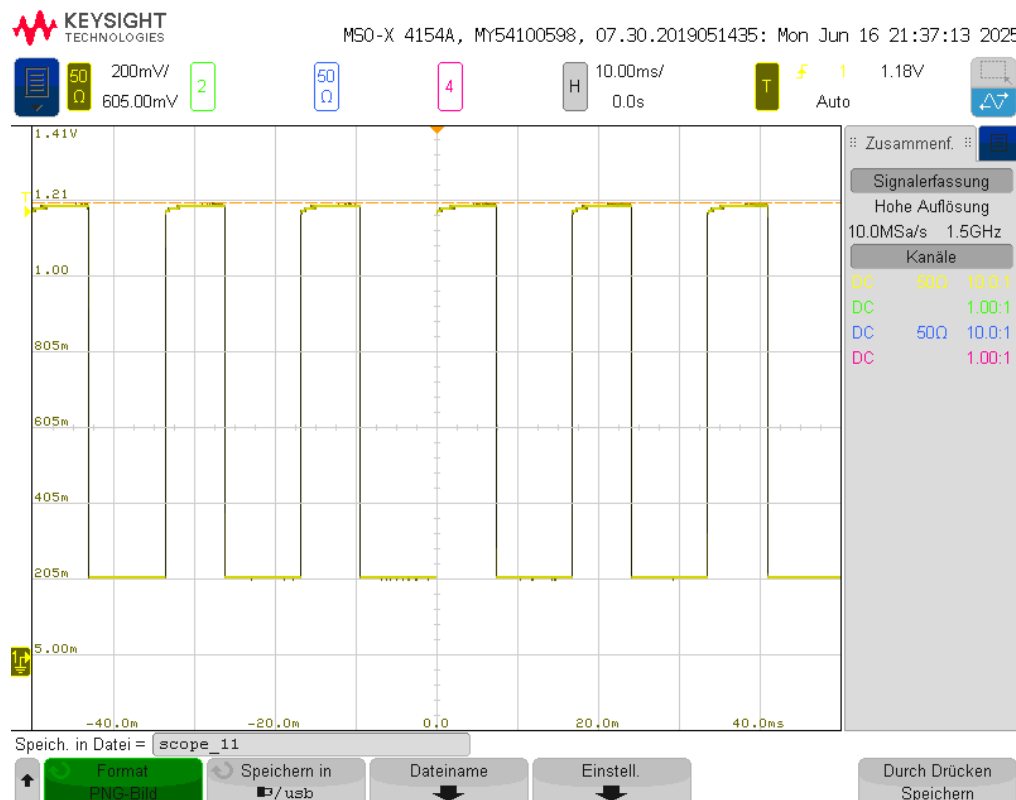


Figure 79: 100ms sample of the D-PHY clock lane activity

A similar pattern is found in the signal on the data lane in Figure 80, with the key difference being the visible activity in between the stop states. This is because the data lanes enter the stop state between each packet transmission, while the clock runs continuously during the frame transmission.

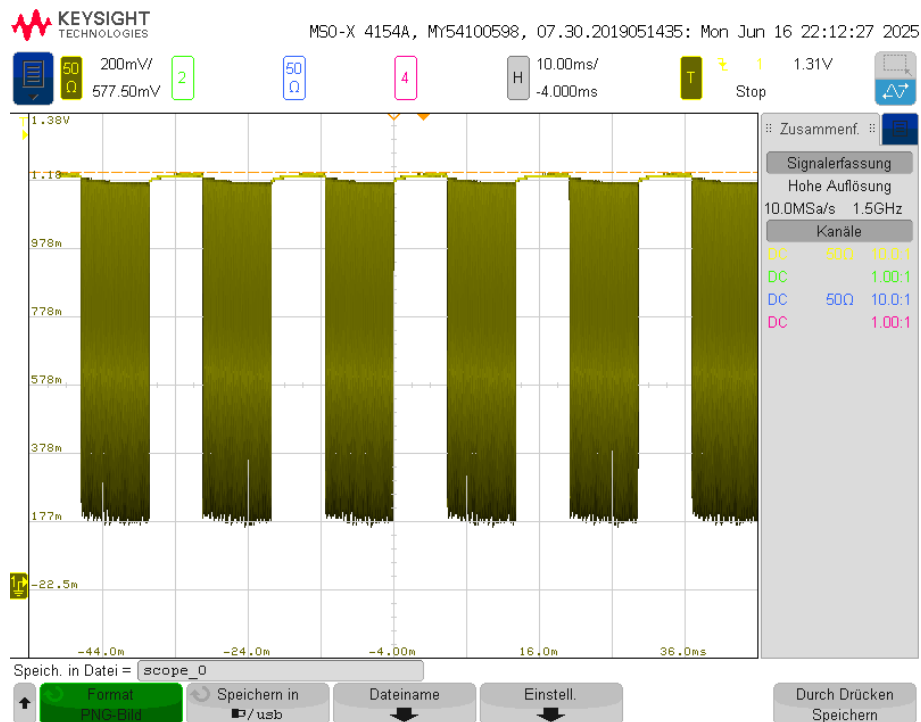


Figure 80: 100ms sample of D-PHY data lane activity

The valid data signal “csi_raw_valid” from the receiver was measured to analyse whether the data is correctly received. The resulting zoomed out waveform is shown in Figure 81, where several frame transmissions can be identified. As expected, the lengths of the active phases and pauses in between match those of the incoming data stream, indicating that the reception works properly.

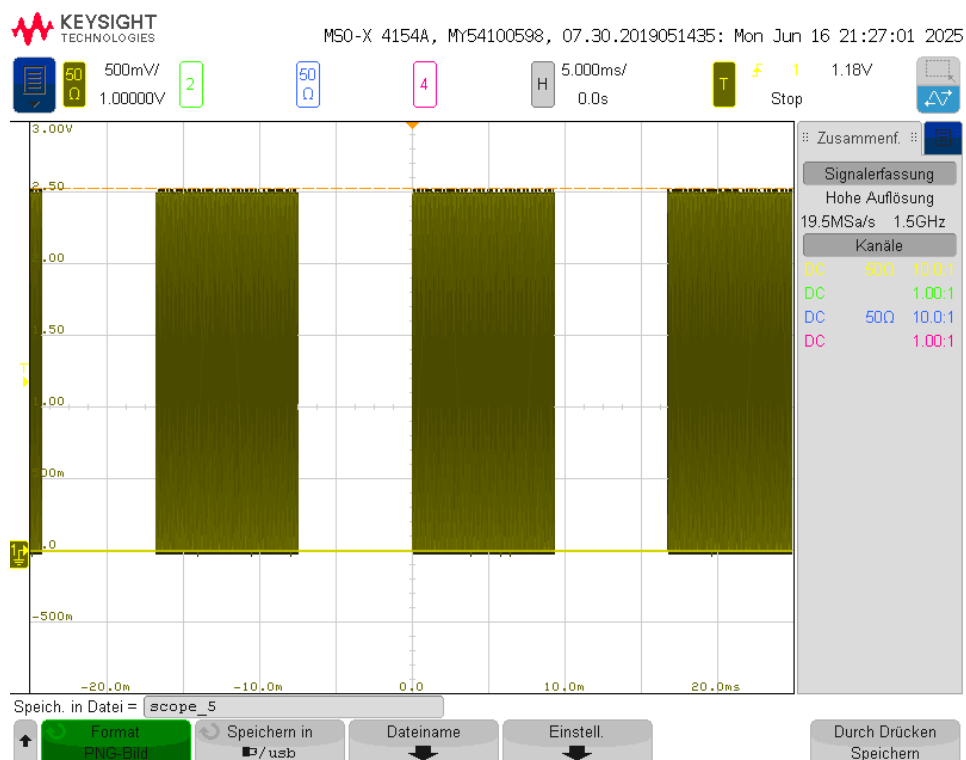


Figure 81: 50ms sample of the “csi_raw_valid” signal

For a more thorough verification of the reception quality, the CRC implemented in the receiver logic was evaluated. This was done by monitoring the activity of the counters for matching and non-matching checksums. As no thorough quantitative analysis was planned, the counters are designed so that they will overflow back to 0 after reaching the maximum possible value. Additionally, a signal is used that permanently latches to HIGH as soon as one non-matching checksum is detected. During testing it was found that no non-matching checksum was detected over a period of several hours, whereas the matching checksum counter reliably incremented with every received packet. Non-matching checksums only occurred when faults were deliberately created, e.g. by deliberately configuring the LFSR wrong, tampering with the transmission lines between camera and FPGA or changing the core supply voltage in operation. This corroborates the impression of a reliable operation of the receiver with a low error rate, although a quantitative estimate cannot be done based on the performed tests.

Finally, the waveforms of the TMDS link were examined, to verify the timing of the output signals and the voltage swing. The latter was done using the clock signal, as there is little distortion of the waveforms from imperfect DC-balancing, as it was observed on the data signals. The levels of the clock signal on both sides of the coupling capacitors are shown in Figure 82 and Figure 83. The comparison shows that the voltage swing is approximately 600mV, as expected, with the DC-levels around 1.3V on the LVDS side and around 3.3V on the TMDS. The period of the clock signal is almost exactly 20ns, which corresponds to the intended frequency of 25MHz.

In Figure 84 the waveforms of the TMDS clock and the data of the green channel is shown. Although the correctness of the data signal could not be verified with this measurement, the oscillogram shows that both signals are aligned with the 250MHz bit clock without problematic skew. As mentioned, in section 5.2.2, the edges of the transmitted pixel clock do not need to be aligned with the word boundaries on the data lines.

A look at the DC-level of the data signal reveals that there appears to be some drift. This effect was observed on all data lines but did not appear to cause any issues, as the upward and downward drift of the DC-level would compensate each other over time. This is most likely caused by the imperfect DC-balancing of the TMDS protocol, which is only DC-balanced over longer timeframes than shown in Figure 84.

The overall results of the final hardware tests are positive and indicate that the system works as intended and can run reliably over several hours under laboratory conditions. For a more detailed characterization of the performance of the demonstrator or any of its sub-systems, however, more extensive testing is required. On one hand, the long-term stability of the system in continuous operation, the effects of changes in ambient temperature and the resistance to electromagnetic interference have not been evaluated yet. On the other hand, the maximum D-PHY clock frequency the receiver and demonstrator can work with has not been investigated, as all tests so far have been performed with a 200MHz D-PHY clock. Both of these aspects, reliability and maximum operating frequency, are important because they may limit the range of possible applications.

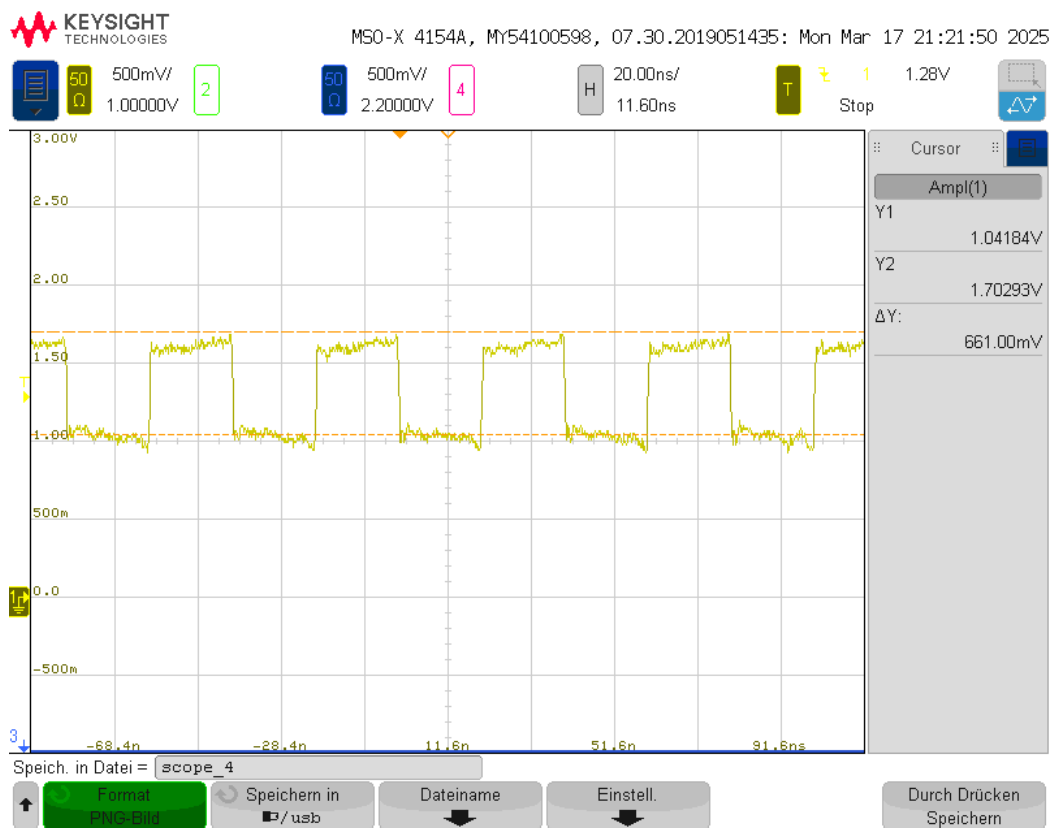


Figure 82: Pixel clock signal of the DVI output on the LVDS side

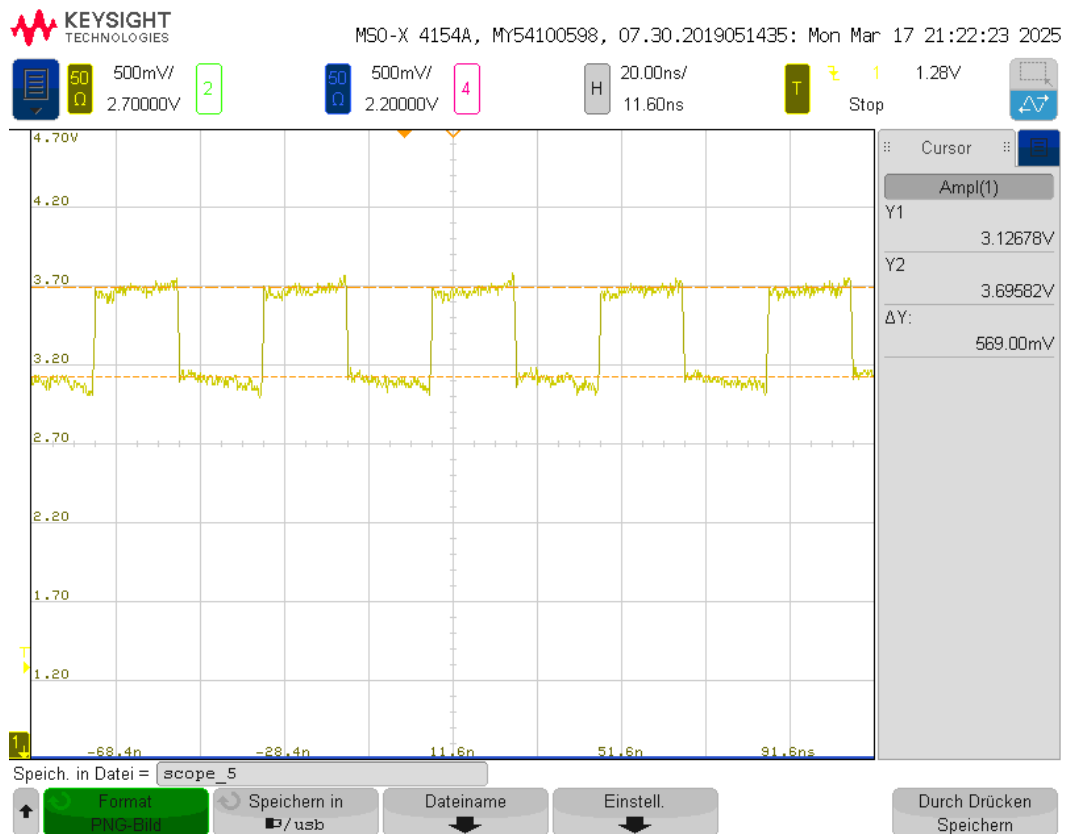


Figure 83: Pixel clock signal of the DVI output on the TMDS side

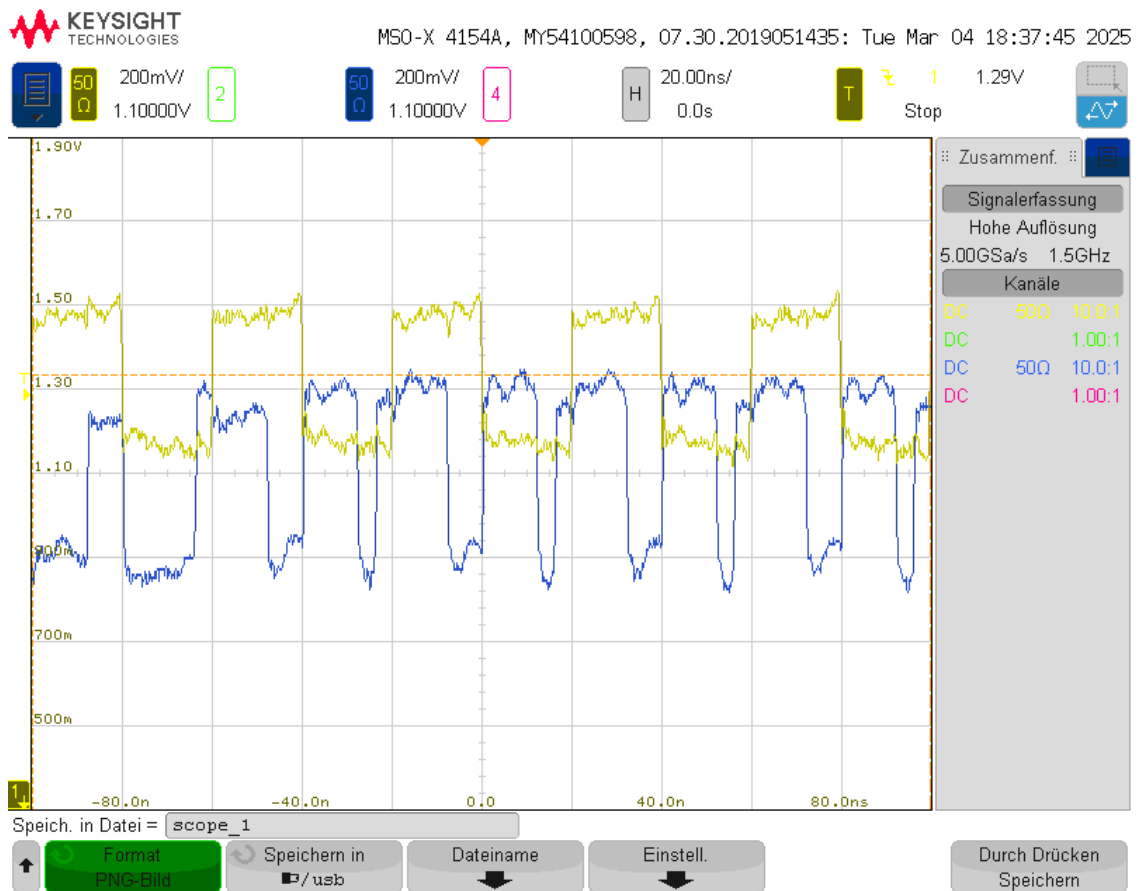


Figure 84: Pixel clock and red channel data signal of the DVI output on the LVDS side

6 Conclusion and Outlook

In this project a lightweight, CSI-2 compatible receiver, and an accompanying demonstrator system were designed in Verilog and implemented on the GateMate FPGA. Additionally, a dedicated evaluation board for the GateMate was designed and assembled. The receiver is designed specifically for compatibility with the Raspberry Pi Camera Version 2, delivering 8-bit raw image data via a two-lane CSI-2 connection using the D-PHY physical layer. To keep the system simple and minimize the required resources, much of this configuration is hardcoded in Verilog.

During the design and testing of the demonstrator, no major design flaws could be detected in the evaluation board. Problems during assembly of the first prototypes arose from the author's lack of dexterity. However, some potential for improvement was found, the most notable example being the solder jumpers for selecting the configuration mode. It was found that, although very compact, these simply do not offer the convenience and flexibility of plug-in jumpers. Another minor issue is that the GPIO connections, that are supposed to be PMOD-compatible (to some extent), are oriented in the wrong direction, so that 90° angle connectors cannot be used. Further possible improvements include additional test points on the board, for supply voltages and data lines, and an improved silkscreen, that provides more information on test points and other components. An improved version of the board may also include better routing for the GPIO connections and connections for additional I/O-ports, that are not used in the current design. Lastly, changing the mini-USB power connector to a more widely used standard, like USB-C, could be more convenient for the user.

The receiver design, like that of the rest of the demonstrator system, is specifically optimized for the GateMate FPGA, in order to achieve the best possible performance with this low-cost, general-purpose FPGA. An important aspect of this optimization is the omission of all unnecessary functionality, crucially including the LP-signals of the D-PHY link. The use of two different signal types on the same pair of wires normally requires either dedicated I/O buffers, or two pairs of I/O pins, as is usual for implementations of D-PHY on devices without dedicated I/O buffers for D-PHY [10]. Eliminating the need for this extra pair of I/O pins for each lane not only halves the number of required I/O pins for the interface (here from 12 to 6) but also simplifies the design of the connections between the FPGA and the camera connector on a PCB such as the evaluation board designed in this project. The receiver and the demonstrator system were successfully tested with the lowest clock frequencies and image resolutions possible with the camera module and other hardware. Whereas the video format of 640*480 pixels with a 60Hz refresh rate and 24-bit colour depth and the corresponding pixel frequency of 25MHz is imposed by the HDMI/DVI specifications, the camera module requires a minimum clock frequency of 200MHz for the CSI-2 interface.

It could be demonstrated that the GateMate can be used to receive and process video data and output it via a DVI interface. The full demonstrator system presented in this work uses roughly 16% of the programmable elements in the FPGA and requires a minimum of 17 I/O pins, leaving a majority of the GateMate's resources available for other functionality. On the other hand, the on-chip memory of the GateMate was found to be a significant bottleneck for the application, as it is insufficient to store a full frame of image data with a reasonable colour depth. This limitation could be overcome by using external memory, however. While the demonstrator designed in this project by itself is of little use beyond its intended purpose as testbench, its components, first and foremost the CSI-2 receiver, can serve as a starting point for further video related projects using the GateMate FPGA.

This utility could be greatly enhanced by further optimization and extensive parameterization of the constituent modules, to facilitate the adaptation to other applications. Furthermore, more extensive testing is required to determine the maximum possible clock frequency at which the components, primarily the CSI-2 receiver and the DVI-output can operate.

7 References

- [1] HDMI Founders, “High Definition Multimedia Interface Specification Version 1.3,” 2006.
- [2] Mathworks, “Configure Blanking Intervals (Mathworks.com),” [Online]. Available: <https://de.mathworks.com/help/visionhdl/ug/configure-blanking-intervals.html>. [Accessed 20 June 2025].
- [3] Consumer technology Association (CTA), “ANSI/CTA-861-I,” 2023.
- [4] Digital Display Working Group, “Digital Visual Interface DVI,” DDWG Promoters, 1999.
- [5] Sensoray, “Color Spaces in Frame Grabbers: RGB vs. YUV,” [Online]. Available: https://www.sensoray.com/support/appnotes/frame_grabber_capture_modes.htm. [Accessed 20 June 2025].
- [6] “Y’UV - Wikipedia,” [Online]. Available: <https://en.wikipedia.org/wiki/Y%E2%80%B2UV>. [Accessed 20 June 2025].
- [7] MIPI Alliance, “Specification for Camera Serial Interface 2 (CSI-2) Version 1.3,” 2014.
- [8] MIPI Alliance, “Specification for D-PHY,” 2014.
- [9] Sony, “IMX219PQH5-C Datasheet”.
- [10] Xilinx, “Application Note: Spartan-6 and 7 Series FPGAs - D-PHY Solutions,” 2021.
- [11] Texas Instruments, “The Many Flavors of LVDS,” 2011.
- [12] J. P. Nicolle, “fga4fun,” [Online]. Available: <https://www.fpga4fun.com/HDMI.html>. [Accessed 7 February 2025].
- [13] “Farbwahrnehmung - Wikipedia,” [Online]. Available: <https://de.wikipedia.org/wiki/Farbwahrnehmung>. [Accessed 20 June 2025].
- [14] Raspberry Pi, “Raspberry Pi Camera V2.1 Schematic,” 2018.
- [15] ArduCam, “Raspberry Pi Camera Pinout,” [Online]. Available: <https://www.arducam.com/raspberry-pi-camera-pinout/>. [Accessed 7 February 2025].
- [16] Vision Components, “Hardware specifications of VC MIPI Camera Modules,” [Online]. Available: https://www.vision-components.com/fileadmin/external/documentation/hardware/VC_MIPI_Camera_Modul/index.html#flexible-printed-circuit-fpc-cable. [Accessed 7 February 2025].

- [17] CologneChip AG, "GateMate FPGA Datasheet," 2024.
- [18] AMD, "AMD Vivado Design Suite," [Online]. Available: <https://www.amd.com/de/products/software/adaptive-socs-and-fpgas/vivado.html>. [Accessed 7 Februar 2025].
- [19] Cologne Chip AG, "Gatemate FPGA Programmer Datasheet," 2023.
- [20] Cologne Chip AG, "GateMate FPGA Evaluation Board," [Online]. Available: <https://colognechip.com/programmable-logic/gatemate-evaluation-board/>. [Accessed 7 February 2025].
- [21] CologneChip AG, "GateMate FPGA Evaluation Board Datasheet Version 3.2," 2024.
- [22] Monolithic Power Systems, "MPM3833C Datasheet," 2018.
- [23] Texas Instruments, "Decoupling capacitors (TI Precision Lab)," 2022.
- [24] AMD, "UltraScale Architecture SelectIO Resources," 2025.
- [25] Digilent Inc., "Digilent Pmod™ Interface Specification 1.2.0," 2017.
- [26] Eurocircuits, "Selecting the right Surface Finish for your PCB," [Online]. Available: <https://www.eurocircuits.com/blog/which-surface-finish-fits-your-design/>. [Accessed 7 February 2025].
- [27] Altium, "How to Design to a Differential Impedance Specification," [Online]. Available: <https://resources.altium.com/p/how-design-differential-impedance-specification>. [Accessed 7 February 2025].
- [28] Chili.CHIPS*ba, "openeye-CamSI (github)," [Online]. Available: https://github.com/chili-chips-ba/openeye-CamSI/tree/main/1.hw/csi_rx. [Accessed 20 June 2025].
- [29] D. S. Arar, "Introduction to Clock Domain Crossing: Double Flopping," [Online]. Available: <https://www.allaboutcircuits.com/technical-articles/introduction-to-clock-domain-crossing-double-flopping/>. [Accessed 7 February 2025].
- [30] A. Forencich, "Verilog LFSR (github)," [Online]. Available: <https://github.com/alexforencich/verilog-lfsr?tab=readme-ov-file>. [Accessed 20 June 2025].
- [31] A. Isagholian, "Tiny_But_Mighty_I2C_Master_Verilog (github)," [Online]. Available: https://github.com/0xArt/Tiny_But_Mighty_I2C_Master_Verilog?tab=readme-ov-file. [Accessed 20 June 2025].
- [32] NXP Semiconductors, "I2C-bus specification and user manual - Rev. 7.0," 2021.
- [33] G. Singh, "USB_C_Industrial_Camera_FPGA_USB3 (github)," [Online]. Available: https://github.com/circuitvalley/USB_C_Industrial_Camera_FPGA_USB3. [Accessed 20 June 2025].
- [34] Cologne Chip AG, "GateMate FPGA User Guide Primitives Library," 2025.

- [35] G. Goavec-Merou, “GateMate_demos (github),” [Online]. Available: https://github.com/trabucayre/GateMate_demos. [Accessed 20 June 2025].
- [36] Cologne Chip, “Integrated logic analyser (ILA) for the GateMate FPGA from Cologne Chip (github),” [Online]. Available: https://github.com/colognechip/gatemate_ila. [Accessed 7 February 2025].
- [37] Marconix International, „MX25V8035F Datasheet,“ 2015.

8 Appendix

The HDL code and the design files of the evaluation board can be found at:
<https://github.com/FHDO-ICLAB/MIPI-CSI-2-HDMI-Bridge>